

C Language Introduction

What is C Language

The C Language is developed by Dennis Ritchie for creating system applications that directly interact with the hardware devices such as drivers, kernels, etc.

C programming is considered as the base for other programming languages, that is why it is known as mother language.

It can be defined by the following ways:

1. Mother language
2. System programming language
3. Procedure-oriented programming language
4. Structured programming language
5. Mid-level programming language

1) C as a mother language

C language is considered as the mother language of all the modern programming languages because **most of the compilers, JVMs, Kernels, etc. are written in C language**, and most of the programming languages follow C syntax, for example, C++, Java, C#, etc.

It provides the core concepts like the array, strings, functions, file handling, etc. that are being used in many languages like C++, Java, C#, etc.

2) C as a system programming language

A system programming language is used to create system software. C language is a system programming language because it **can be used to do low-level programming (for example driver and kernel)**. It is generally used to create hardware devices, OS, drivers, kernels, etc. For example, Linux kernel is written in C.

It can't be used for internet programming like Java, .Net, PHP, etc.

3) C as a procedural language

A procedure is known as a function, method, routine, subroutine, etc. A procedural language **specifies a series of steps for the program to solve the problem**.

A procedural language breaks the program into functions, data structures, etc.

C is a procedural language. In C, variables and function prototypes must be declared before being used.

4) C as a structured programming language

A structured programming language is a subset of the procedural language. **Structure means to break a program into parts or blocks** so that it may be easy to understand.

In the C language, we break the program into parts using functions. It makes the program easier to understand and modify.

5) C as a mid-level programming language

C is considered as a middle-level language because it **supports the feature of both low-level and high-level languages**. C language program is converted into assembly code, it supports pointer arithmetic (low-level), but it is machine independent (a feature of high-level).

A **Low-level language** is specific to one machine, i.e., machine dependent. It is machine dependent, fast to run. But it is not easy to understand.

A **High-Level language** is not specific to one machine, i.e., machine independent. It is easy to understand.

C Program

In this tutorial, all C programs are given with C compiler so that you can quickly change the C program code.

File: main.c

```
#include <stdio.h>

int main() {

    printf("Hello C Programming\n");

    return 0;

}
```

A detailed description of above program is given in next chapters.

History of C

History of C language is interesting to know. Here we are going to discuss a brief history of the c language.

C programming language was developed in 1972 by Dennis Ritchie at bell laboratories of AT&T (American Telephone & Telegraph), located in the U.S.A.



Dennis Ritchie is known as the **founder of the c language**.

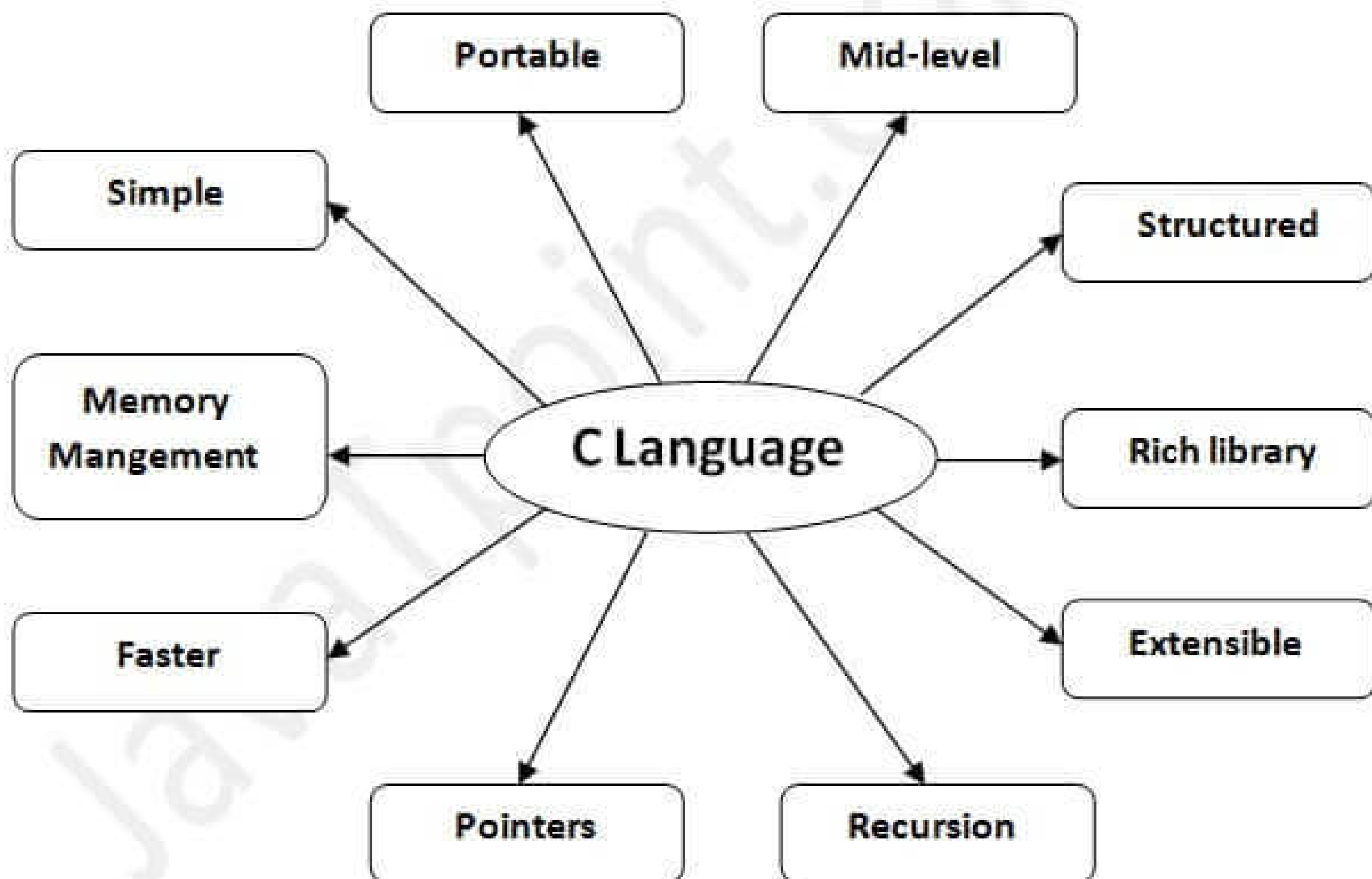
It was developed to overcome the problems of previous languages such as B, BCPL, etc.

Initially, C language was developed to be used in **UNIX operating system**. It inherits many features of previous languages such as B and BCPL.

Let's see the programming languages that were developed before C language.

Language	Year	Developed By
Algol	1960	International Group
BCPL	1967	Martin Richard
B	1970	Ken Thompson
Traditional C	1972	Dennis Ritchie
K & R C	1978	Kernighan & Dennis Ritchie
ANSI C	1989	ANSI Committee
ANSI/ISO C	1990	ISO Committee
C99	1999	Standardization Committee

Features of C



JavaTpoint.com

C is the widely used language. It provides many **features** that are given below.

1. Simple
2. Machine Independent or Portable
3. Mid-level programming language
4. structured programming language
5. Rich Library
6. Memory Management
7. Fast Speed
8. Pointers

9. Recursion

10. Extensible

1) Simple

C is a simple language in the sense that it provides a **structured approach** (to break the problem into parts), **the rich set of library functions, data types**, etc.

2) Machine Independent or Portable

Unlike assembly language, c programs **can be executed on different machines** with some machine specific changes. Therefore, C is a machine independent language.

3) Mid-level programming language

Although, C is **intended to do low-level programming**. It is used to develop system applications such as kernel, driver, etc. It **also supports the features of a high-level language**. That is why it is known as mid-level language.

4) Structured programming language

C is a structured programming language in the sense that **we can break the program into parts using functions**. So, it is easy to understand and modify. Functions also provide code reusability.

5) Rich Library

C **provides a lot of inbuilt functions** that make the development fast.

6) Memory Management

It supports the feature of **dynamic memory allocation**. In C language, we can free the allocated memory at any time by calling the **free()** function.

7) Speed

The compilation and execution time of C language is fast since there are lesser inbuilt functions and hence the lesser overhead.

8) Pointer

C provides the feature of pointers. We can directly interact with the memory by using the pointers. We can use pointers for memory, structures, functions, array, etc.

9) Recursion

In C, we can call the function within the function. It provides code reusability for every function. Recursion enables us to use the approach of backtracking.

10) Extensible

C language is extensible because it can easily adopt new features.

First C Program

Before starting the abcd of C language, you need to learn how to write, compile and run the first c program.

To write the first c program, open the C console and write the following code:

```
#include <stdio.h>

int main(){

printf("Hello C Language");

return 0;

}
```

#include <stdio.h> includes the standard input output library functions. The **printf()** function is defined in **stdio.h**.

int main() The **main()** function is the entry point of every program in c language.

printf() The **printf()** function is used to print data on the console.

return 0 The return 0 statement, returns execution status to the OS. The 0 value is used for successful execution and 1 for unsuccessful execution.

How to compile and run the c program

There are 2 ways to compile and run the c program, by menu and by shortcut.

By menu

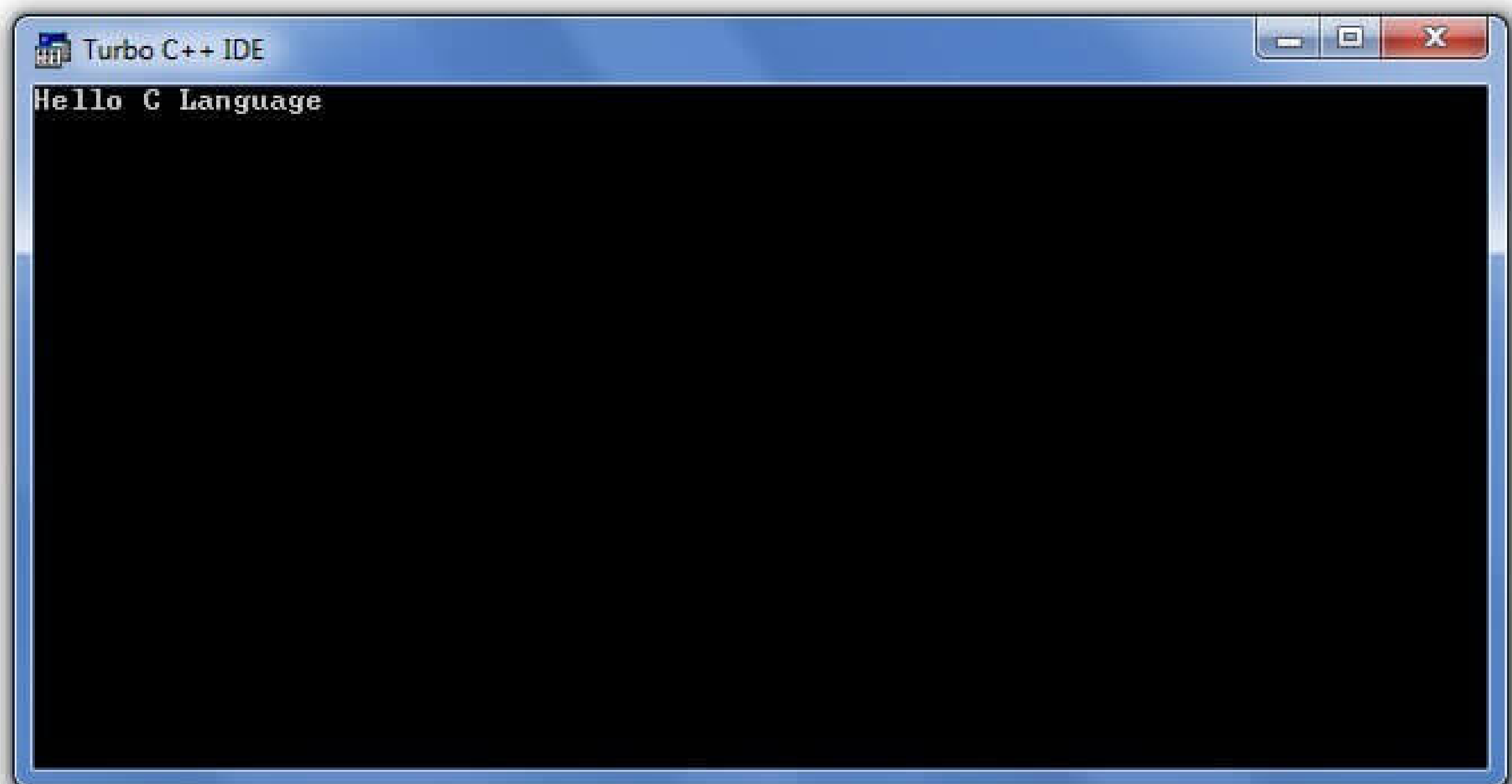
Now click on the compile menu then compile sub menu to compile the c program.

Then click on the run menu then run sub menu to run the c program.

By shortcut

Or, press **ctrl+f9** keys compile and run the program directly.

You will see the following output on user screen.



You can view the user screen any time by pressing the **alt+f5** keys.

Now press **Esc** to return to the turbo c++ console.

Flow of C Program

The C program follows many steps in execution. To understand the flow of C program well, let us see a simple program first.

File: simple.c

```
#include <stdio.h>

int main(){

printf("Hello C Language");

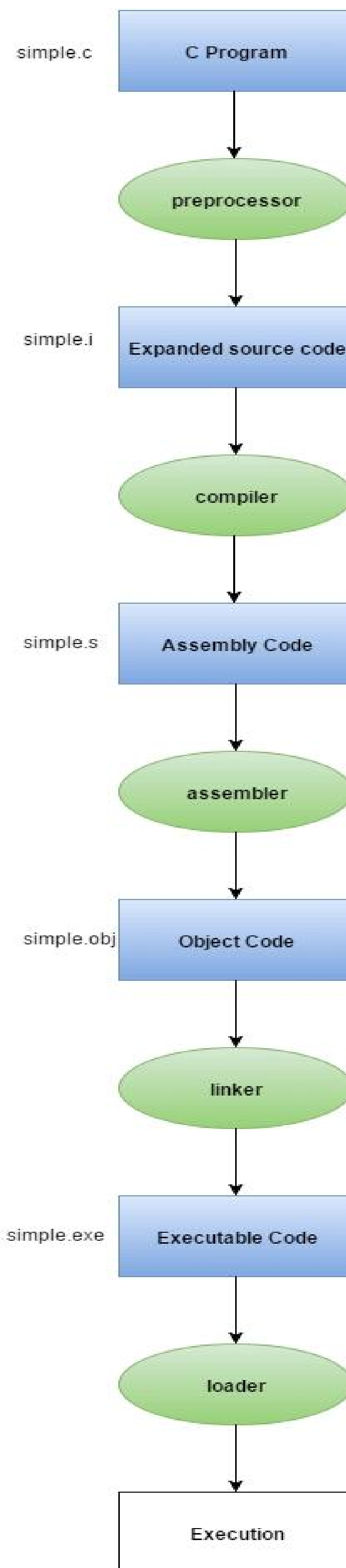
return 0;

}
```

Execution Flow

Let's try to understand the flow of above program by the figure given below.

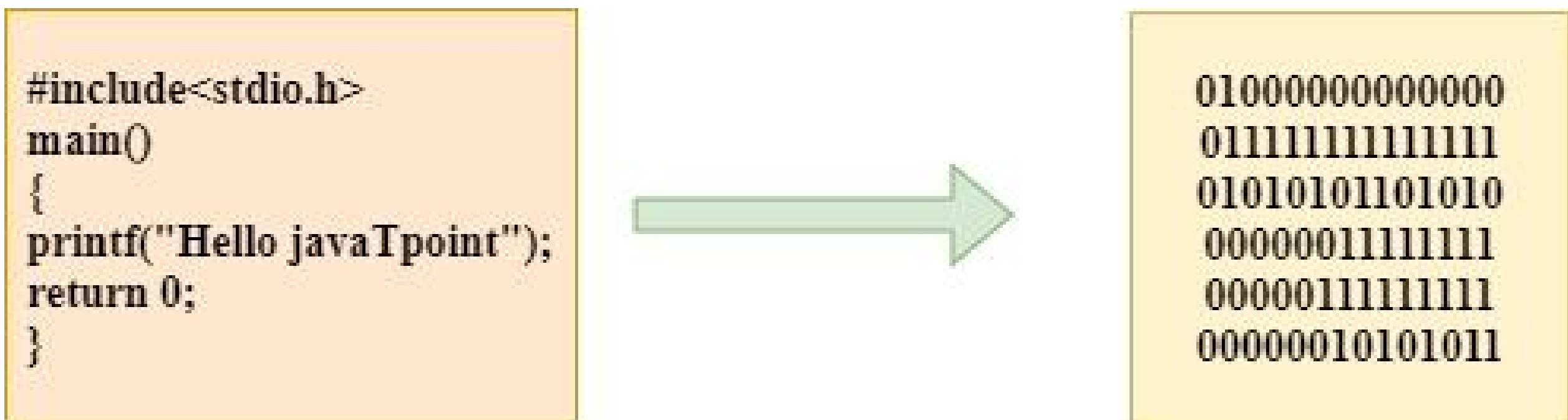
- 1) C program (source code) is sent to preprocessor first. The preprocessor is responsible to convert preprocessor directives into their respective values. The preprocessor generates an expanded source code.
- 2) Expanded source code is sent to compiler which compiles the code and converts it into assembly code.
- 3) The assembly code is sent to assembler which assembles the code and converts it into object code. Now a simple.obj file is generated.
- 4) The object code is sent to linker which links it to the library such as header files. Then it is converted into executable code. A simple.exe file is generated.
- 5) The executable code is sent to loader which loads it into memory and then it is executed. After execution, output is sent to console.



Compilation process in c

What is a compilation?

The compilation is a process of converting the source code into object code. It is done with the help of the compiler. The compiler checks the source code for the syntactical or structural errors, and if the source code is error-free, then it generates the object code.

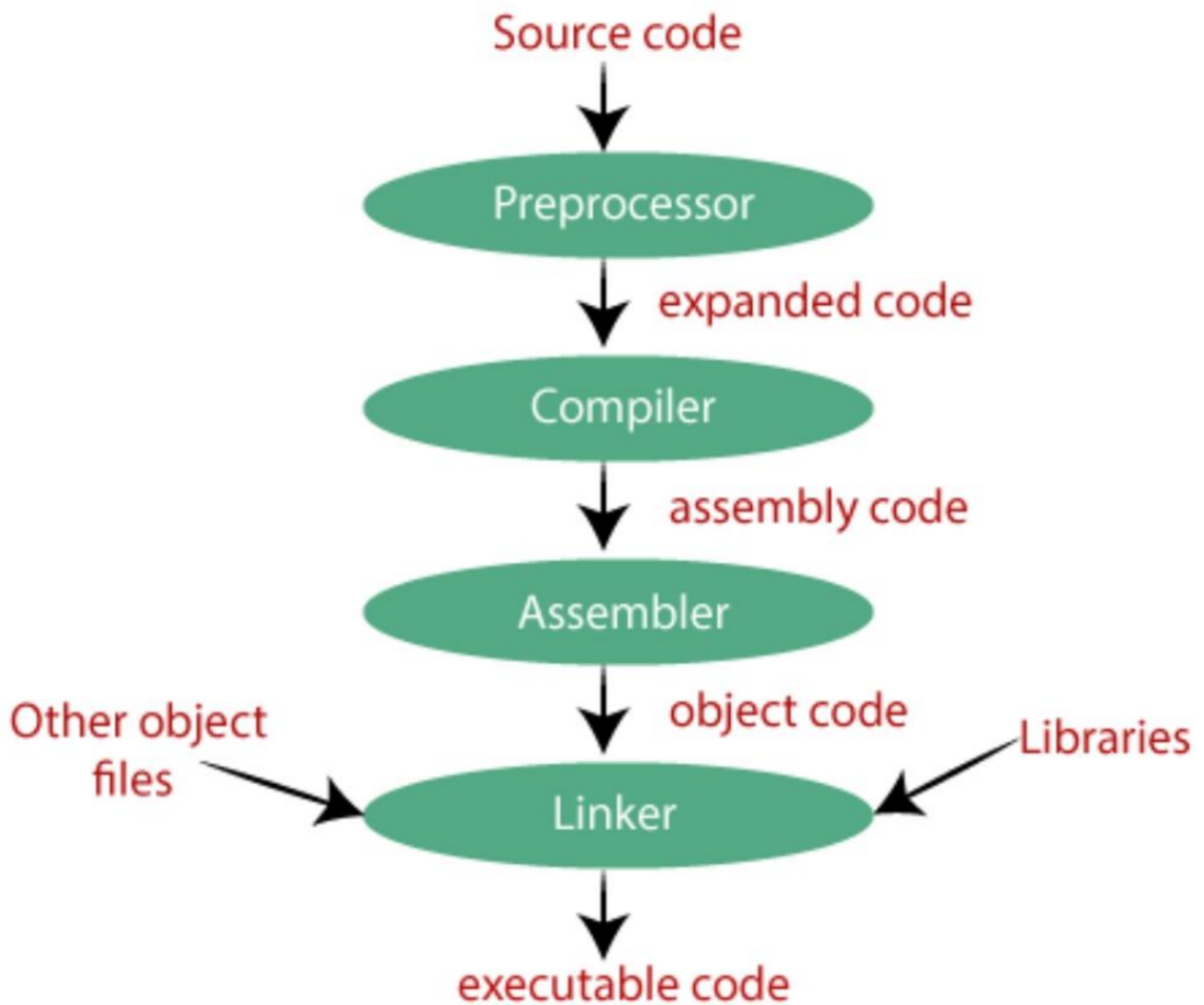


The c compilation process converts the source code taken as input into the object code or machine code. The compilation process can be divided into four steps, i.e., Pre-processing, Compiling, Assembling, and Linking.

The preprocessor takes the source code as an input, and it removes all the comments from the source code. The preprocessor takes the preprocessor directive and interprets it. For example, if `<stdio.h>`, the directive is available in the program, then the preprocessor interprets the directive and replace this directive with the content of the 'stdio.h' file.

The following are the phases through which our program passes before being transformed into an executable form:

- Preprocessor
- Compiler
- Assembler
- Linker



Preprocessor

The source code is the code which is written in a text editor and the source code file is given an extension ".c". This source code is first passed to the preprocessor, and then the preprocessor expands this code. After expanding the code, the expanded code is passed to the compiler.

Compiler

The code which is expanded by the preprocessor is passed to the compiler. The compiler converts this code into assembly code. Or we can say that the C compiler converts the pre-processed code into assembly code.

Assembler

The assembly code is converted into object code by using an assembler. The name of the object file generated by the assembler is the same as the source file. The extension of the object file in DOS is '.obj,' and in UNIX, the extension is 'o'. If the name of the source file is 'hello.c', then the name of the object file would be 'hello.obj'.

Linker

Mainly, all the programs written in C use library functions. These library functions are pre-compiled, and the object code of these library files is stored with '.lib' (or '.a') extension. The main working of the linker is to combine the object code of library files with the object code of our program. Sometimes the situation arises when our program refers to the functions defined in other files; then linker plays a very important role in this. It links the object code of these files to our program. Therefore, we conclude that the job of the linker is to link the object code of our program with the object code of the library files and other files. The output of the linker is the executable file. The name of the executable file is the same as the source file but differs only in their extensions. In DOS, the extension of the executable file is '.exe', and in UNIX, the executable file can be named as 'a.out'. For example, if we are using printf() function in a program, then the linker adds its associated code in an output file.

Let's understand through an example.

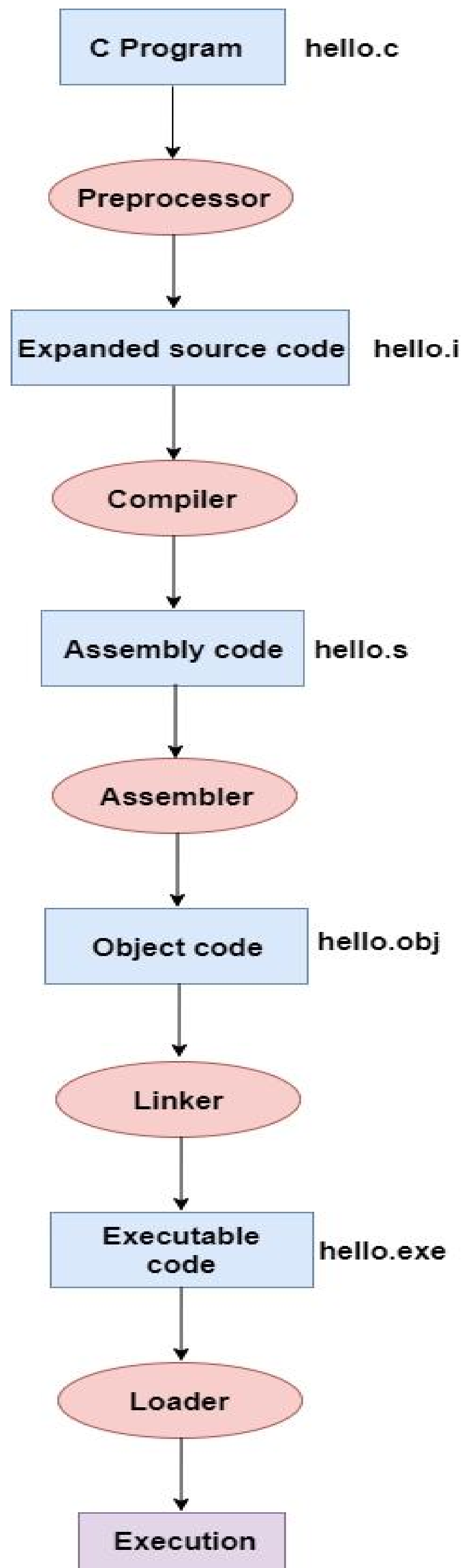
hello.c

```
#include <stdio.h>

int main()
{
    printf("Hello javaTpoint");

    return 0;
}
```

Now, we will create a flow diagram of the above program:



In the above flow diagram, the following steps are taken to execute a program:

- Firstly, the input file, i.e., **hello.c**, is passed to the preprocessor, and the preprocessor converts the source code into expanded source code. The extension of the expanded source code would be **hello.i**.
- The expanded source code is passed to the compiler, and the compiler converts this expanded source code into assembly code. The extension of the assembly code would be **hello.s**.
- This assembly code is then sent to the assembler, which converts the assembly code into object code.
- After the creation of an object code, the linker creates the executable file. The loader will then load the executable file for the execution.

printf() and scanf() in C

The `printf()` and `scanf()` functions are used for input and output in C language. Both functions are inbuilt library functions, defined in `stdio.h` (header file).

printf() function

The `printf()` function is used for output. It prints the given statement to the console.

The syntax of `printf()` function is given below:

```
printf("format string",argument_list);
```

The **format string** can be `%d` (integer), `%c` (character), `%s` (string), `%f` (float) etc.

scanf() function

The `scanf()` function is used for input. It reads the input data from the console.

```
scanf("format string",argument_list);
```

Program to print cube of given number

Let's see a simple example of c language that gets input from the user and prints the cube of the given number.

```
#include<stdio.h>

int main(){

int number;

printf("enter a number:");

scanf("%d",&number);

printf("cube of number is:%d ",number*number*number);

return 0;

}
```

Output

```
enter a number:5

cube of number is:125
```

The `scanf("%d",&number)` statement reads integer number from the console and stores the given value in number variable.

The `printf("cube of number is: %d ",number*number*number)` statement prints the cube of number on the console.

Program to print sum of 2 numbers

Let's see a simple example of input and output in C language that prints addition of 2 numbers.

```
#include<stdio.h>

int main(){

int x=0,y=0,result=0;

printf("enter first number:");

scanf("%d",&x);
```

```

printf("enter second number:");

scanf("%d",&y);

result=x+y;

printf("sum of 2 numbers:%d ",result);

return 0;

}

```

Output

```

enter first number:9
enter second number:9
sum of 2 numbers:18

```

Variables in C

A **variable** is a name of the memory location. It is used to store data. Its value can be changed, and it can be reused many times.

It is a way to represent memory location through symbol so that it can be easily identified.

Let's see the syntax to declare a variable:

```
type variable_list;
```

The example of declaring the variable is given below:

- i. `int a;`
- ii. `float b;`
- iii. `char c;`

Here, a, b, c are variables. The int, float, char are the data types.

We can also provide values while declaring the variables as given below:

- i. `int a=10,b=20; //declaring 2 variable of integer type`

ii. `float f=20.8;`

iii. `char c='A';`

Rules for defining variables

- A variable can have alphabets, digits, and underscore.
- A variable name can start with the alphabet, and underscore only. It can't start with a digit.
- No whitespace is allowed within the variable name.
- A variable name must not be any reserved word or keyword, e.g. int, float, etc.

Valid variable names:

i. `int a;`

ii. `int _ab;`

iii. `int a30;`

Invalid variable names:

i. `int 2;`

ii. `int a b;`

iii. `int long;`

Types of Variables in C

There are many types of variables in c:

1. local variable
2. global variable
3. static variable
4. automatic variable

5. external variable

Local Variable

A variable that is declared inside the function or block is called a local variable.

It must be declared at the start of the block.

```
void function1(){  
  
    int x=10;//local variable  
  
}
```

You must have to initialize the local variable before it is used.

Global Variable

A variable that is declared outside the function or block is called a global variable. Any function can change the value of the global variable. It is available to all the functions.

It must be declared at the start of the block.

```
int value=20;//global variable  
  
void function1(){  
  
    int x=10;//local variable  
  
}
```

Static Variable

A variable that is declared with the static keyword is called static variable.

It retains its value between multiple function calls.

```
void function1(){  
  
    int x=10;//local variable  
  
    static int y=10;//static variable  
  
    x=x+1;
```

```

y=y+1;

printf("%d,%d",x,y);

}

```

If you call this function many times, the **local variable** will print the same value for each function call, e.g, 11,11,11 and so on. But the **static variable** will print the incremented value in each function call, e.g. 11, 12, 13 and so on.

Automatic Variable

All variables in C that are declared inside the block, are automatic variables by default.

We can explicitly declare an automatic variable using **auto keyword**.

```

void main(){

int x=10;//local variable (also automatic)

auto int y=20;//automatic variable

}

```

External Variable

We can share a variable in multiple C source files by using an external variable. To declare an external variable, you need to use **extern keyword**.

myfile.h

```

extern int x=10;//external variable (also global)

```

program1.c

```

#include "myfile.h"

#include <stdio.h>

void printValue(){

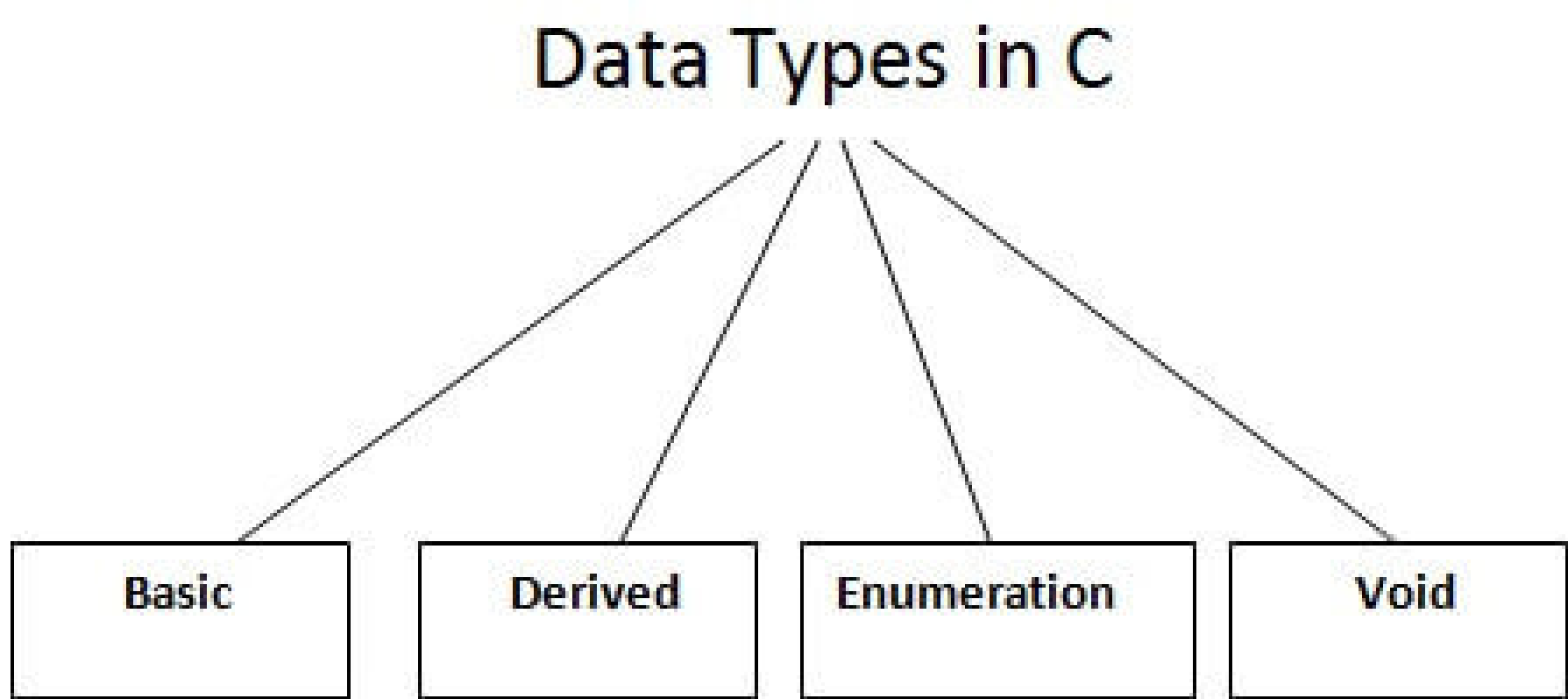
printf("Global variable: %d", global_variable);

}

```

Data Types in C

A data type specifies the type of data that a variable can store such as integer, floating, character, etc.



There are the following data types in C language.

Types	Data Types
Basic Data Type	int, char, float, double
Derived Data Type	array, pointer, structure, union
Enumeration Data Type	enum
Void Data Type	void

Basic Data Types

The basic data types are integer-based and floating-point based. C language supports both signed and unsigned literals.

The memory size of the basic data types may change according to 32 or 64-bit operating system.

Let's see the basic data types. Its size is given according to 32-bit architecture.

Data Types	Memory Size	Range
char	1 byte	-128 to 127
signed char	1 byte	-128 to 127
unsigned char	1 byte	0 to 255
short	2 byte	-32,768 to 32,767
signed short	2 byte	-32,768 to 32,767
unsigned short	2 byte	0 to 65,535
int	2 byte	-32,768 to 32,767
signed int	2 byte	-32,768 to 32,767
unsigned int	2 byte	0 to 65,535
short int	2 byte	-32,768 to 32,767
signed short int	2 byte	-32,768 to 32,767
unsigned short int	2 byte	0 to 65,535
long int	4 byte	-2,147,483,648 to 2,147,483,647

signed long int	4 byte	-2,147,483,648 to 2,147,483,647
unsigned long int	4 byte	0 to 4,294,967,295
float	4 byte	
double	8 byte	
long double	10 byte	

Keywords in C

A keyword is a **reserved word**. You cannot use it as a variable name, constant name, etc. There are only 32 reserved words (keywords) in the C language.

A list of 32 keywords in the c language is given below:

auto	break	case	char	const	continue	default	do
Double	else	enum	extern	float	for	goto	if
Int	long	register	return	short	signed	sizeof	static
struct	switch	typedef	union	unsigned	void	volatile	while

We will learn about all the C language keywords later.

C Identifiers

C identifiers represent the name in the C program, for example, variables, functions, arrays, structures, unions, labels, etc. An identifier can be composed of letters such as

uppercase, lowercase letters, underscore, digits, but the starting letter should be either an alphabet or an underscore. If the identifier is not used in the external linkage, then it is called as an internal identifier. If the identifier is used in the external linkage, then it is called as an external identifier.

We can say that an identifier is a collection of alphanumeric characters that begins either with an alphabetical character or an underscore, which are used to represent various programming elements such as variables, functions, arrays, structures, unions, labels, etc. There are 52 alphabetical characters (uppercase and lowercase), underscore character, and ten numerical digits (0-9) that represent the identifiers. There is a total of 63 alphanumeric characters that represent the identifiers.

Rules for constructing C identifiers

- The first character of an identifier should be either an alphabet or an underscore, and then it can be followed by any of the character, digit, or underscore.
- It should not begin with any numerical digit.
- In identifiers, both uppercase and lowercase letters are distinct. Therefore, we can say that identifiers are case sensitive.
- Commas or blank spaces cannot be specified within an identifier.
- Keywords cannot be represented as an identifier.
- The length of the identifiers should not be more than 31 characters.
- Identifiers should be written in such a way that it is meaningful, short, and easy to read.

Example of valid identifiers

- i. total, sum, average, _m_, sum_1, etc.

Example of invalid identifiers

- i. 2sum (starts with a numerical digit)
- ii. `int` (reserved word)
- iii. `char` (reserved word)
- iv. `m+n` (special character, i.e., '+')

Types of identifiers

- Internal identifier
- External identifier

Internal Identifier

If the identifier is not used in the external linkage, then it is known as an internal identifier. The internal identifiers can be local variables.

External Identifier

If the identifier is used in the external linkage, then it is known as an external identifier. The external identifiers can be function names, global variables.

Differences between Keyword and Identifier

Keyword	Identifier
Keyword is a pre-defined word.	The identifier is a user-defined word
It must be written in a lowercase letter.	It can be written in both lowercase and uppercase letters.
Its meaning is pre-defined in the c compiler.	Its meaning is not defined in the c compiler.

It is a combination of alphabetical characters.	It is a combination of alphanumeric characters.
It does not contain the underscore character.	It can contain the underscore character.

Let's understand through an example.

```
int main()
{
    int a=10;

    int A=20;

    printf("Value of a is : %d",a);

    printf("\nValue of A is :%d",A);

    return 0;
}
```

Output

```
Value of a is : 10

Value of A is :20
```

The above output shows that the values of both the variables, 'a' and 'A' are different. Therefore, we conclude that the identifiers are case sensitive.

C Operators

An operator is simply a symbol that is used to perform operations. There can be many types of operations like arithmetic, logical, bitwise, etc.

There are following types of operators to perform different types of operations in C language.

- Arithmetic Operators
- Relational Operators
- Shift Operators
- Logical Operators
- Bitwise Operators
- Ternary or Conditional Operators
- Assignment Operator
- Misc Operator

Precedence of Operators in C

The precedence of operator species that which operator will be evaluated first and next.

The associativity specifies the operator direction to be evaluated; it may be left to right or right to left.

Let's understand the precedence by the example given below:

```
int value=10+20*10;
```

The value variable will contain **210** because `*` (multiplicative operator) is evaluated before `+` (additive operator).

The precedence and associativity of C operators is given below:

Category	Operator	Associativity
Postfix	<code>() [] -> . ++ --</code>	Left to right
Unary	<code>+ - ! ~ ++ -- (type)*</code>	Right to left

	& sizeof	
Multiplicative	* / %	Left to right
Additive	+ -	Left to right
Shift	<< >>	Left to right
Relational	< <= > >=	Left to right
Equality	== !=	Left to right
Bitwise AND	&	Left to right
Bitwise XOR	^	Left to right
Bitwise OR		Left to right
Logical AND	&&	Left to right
Logical OR		Left to right
Conditional	?:	Right to left
Assignment	= += -= *= /= %>>= <<= &= ^= =	Right to left
Comma	,	Left to right

Comments in C

Comments in C language are used to provide information about lines of code. It is widely used for documenting code. There are 2 types of comments in the C language.

1. Single Line Comments
2. Multi-Line Comments

Single Line Comments

Single line comments are represented by double slash `//`. Let's see an example of a single line comment in C.

```
#include<stdio.h>

int main(){

    //printing information

    printf("Hello C");

    return 0;

}
```

Output:

Hello C

Even you can place the comment after the statement. For example:

```
printf("Hello C");//printing information
```

Multi Line Comments

Multi-Line comments are represented by slash asterisk `/* ... */`. It can occupy many lines of code, but it can't be nested. Syntax:

```
/*
code
*/
```

to be commented

*/

Let's see an example of a multi-Line comment in C.

```
#include<stdio.h>

int main(){

    /*printing information

    Multi-Line Comment*/

    printf("Hello C");

    return 0;

}
```

Output:

Hello C

C Format Specifier

The Format specifier is a string used in the formatted input and output functions. The format string determines the format of the input and output. The format string always starts with a '%' character.

The commonly used format specifiers in printf() function are:

Format specifier	Description
%d or %i	It is used to print the signed integer value where signed integer means that the variable can hold both positive and negative values.
%u	It is used to print the unsigned integer value where the unsigned

	integer means that the variable can hold only positive value.
%o	It is used to print the octal unsigned integer where octal integer value always starts with a 0 value.
%x	It is used to print the hexadecimal unsigned integer where the hexadecimal integer value always starts with a 0x value. In this, alphabetical characters are printed in small letters such as a, b, c, etc.
%X	It is used to print the hexadecimal unsigned integer, but %X prints the alphabetical characters in uppercase such as A, B, C, etc.
%f	It is used for printing the decimal floating-point values. By default, it prints the 6 values after '.'.
%e/%E	It is used for scientific notation. It is also known as Mantissa or Exponent.
%g	It is used to print the decimal floating-point values, and it uses the fixed precision, i.e., the value after the decimal in input would be exactly the same as the value in the output.
%p	It is used to print the address in a hexadecimal form.
%c	It is used to print the unsigned character.
%s	It is used to print the strings.

%ld	It is used to print the long-signed integer value.
-----	--

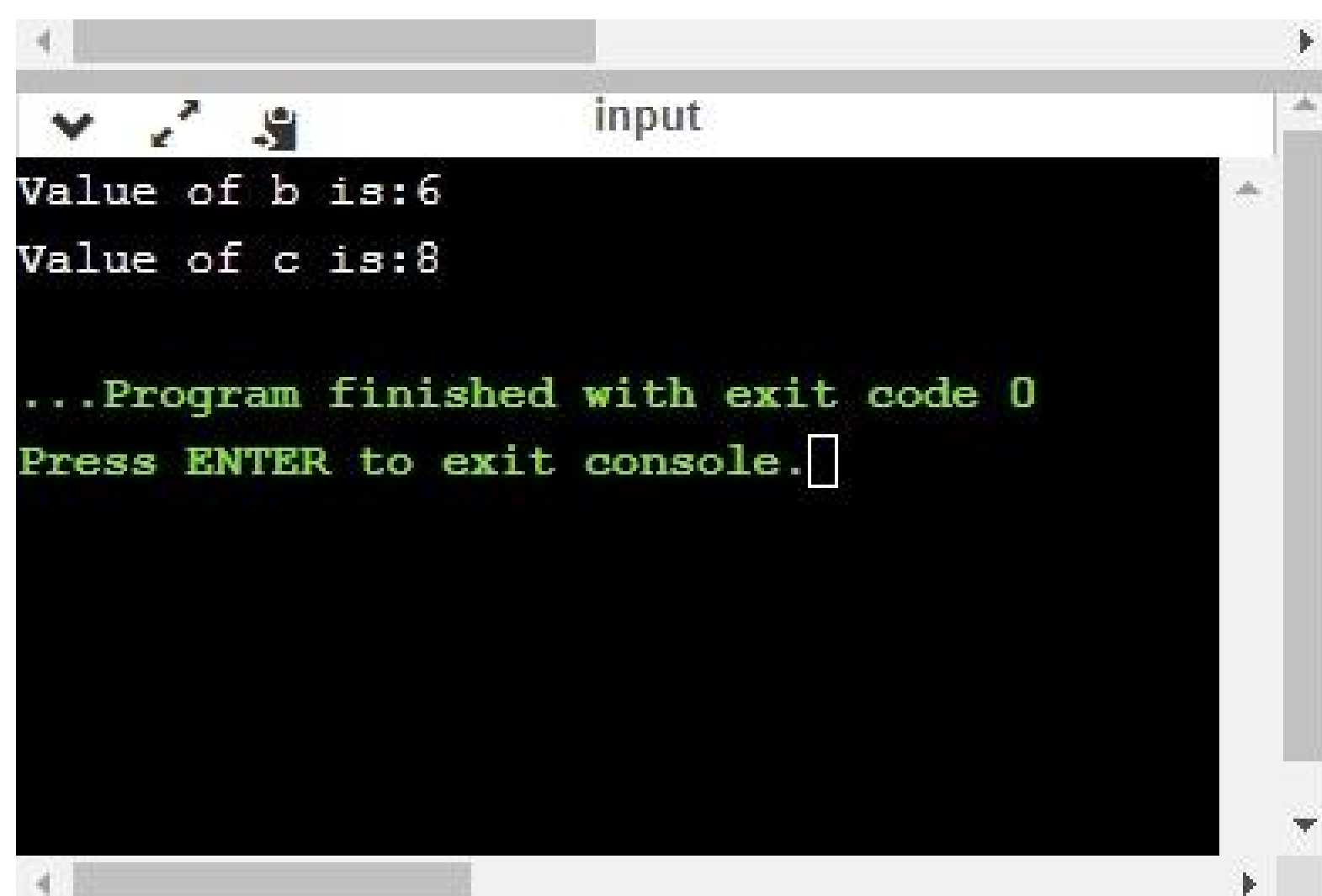
Let's understand the format specifiers in detail through an example.

%d

```
int main()  
{  
    int b=6;  
    int c=8;  
    printf("Value of b is:%d", b);  
    printf("\nValue of c is:%d",c);  
    return 0;  
}
```

In the above code, we are printing the integer value of b and c by using the %d specifier.

Output



```
input  
Value of b is:6  
Value of c is:8  
  
...Program finished with exit code 0  
Press ENTER to exit console.
```

%u

```
int main()
```

```

{

    int b=10;

    int c= -10;

    printf("Value of b is:%u", b);

    printf("\nValue of c is:%u",c);

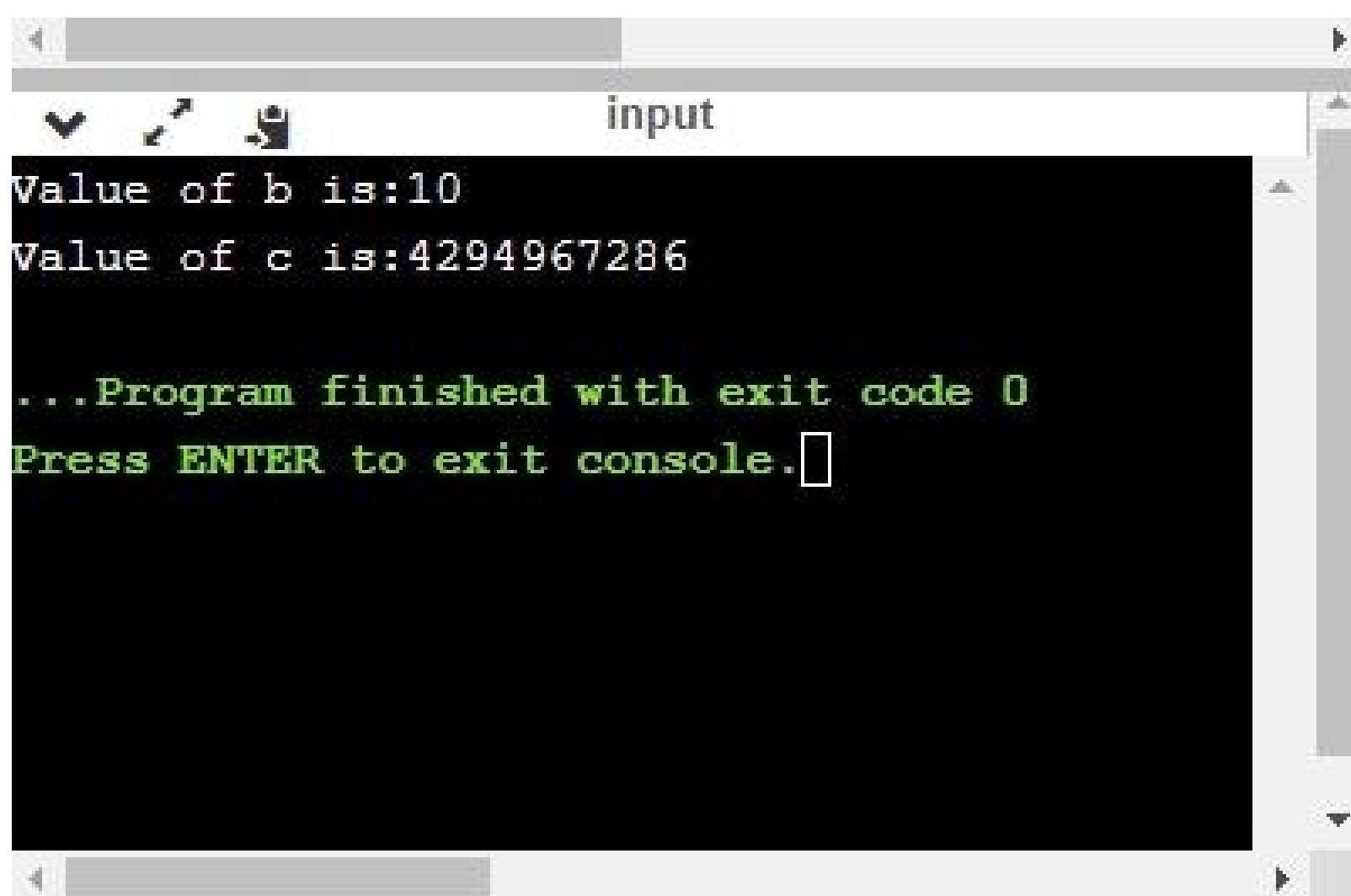
    return 0;

}

```

In the above program, we are displaying the value of b and c by using an unsigned format specifier, i.e., %u. The value of b is positive, so %u specifier prints the exact value of b, but it does not print the value of c as c contains the negative value.

Output



```

input
Value of b is:10
Value of c is:4294967286

...Program finished with exit code 0
Press ENTER to exit console.

```

%o

```

int main()

{

    int a=0100;

    printf("Octal value of a is: %o", a);

```

```

printf("\nInteger value of a is: %d",a);

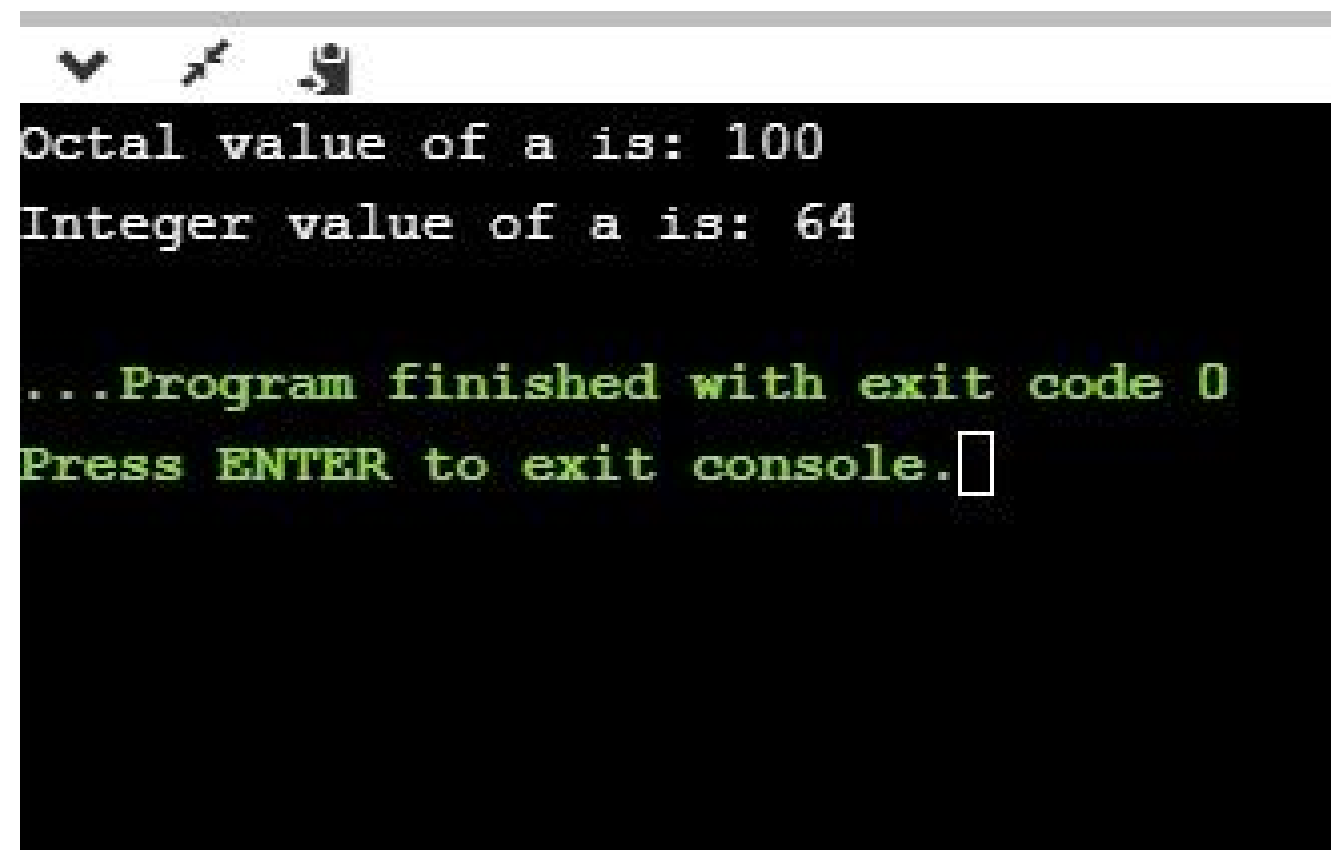
return 0;

}

```

In the above code, we are displaying the octal value and integer value of a.

Output



```

Octal value of a is: 100
Integer value of a is: 64

...Program finished with exit code 0
Press ENTER to exit console.

```

%x and %X

```

int main()

{

    int y=0xA;

    printf("Hexadecimal value of y is: %x", y);

    printf("\nHexadecimal value of y is: %X",y);

    printf("\nInteger value of y is: %d",y);

    return 0;

}

```

In the above code, y contains the hexadecimal value 'A'. We display the hexadecimal value of y in two formats. We use %x and %X to print the hexadecimal value where %x displays the value in small letters, i.e., 'a' and %X displays the value in a capital letter, i.e., 'A'.

Output

```
Hexadecimal value of y is: a
Hexadecimal value of y is: A
Integer value of y is: 10

...Program finished with exit code 0
Press ENTER to exit console.
```

%f

```
int main()
{
    float y=3.4;

    printf("Floating point value of y is: %f", y);

    return 0;
}
```

The above code prints the floating value of y.

Output

```
Floating point value of y is: 3.400000

...Program finished with exit code 0
Press ENTER to exit console.
```

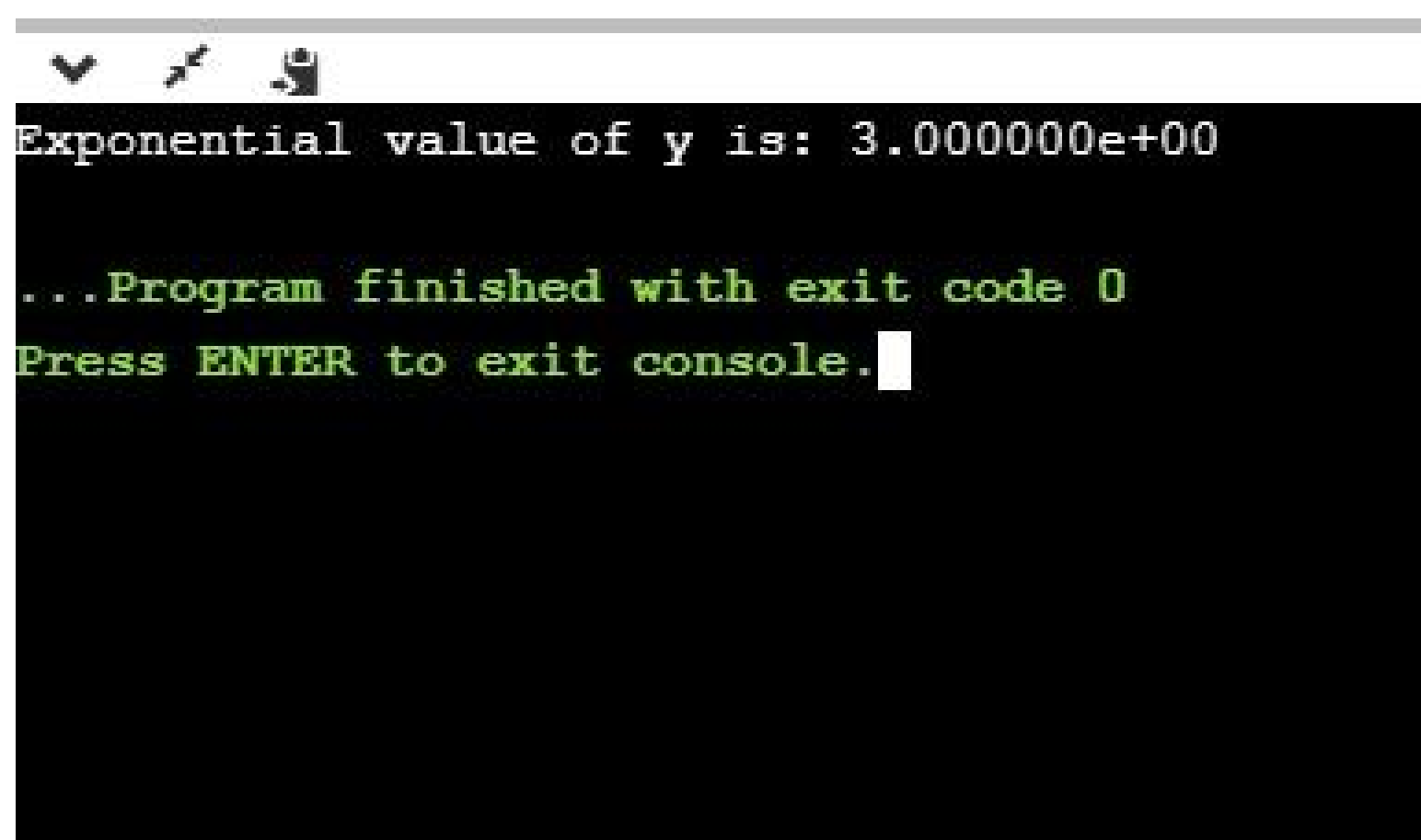
%e

```
int main()
{
    float y=3;

    printf("Exponential value of y is: %e", y);

    return 0;
}
```

Output



```
Exponential value of y is: 3.000000e+00
...Program finished with exit code 0
Press ENTER to exit console.
```

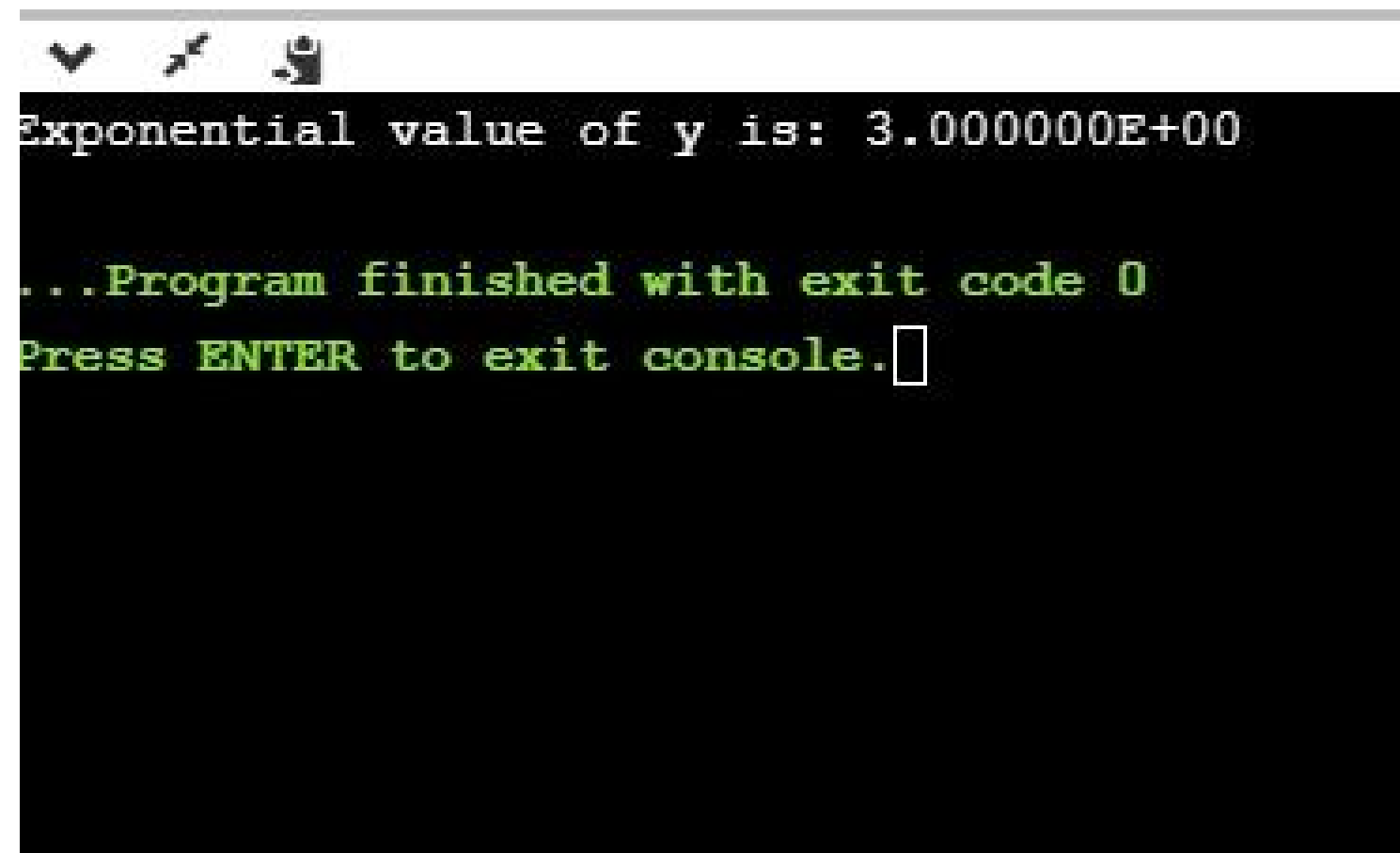
%E

```
int main()
{
    float y=3;

    printf("Exponential value of y is: %E", y);

    return 0;
}
```

Output

A terminal window with a black background and green text. The text reads: "Exponential value of y is: 3.000000E+00", followed by "...Program finished with exit code 0" and "Press ENTER to exit console." with a cursor. The window has standard OS icons at the top.

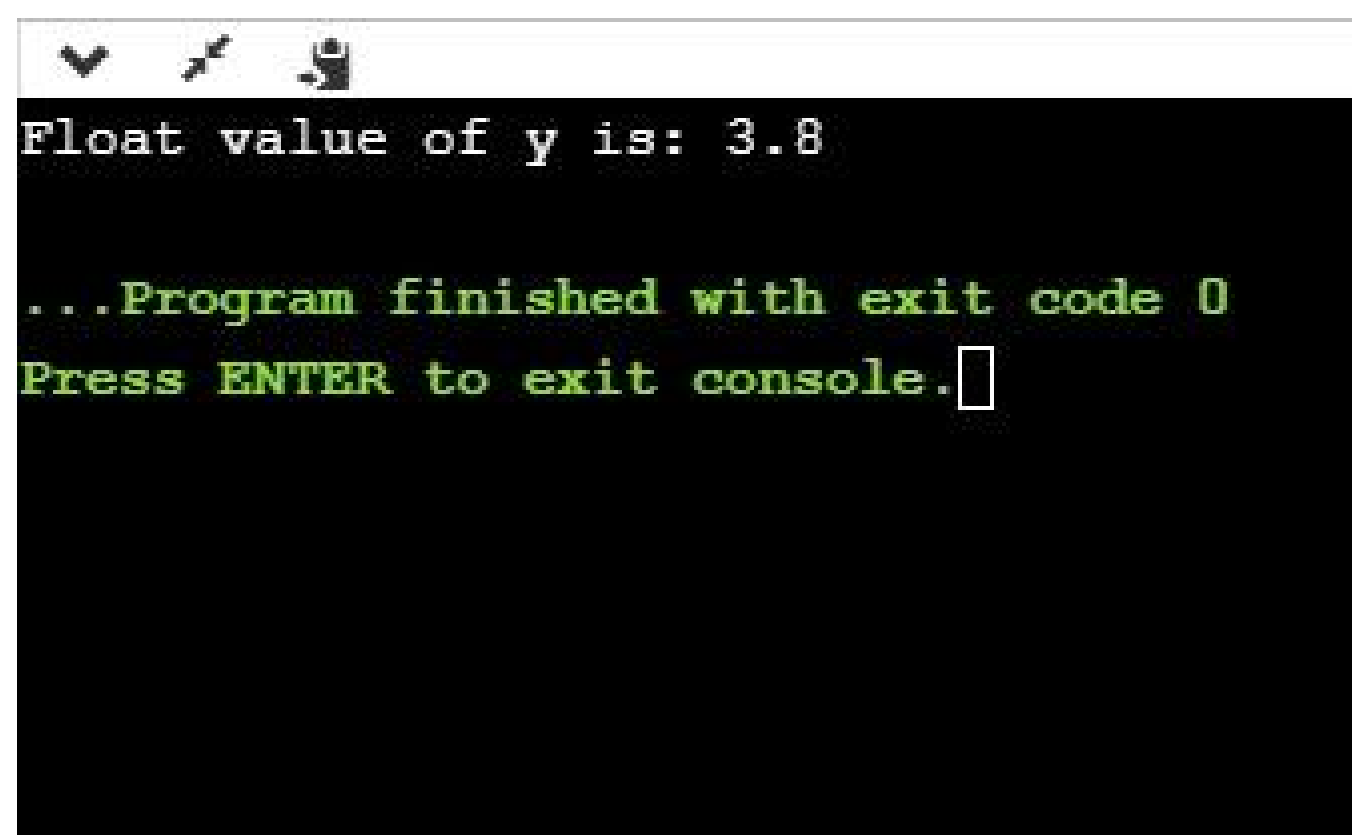
%g

```
int main()  
{  
    float y=3.8;  
    printf("Float value of y is: %g", y);  
    return 0;  
}
```

In the above code, we are displaying the floating value of y by using `%g` specifier. The `%g` specifier displays the output same as the input with a same precision.

Output

%p

A terminal window with a black background and green text. The text reads: "Float value of y is: 3.8", followed by "...Program finished with exit code 0" and "Press ENTER to exit console." with a cursor. The window has standard OS icons at the top.

```
int main()  
{
```

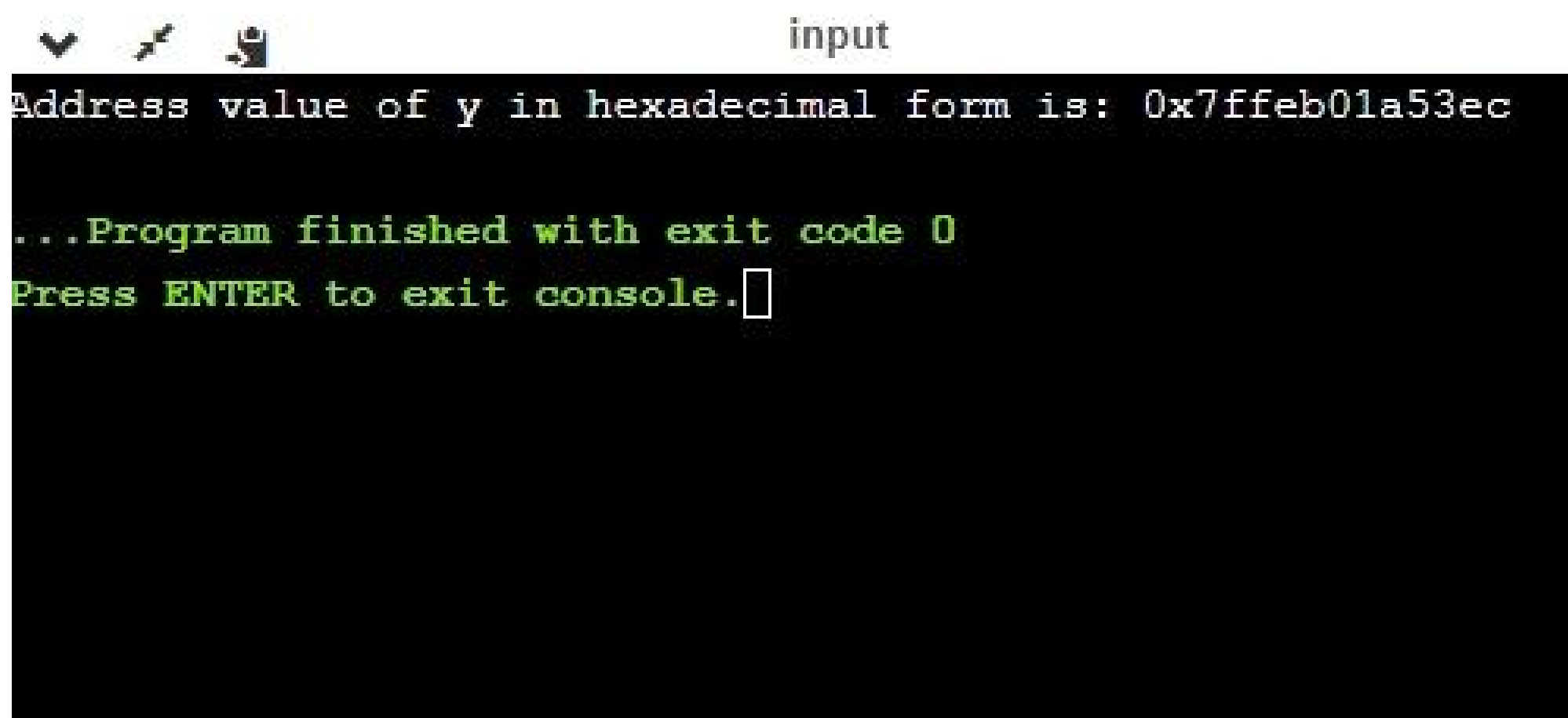
```
int y=5;

printf("Address value of y in hexadecimal form is: %p", &y);

return 0;

}
```

Output



```
input
Address value of y in hexadecimal form is: 0x7ffeb01a53ec
...Program finished with exit code 0
Press ENTER to exit console.█
```

%c

```
int main()

{

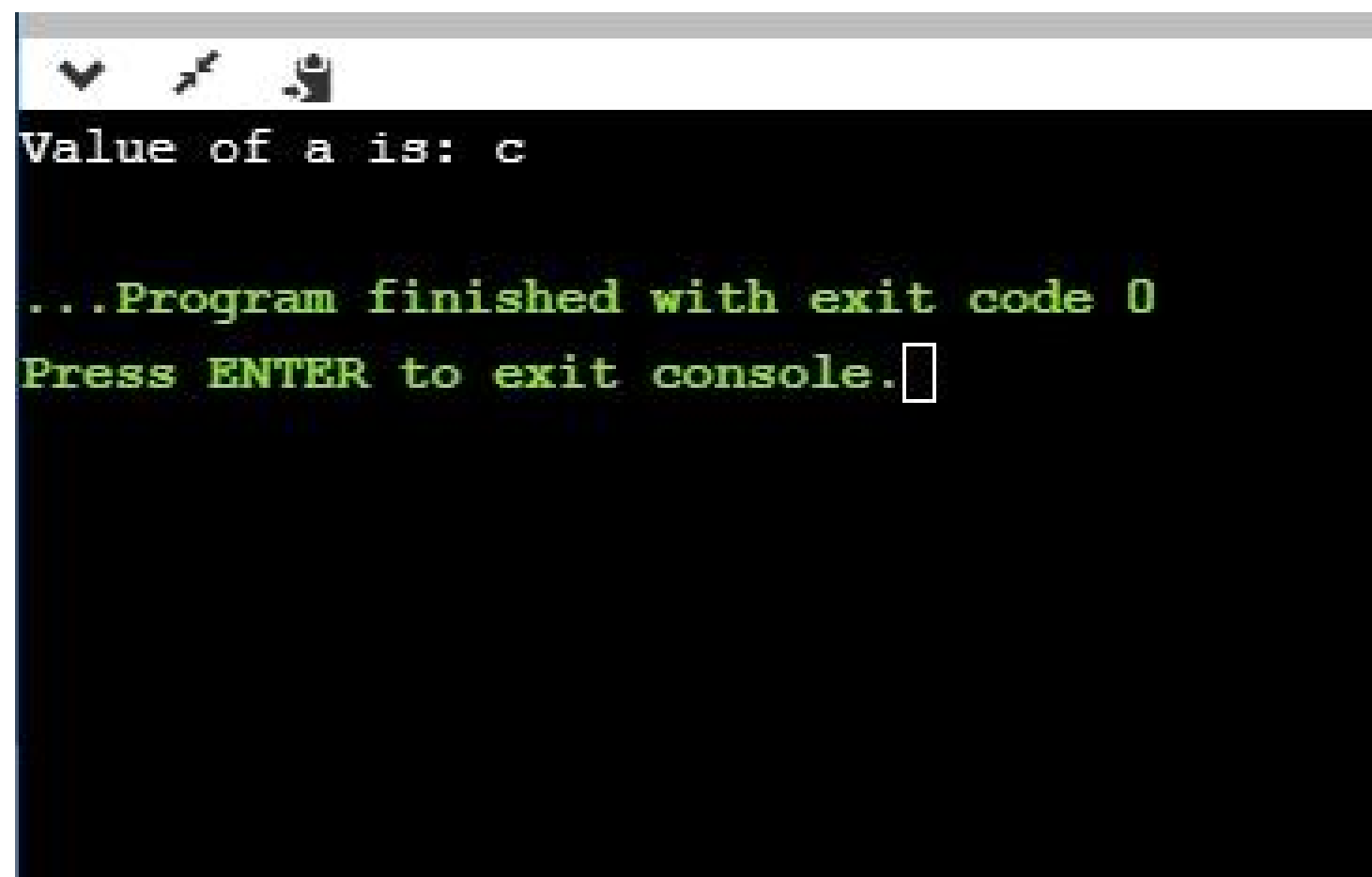
char a='c';

printf("Value of a is: %c", a);

return 0;

}
```

Output



```
Value of a is: c

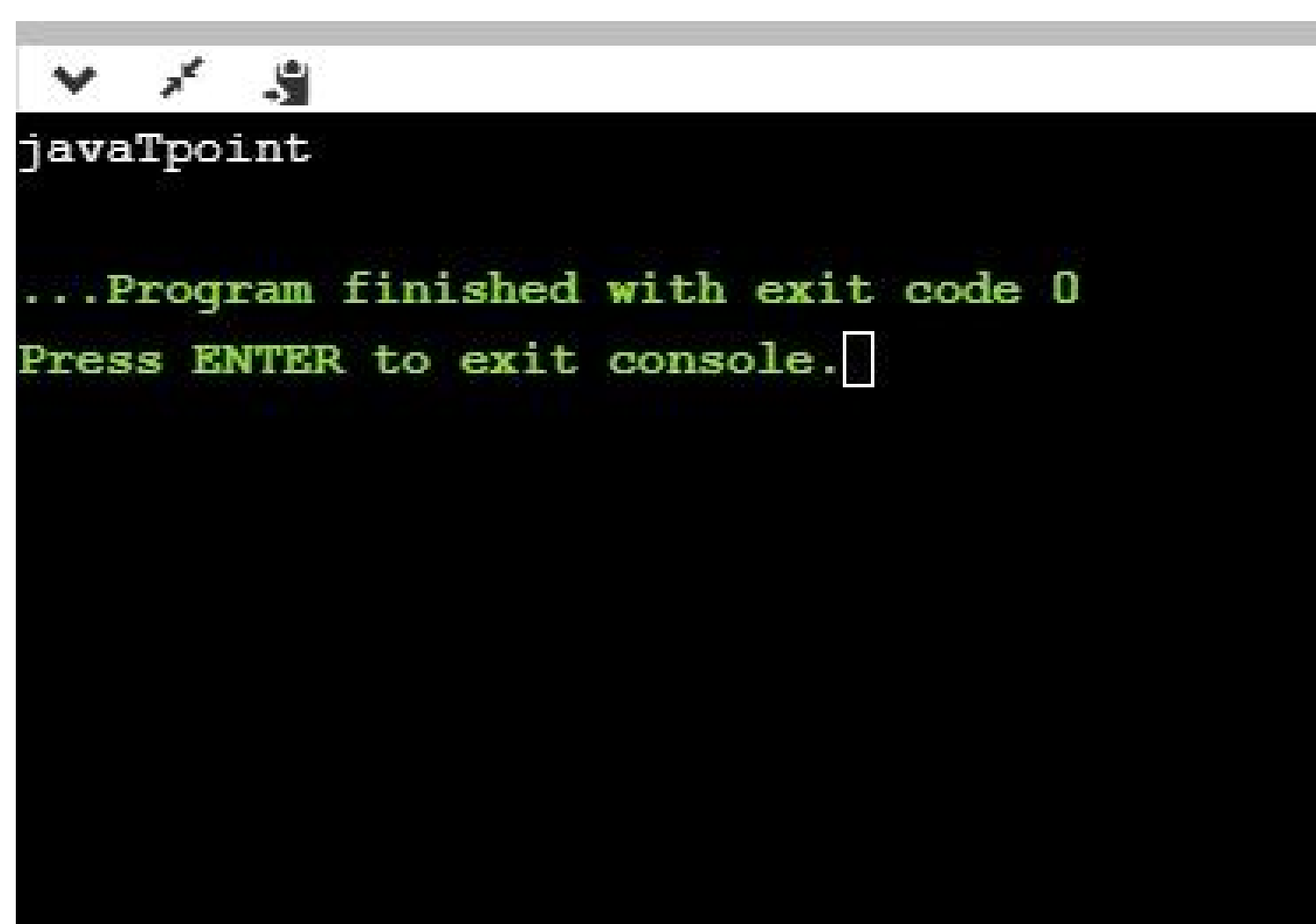
...Program finished with exit code 0
Press ENTER to exit console.
```

%s

```
int main()
{
    printf("%s", "javaTpoint");

    return 0;
}
```

Output



```
javaTpoint

...Program finished with exit code 0
Press ENTER to exit console.
```

Minimum Field Width Specifier

Suppose we want to display an output that occupies a minimum number of spaces on the screen. You can achieve this by displaying an integer number after the percent sign of the format specifier.

```
int main()
{
    int x=900;

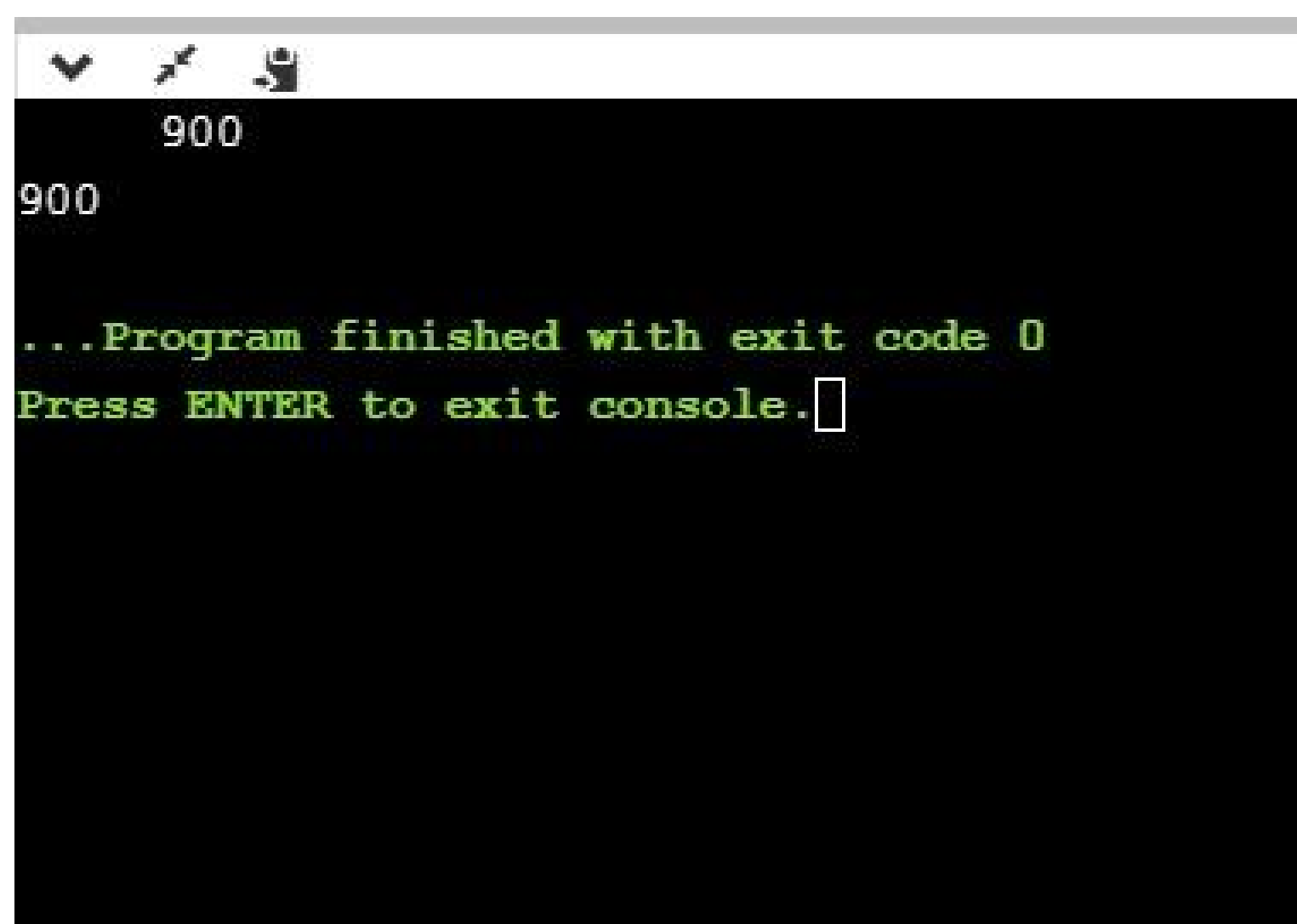
    printf("%8d", x);

    printf("\n%-8d",x);

    return 0;
}
```

In the above program, %8d specifier displays the value after 8 spaces while %-8d specifier will make a value left-aligned.

Output



```
          900
900
...Program finished with exit code 0
Press ENTER to exit console.
```

Now we will see how to fill the empty spaces. It is shown in the below code:

```
int main()
{
    int x=12;
```

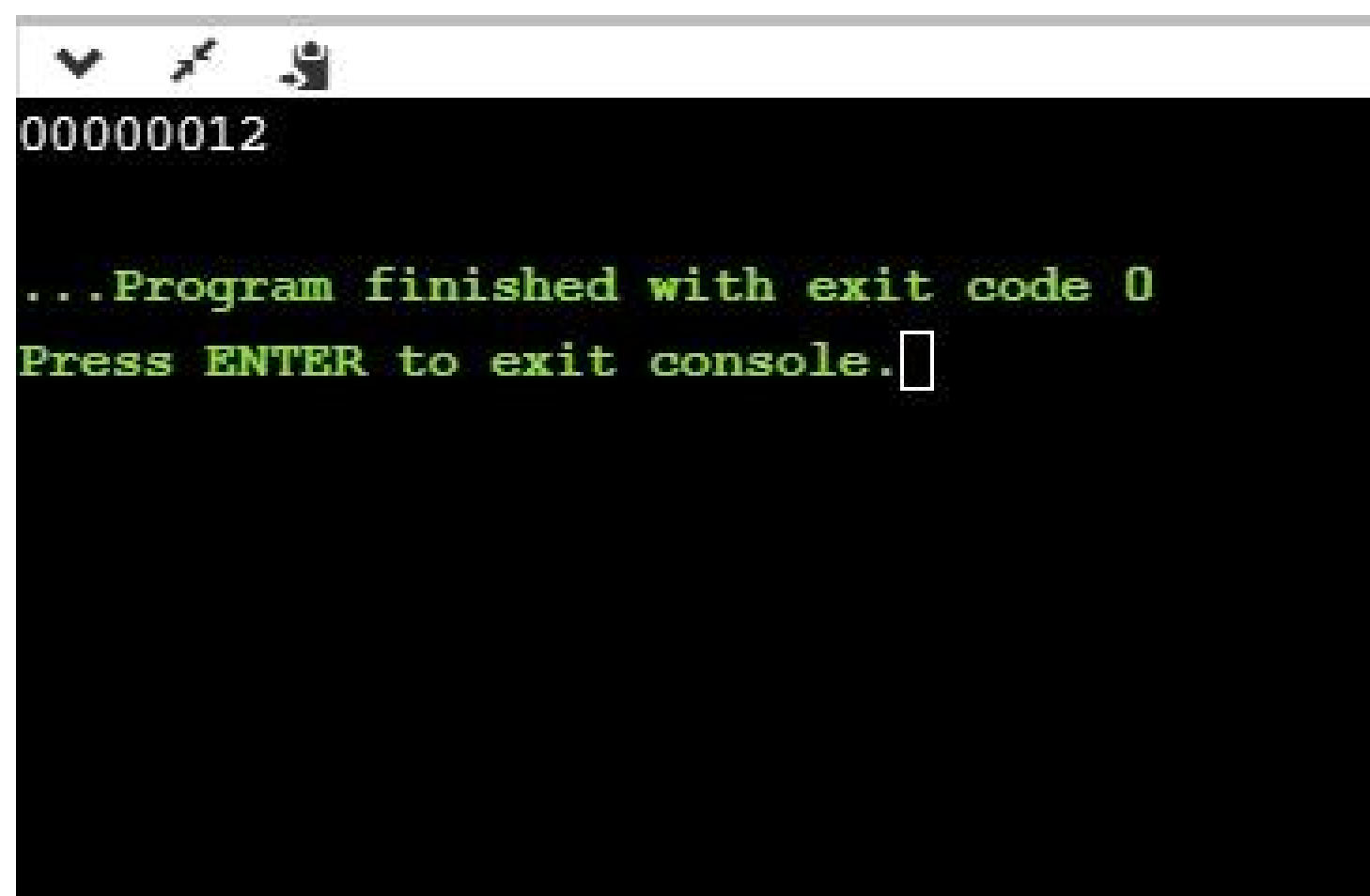
```
printf("%08d", x);

return 0;

}
```

In the above program, %08d means that the empty space is filled with zeroes.

Output



```
00000012

...Program finished with exit code 0
Press ENTER to exit console.█
```

Specifying Precision

We can specify the precision by using '.' (Dot) operator which is followed by integer and format specifier.

```
int main()

{

float x=12.2;

printf("%.2f", x);

return 0;

}
```

Output

```
12.20

...Program finished with exit code 0
Press ENTER to exit console.
```

Escape Sequence in C

An escape sequence in C language is a sequence of characters that doesn't represent itself when used inside string literal or character.

It is composed of two or more characters starting with backslash \. For example: \n represents new line.

List of Escape Sequences in C

Escape Sequence	Meaning
\a	Alarm or Beep
\b	Backspace
\f	Form Feed
\n	New Line
\r	Carriage Return

\t	Tab (Horizontal)
\v	Vertical Tab
\\	Backslash
\'	Single Quote
\"	Double Quote
\?	Question Mark
\nnn	octal number
\xhh	hexadecimal number
\0	Null

Escape Sequence Example

```
#include<stdio.h>

int main(){

    int number=50;

    printf("You\nare\nlearning\n'c' language\n\"Do you know C language\");

    return 0;

}
```

Output:

You
are

learning

'c' language

"Do you know C language"

ASCII value in C

What is ASCII code?

The full form of ASCII is the **American Standard Code for information interchange**. It is a character encoding scheme used for electronics communication. Each character or a special character is represented by some ASCII code, and each ascii code occupies 7 bits in memory.

In C programming language, a character variable does not contain a character value itself rather the ascii value of the character variable. The ascii value represents the character variable in numbers, and each character variable is assigned with some number range from 0 to 127. For example, the ascii value of 'A' is 65.

In the above example, we assign 'A' to the character variable whose ascii value is 65, so 65 will be stored in the character variable rather than 'A'.

Let's understand through an example.

We will create a program which will display the ascii value of the character variable.

```
#include <stdio.h>

int main()
{
    char ch; // variable declaration

    printf("Enter a character");

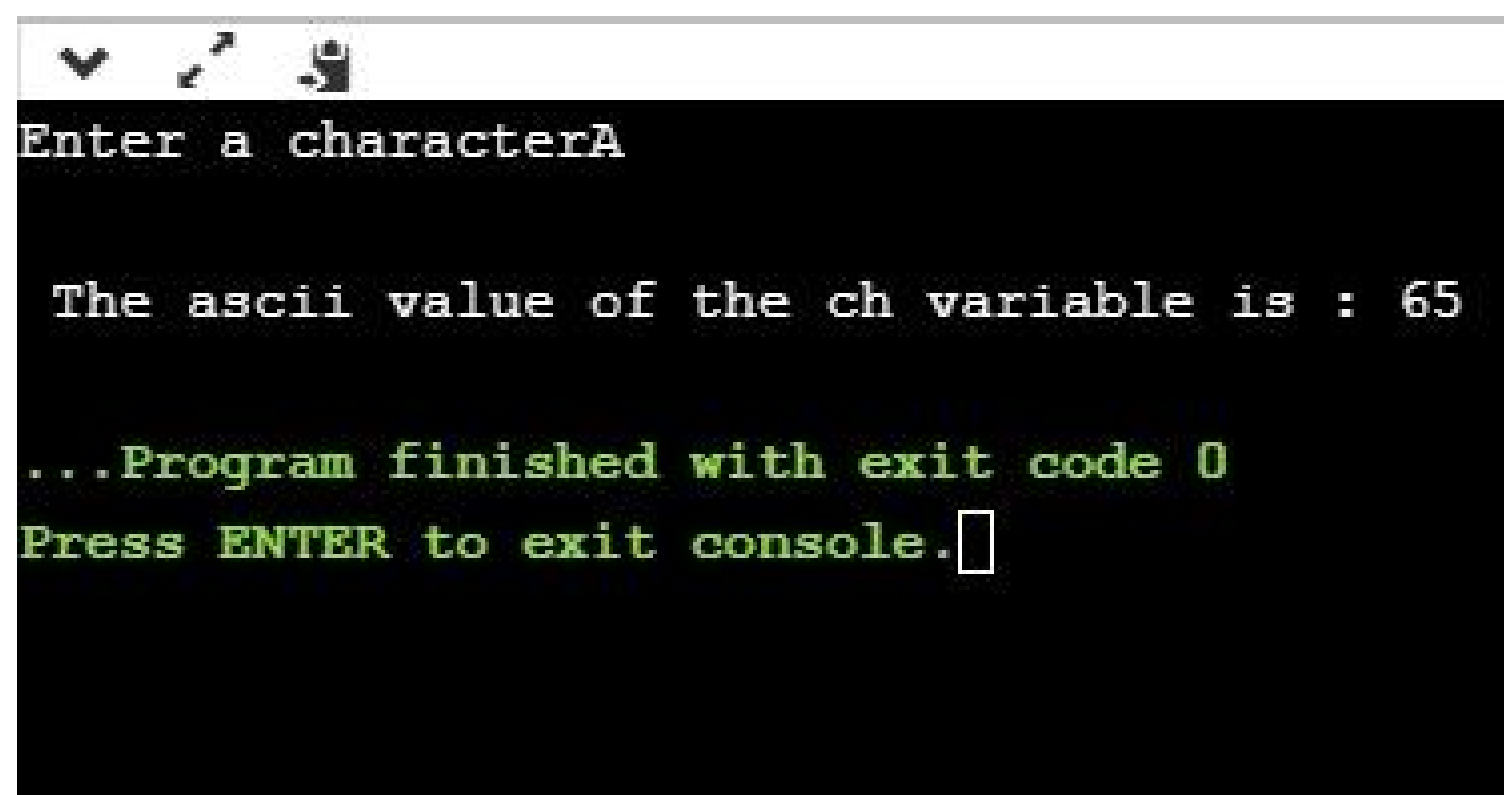
    scanf("%c",&ch); // user input

    printf("\n The ascii value of the ch variable is : %d", ch);
```

```
    return 0;
}
```

In the above code, the first user will give the character input, and the input will get stored in the 'ch' variable. If we print the value of the 'ch' variable by using %c format specifier, then it will display 'A' because we have given the character input as 'A', and if we use the %d format specifier then its ascii value will be displayed, i.e., 65.

Output



```
Enter a characterA

The ascii value of the ch variable is : 65

...Program finished with exit code 0
Press ENTER to exit console.
```

The above output shows that the user gave the input as 'A', and after giving input, the ascii value of 'A' will get printed, i.e., 65.

Now, we will create a program which will display the ascii value of all the characters.

```
#include <stdio.h>

int main()
{
    int k; // variable declaration

    for(int k=0;k<=255;k++) // for loop from 0-255
    {
        printf("\nThe ascii value of %c is %d", k,k);
    }

    return 0;
}
```

```
}
```

The above program will display the ascii value of all the characters. As we know that ascii value of all the characters starts from 0 and ends at 255, so we iterate the for loop from 0 to 255.

Now we will create the program which will sum the ascii value of a string.

```
#include <stdio.h>

int main()

{

    int sum=0; // variable initialization

    char name[20]; // variable initialization

    int i=0; // variable initialization

    printf("Enter a name: ");

    scanf("%s", name);

    while(name[i]!='\0') // while loop

    {

        printf("\nThe ascii value of the character %c is %d", name[i],name[i]);

        sum=sum+name[i];

        i++;

    }

    printf("\nSum of the ascii value of a string is : %d", sum);

    return 0;

}
```

In the above code, we are taking user input as a string. After taking user input, we execute the **while** loop which adds the ascii value of all the characters of a string and stores it in a 'sum' variable.

Output

```
Enter a name: akshita

The ascii value of the character a is 97
The ascii value of the character k is 107
The ascii value of the character s is 115
The ascii value of the character h is 104
The ascii value of the character i is 105
The ascii value of the character t is 116
The ascii value of the character a is 97
Sum of the ascii value of a string is : 741

...Program finished with exit code 0
Press ENTER to exit console.
```

Constants in C

A constant is a value or variable that can't be changed in the program, for example: 10, 20, 'a', 3.4, "c programming" etc.

There are different types of constants in C programming.

List of Constants in C

Constant	Example
Decimal Constant	10, 20, 450 etc.
Real or Floating-point Constant	10.3, 20.2, 450.6 etc.
Octal Constant	021, 033, 046 etc.
Hexadecimal Constant	0x2a, 0x7b, 0xaa etc.

Character Constant	'a', 'b', 'x' etc.
String Constant	"c", "c program", "c in javatpoint" etc.

2 ways to define constant in C

There are two ways to define constant in C programming.

1. const keyword
2. #define preprocessor

1) C const keyword

The const keyword is used to define constant in C programming.

```
const float PI=3.14;
```

Now, the value of PI variable can't be changed.

```
#include<stdio.h>

int main(){

    const float PI=3.14;

    printf("The value of PI is: %f",PI);

    return 0;

}
```

Output:

```
The value of PI is: 3.140000
```

If you try to change the the value of PI, it will render compile time error.

```
#include<stdio.h>

int main(){
```

```

const float PI=3.14;

PI=4.5;

printf("The value of PI is: %f",PI);

return 0;

}

```

Output:

Compile Time Error: Cannot modify a const object

2) C #define preprocessor

The #define preprocessor directive is used to define constant or micro substitution. It can use any basic data type.

Syntax:

```
#define token value
```

Let's see an example of #define to define a constant.

```

#include <stdio.h>

#define PI 3.14

main() {

    printf("%f",PI);

}

```

Output:

3.140000

Let's see an example of #define to create a macro.

```

#include <stdio.h>

#define MIN(a,b) ((a)<(b)?(a):(b))

void main() {

```

```
printf("Minimum between 10 and 20 is: %d\n", MIN(10,20));  
}
```

Output:

Minimum between 10 and 20 is: 10

Literals in C

What are literals?

Literals are the constant values assigned to the constant variables. We can say that the literals represent the fixed values that cannot be modified. It also contains memory but does not have references as variables. For example, `const int =10;` is a constant integer expression in which 10 is an integer literal.

Types of literals

There are four types of literals that exist in C programming:

- Integer literal
- Float literal
- Character literal
- String literal

Integer literal

It is a numeric literal that represents only integer type values. It represents the value neither in fractional nor exponential part.

It can be specified in the following three ways:

Decimal number (base 10)

It is defined by representing the digits between 0 to 9. For example, 45, 67, etc.

Octal number (base 8)

It is defined as a number in which 0 is followed by digits such as 0,1,2,3,4,5,6,7. For example, 012, 034, 055, etc.

Hexadecimal number (base 16)

It is defined as a number in which 0x or 0X is followed by the hexadecimal digits (i.e., digits from 0 to 9, alphabetical characters from (a-z) or (A-Z)).

An integer literal is suffixed by following two sign qualifiers:

L or l: It is a size qualifier that specifies the size of the integer type as long.

U or u: It is a sign qualifier that represents the type of the integer as unsigned. An unsigned qualifier contains only positive values.

Note: The order of the qualifier is not considered, i.e., both lu and ul are the same.

Let's look at a simple example of integer literal.

```
#include <stdio.h>

int main()
{
    const int a=23; // constant integer literal

    printf("Integer literal : %d", a);

    return 0;
}
```

Output

Integer literal : 23

Float literal

It is a literal that contains only floating-point values or real numbers. These real numbers contain the number of parts such as integer part, real part, exponential part,

and fractional part. The floating-point literal must be specified either in decimal or in exponential form. Let's understand these forms in brief.

Decimal form

The decimal form must contain either decimal point, exponential part, or both. If it does not contain either of these, then the compiler will throw an error. The decimal notation can be prefixed either by '+' or '-' symbol that specifies the positive and negative numbers.

Examples of float literal in decimal form are:

1.2, +9.0, -4.5

Let's see a simple example of float literal in decimal form.

```
#include <stdio.h>

int main()
{
    const float a=4.5; // constant float literal
    const float b=5.6; // constant float literal

    float sum;

    sum=a+b;

    printf("%f", sum);

    return 0;
}
```

Output

10.100000

Exponential form

The exponential form is useful when we want to represent the number, which is having a big magnitude. It contains two parts, i.e., mantissa and exponent. For example, the number is 2340000000000, and it can be expressed as 2.34e12 in an exponential form.

Syntax of float literal in exponential form

[+/-] <Mantissa> <e/E> [+/-] <Exponent>

Examples of real literal in exponential notation are:

+1e23, -9e2, +2e-25

Rules for creating an exponential notation

The following are the rules for creating a float literal in exponential notation:

- In exponential notation, the mantissa can be specified either in decimal or fractional form.
- An exponent can be written in both uppercase and lowercase, i.e., e and E.
- We can use both the signs, i.e., positive and negative, before the mantissa and exponent.
- Spaces are not allowed

Character literal

A character literal contains a single character enclosed within single quotes. If multiple characters are assigned to the variable, then we need to create a character array. If we try to store more than one character in a variable, then the warning of a **multi-character character constant** will be generated. Let's observe this scenario through an example.

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```

const char c='ak';

printf("%c",c);

return 0;

}

```

In the above code, we have used two characters, i.e., 'ak', within single quotes. So, this statement will generate a warning as shown below.

Warning generated:

```

main.c:6:18: warning: multi-character character constant
               [-Wmultichar]
const char c='ak';
main.c:6:18: warning: implicit conversion from 'int' to 'char'
               changes value from 24939 to 107 [-Wconstant-conversion]
const char c='ak';
               ~ ^~~~

2 warnings generated.

? ./main

```

Representation of character literal

A character literal can be represented in the following ways:

- It can be represented by specifying a single character within single quotes. For example, 'a', 'b', etc.
- We can specify the escape sequence character within single quotes to represent a character literal. For example, '\n', '\a', '\b'.
- We can also use the ASCII in integer to represent a character literal. For example, the ascii value of 65 is 'A'.

- The octal and hexadecimal notation can be used as an escape sequence to represent a character literal. For example, '\023', '\0x12'.

String literal

A string literal represents multiple characters enclosed within double-quotes. It contains an additional character, i.e., '\0' (null character), which gets automatically inserted. This null character specifies the termination of the string. We can use the '+' symbol to concatenate two strings.

For example,

- String1= "javatpoint";
- String2= "family";

To concatenate the above two strings, we use '+' operator, as shown in the below statement:

```
"javatpoint " + "family"= javatpoint family
```

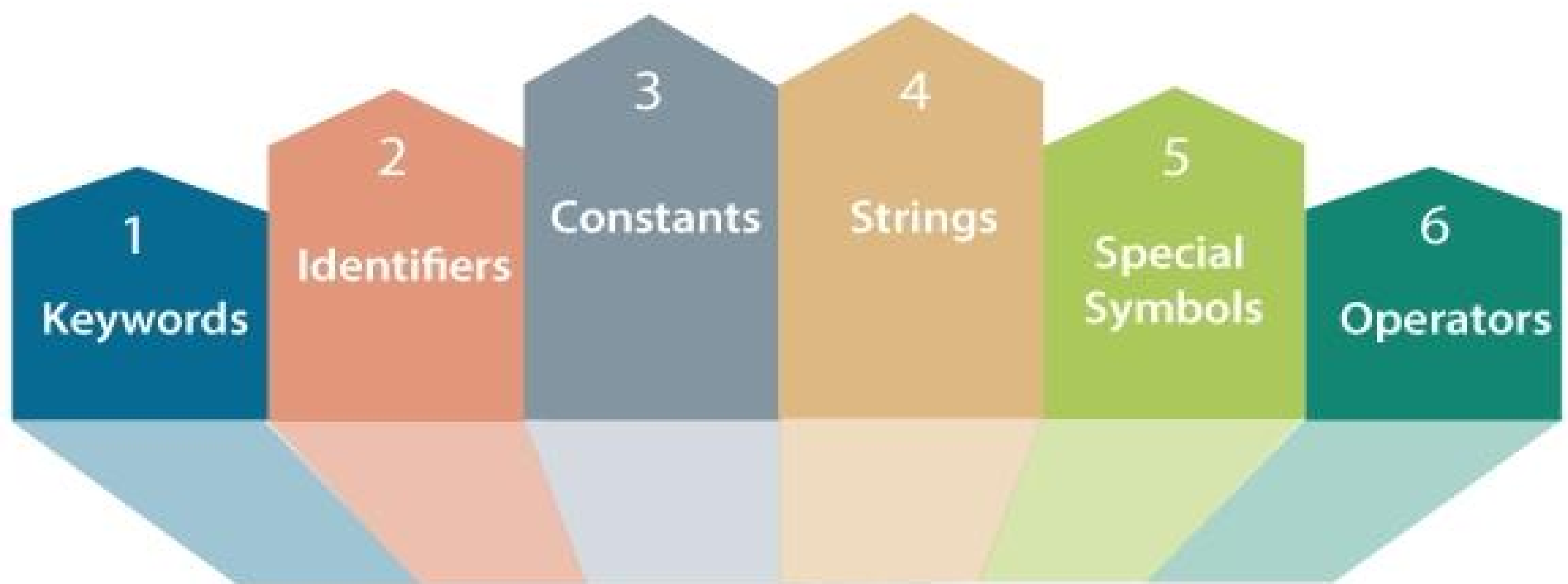
Note: If we represent a single character, i.e., 'b', then this character will occupy a single byte as it is a character literal. And, if we represent the character within double quotes "b" then it will occupy more bytes as it is a string literal.

Tokens in C

Tokens in C is the most important element to be used in creating a program in C. We can define the token as the smallest individual element in C. For example, we cannot create a sentence without using words; similarly, we cannot create a program in C without using tokens in C. Therefore, we can say that tokens in C is the building block or the basic component for creating a program in C language.

Classification of tokens in C

Tokens in C language can be divided into the following categories:



Classification of C Tokens

- Keywords in C
- Identifiers in C
- Strings in C
- Operators in C
- Constant in C
- Special Characters in C

Let's understand each token one by one.

Keywords in C

Keywords in C can be defined as the **pre-defined** or the **reserved words** having its own importance, and each keyword has its own functionality. Since keywords are the pre-defined words used by the compiler, so they cannot be used as the variable names.

If the keywords are used as the variable names, it means that we are assigning a different meaning to the keyword, which is not allowed. C language supports 32 keywords given below:

auto	double	int	struct
break	else	long	switch
case	enum	register	typedef
char	extern	return	union
const	float	short	unsigned
continue	for	signed	void
default	goto	sizeof	volatile
do	if	static	while

Identifiers in C

Identifiers in C are used for naming variables, functions, arrays, structures, etc. Identifiers in C are the user-defined words. It can be composed of uppercase letters, lowercase letters, underscore, or digits, but the starting letter should be either an underscore or an alphabet. Identifiers cannot be used as keywords. Rules for constructing identifiers in C are given below:

- The first character of an identifier should be either an alphabet or an underscore, and then it can be followed by any of the character, digit, or underscore.
- It should not begin with any numerical digit.
- In identifiers, both uppercase and lowercase letters are distinct. Therefore, we can say that identifiers are case sensitive.

- Commas or blank spaces cannot be specified within an identifier.
- Keywords cannot be represented as an identifier.
- The length of the identifiers should not be more than 31 characters.
- Identifiers should be written in such a way that it is meaningful, short, and easy to read.

Strings in C

Strings in C are always represented as an array of characters having null character '\0' at the end of the string. This null character denotes the end of the string. Strings in C are enclosed within double quotes, while characters are enclosed within single characters. The size of a string is a number of characters that the string contains.

Now, we describe the strings in different ways:

```
char a[10] = "javatpoint"; // The compiler allocates the 10 bytes to the 'a' array.
```

```
char a[] = "javatpoint"; // The compiler allocates the memory at the run time.
```

```
char a[10] = {'j','a','v','a','t','p','o','i','n','t','\0'}; // String is represented in the form of characters.
```

Operators in C

Operators in C is a special symbol used to perform the functions. The data items on which the operators are applied are known as operands. Operators are applied between the operands. Depending on the number of operands, operators are classified as follows:

Unary Operator

A unary operator is an operator applied to the single operand. For example: increment operator (`++`), decrement operator (`--`), `sizeof`, `(type)*`.

Binary Operator

The binary operator is an operator applied between two operands. The following is the list of the binary operators:

- Arithmetic Operators
- Relational Operators
- Shift Operators
- Logical Operators
- Bitwise Operators
- Conditional Operators
- Assignment Operator
- Misc Operator

Constants in C

A constant is a value assigned to the variable which will remain the same throughout the program, i.e., the constant value cannot be changed.

There are two ways of declaring constant:

- Using const keyword
- Using #define pre-processor

Types of constants in C

Constant	Example
Integer constant	10, 11, 34, etc.

Floating-point constant	45.6, 67.8, 11.2, etc.
Octal constant	011, 088, 022, etc.
Hexadecimal constant	0x1a, 0x4b, 0x6b, etc.
Character constant	'a', 'b', 'c', etc.
String constant	"java", "c++", ".net", etc.

Special characters in C

Some special characters are used in C, and they have a special meaning which cannot be used for another purpose.

- **Square brackets []**: The opening and closing brackets represent the single and multidimensional subscripts.
- **Simple brackets ()**: It is used in function declaration and function calling. For example, `printf()` is a pre-defined function.
- **Curly braces { }**: It is used in the opening and closing of the code. It is used in the opening and closing of the loops.
- **Comma (,)**: It is used for separating for more than one statement and for example, separating function parameters in a function call, separating the variable when printing the value of more than one variable using a single `printf` statement.
- **Hash/pre-processor (#)**: It is used for pre-processor directive. It basically denotes that we are using the header file.

- **Asterisk (*)**: This symbol is used to represent pointers and also used as an operator for multiplication.
- **Tilde (~)**: It is used as a destructor to free memory.
- **Period (.)**: It is used to access a member of a structure or a union.

C Boolean

In C, Boolean is a data type that contains two types of values, i.e., 0 and 1. Basically, the bool type value represents two types of behavior, either true or false. Here, '0' represents false value, while '1' represents true value.

In C Boolean, '0' is stored as 0, and another integer is stored as 1. We do not require to use any header file to use the Boolean data type in C++, but in C, we have to use the header file, i.e., stdbool.h. If we do not use the header file, then the program will not compile.

Syntax

```
bool variable_name;
```

In the above syntax, **bool** is the data type of the variable, and **variable_name** is the name of the variable.

Let's understand through an example.

```
#include <stdio.h>

#include<stdbool.h>

int main()

{

    bool x=false; // variable initialization.

    if(x==true) // conditional statements

    {
```

```

printf("The value of x is true");

}

else

printf("The value of x is FALSE");

return 0;

}

```

In the above code, we have used `<stdbool.h>` header file so that we can use the `bool` type variable in our program. After the declaration of the header file, we create the `bool` type variable 'x' and assigns a 'false' value to it. Then, we add the conditional statements, i.e., `if..else`, to determine whether the value of 'x' is true or not.

Output

```
The value of x is FALSE
```

Boolean Array

Now, we create a `bool` type array. The Boolean array can contain either true or false value, and the values of the array can be accessed with the help of indexing.

Let's understand this scenario through an example.

```

#include <stdio.h>

#include<stdbool.h>

int main()

{

bool b[2]={true,false}; // Boolean type array

for(int i=0;i<2;i++) // for loop

{

printf("%d",b[i]); // printf statement

}

```

```
return 0;

}
```

In the above code, we have declared a Boolean type array containing two values, i.e., true and false.

Output

```
1,0,
```

typedef

There is another way of using Boolean value, i.e., **typedef**. Basically, typedef is a keyword in C language, which is used to assign the name to the already existing datatype.

Let's see a simple example of typedef.

```
#include <stdio.h>

typedef enum{false,true} b;

int main()

{

    b x=false; // variable initialization

    if(x==true) // conditional statements

    {

        printf("The value of x is true");

    }

    else

    {

        printf("The value of x is false");

    }

    return 0;
```

```
}
```

In the above code, we use the Boolean values, i.e., true and false, but we have not used the bool type. We use the Boolean values by creating a new name of the 'bool' type. In order to achieve this, the **typedef** keyword is used in the program.

```
typedef enum{false,true} b;
```

The above statement creates a new name for the 'bool' type, i.e., 'b' as 'b' can contain either true or false value. We use the 'b' type in our program and create the 'x' variable of type 'b'.

Output

The value of x is false

Boolean with Logical Operators

The Boolean type value is associated with logical operators. There are three types of logical operators in the C language:

&&(AND Operator): It is a logical operator that takes two operands. If the value of both the operands are true, then this operator returns true otherwise false

||(OR Operator): It is a logical operator that takes two operands. If the value of both the operands is false, then it returns false otherwise true.

!(NOT Operator): It is a NOT operator that takes one operand. If the value of the operand is false, then it returns true, and if the value of the operand is true, then it returns false.

Let's understand through an example.

```
#include <stdio.h>

#include<stdbool.h>

int main()

{
```

```

bool x=false;

bool y=true;

printf("The value of x&&y is %d", x&&y);

printf("\nThe value of x||y is %d", x||y);

printf("\nThe value of !x is %d", !x);

}

```

Output

```

The value of x&&y is 0
The value of x||y is 1
The value of !x is 1

```

Static in C

Static is a keyword used in C programming language. It can be used with both variables and functions, i.e., we can declare a static variable and static function as well. An ordinary variable is limited to the scope in which it is defined, while the scope of the static variable is throughout the program.

Static keyword can be used in the following situations:

- **Static global variable**

When a global variable is declared with a static keyword, then it is known as a static global variable. It is declared at the top of the program, and its visibility is throughout the program.

- **Static function**

When a function is declared with a static keyword known as a static function. Its lifetime is throughout the program.

- **Static local variable**

When a local variable is declared with a static keyword, then it is known as a

static local variable. The memory of a static local variable is valid throughout the program, but the scope of visibility of a variable is the same as the automatic local variables. However, when the function modifies the static local variable during the first function call, then this modified value will be available for the next function call also.

- **Static member variables**

When the member variables are declared with a static keyword in a class, then it is known as static member variables. They can be accessed by all the instances of a class, not with a specific instance.

- **Static method**

The member function of a class declared with a static keyword is known as a static method. It is accessible by all the instances of a class, not with a specific instance.

Let's understand through an example.

```
#include <stdio.h>

int main()

{

    printf("%d",func());

    printf("\n%d",func());

    return 0;

}

int func()

{

    int count=0; // variable initialization

    count++; // incrementing counter variable
```

```
return count; }
```

In the above code, the func() function is called. In func(), count variable gets updated.

As soon as the function completes its execution, the memory of the count variable will be removed. If we do not want to remove the count from memory, then we need to use the count variable as static. If we declare the variable as static, then the variable will not be removed from the memory even when the function completes its execution.

Output

1

1

Static variable

A static variable is a variable that persists its value across the various function calls.

Syntax

The syntax of a static variable is given below:

```
static data_type variable_name;
```

Let's look at a simple example of static variable.

```
#include <stdio.h>

int main()
{
    printf("%d",func());
    printf("\n%d",func());

    return 0;
}

int func()
{
    static int count=0;
```

```
    count++;  
  
    return count;  
}
```

In the above code, we have declared the count variable as static. When the func() is called, the value of count gets updated to 1, and during the next function call, the value of the count variable becomes 2. Therefore, we can say that the value of the static variable persists within the function call.

Output

```
1  
2
```

Static Function

As we know that non-static functions are global by default means that the function can be accessed outside the file also, but if we declare the function as static, then it limits the function scope. The static function can be accessed within a file only.

The static function would look like as:

```
static void func()  
{  
    printf("Hello javaTpoint");  
}
```

Differences b/w static and global variable

Global variables are the variables that are declared outside the function. These global variables exist at the beginning of the program, and its scope remains till the end of the program. It can be accessed outside the program also.

Static variables are limited to the source file in which they are defined, i.e., they are not accessible by the other source files.

Both the static and global variables have static initialization. Here, static initialization means if we do not assign any value to the variable then by default, 0 value will be assigned to the variable.

Differences b/w static local and static global variable

Static global variable

If the variable declared with a static keyword outside the function, then it is known as a static global variable. It is accessible throughout the program.

Static local variable

The variable with a static keyword is declared inside a function is known as a static local variable. The scope of the static local variable will be the same as the automatic local variables, but its memory will be available throughout the program execution. When the function modifies the value of the static local variable during one function call, then it will remain the same even during the next function call.

Properties of a static variable

The following are the properties of a static variable:

- The memory of a static variable is allocated within a static variable.
- Its memory is available throughout the program, but the scope will remain the same as the automatic local variables. Its
- value will persist across the various function calls.
- If we do not assign any value to the variable, then the default value will be 0.
- A global static variable cannot be accessed outside the program, while a global variable can be accessed by other source files.

Programming Errors in C

Errors are the problems or the faults that occur in the program, which makes the behavior of the program abnormal, and experienced developers can also make these faults. Programming errors are also known as the bugs or faults, and the process of removing these bugs is known as **debugging**.

These errors are detected either during the time of compilation or execution. Thus, the errors must be removed from the program for the successful execution of the program.

There are mainly five types of errors exist in C programming:

- Syntax error
- Run-time error
- Linker error
- Logical error
- Semantic error

Types of
errors



Syntax error

Syntax errors are also known as the compilation errors as they occurred at the compilation time, or we can say that the syntax errors are thrown by the compilers. These errors are mainly occurred due to the mistakes while typing or do not follow the syntax of the specified programming language. These mistakes are generally made by beginners only because they are new to the language. These errors can be easily debugged or corrected.

For example:

If we want to declare the variable of type integer,

```
int a; // this is the correct form
```

```
Int a; // this is an incorrect form.
```

Commonly occurred syntax errors are:

- If we miss the parenthesis (}) while writing the code.
- Displaying the value of a variable without its declaration.
- If we miss the semicolon (;) at the end of the statement.

Let's understand through an example.

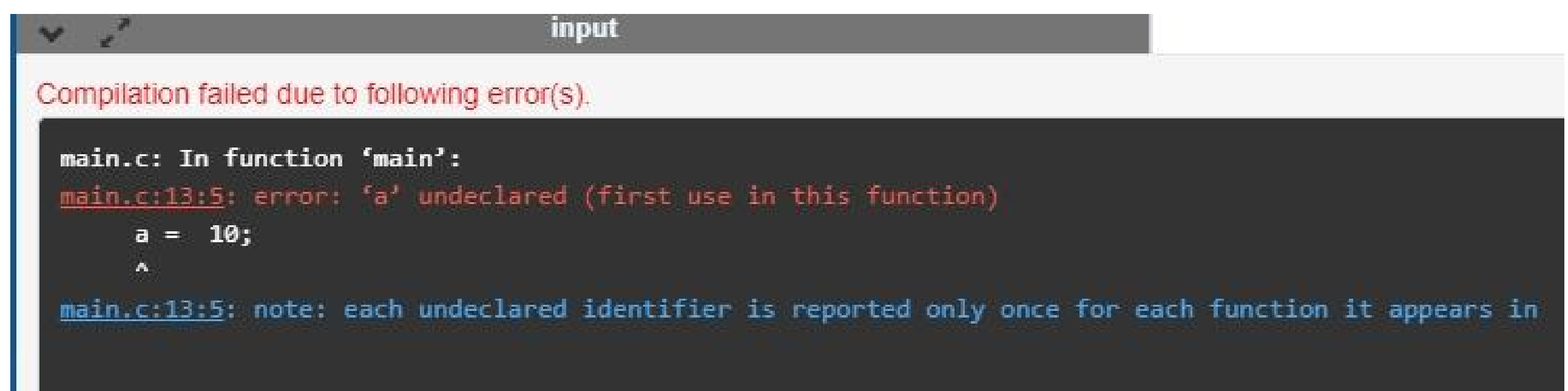
```
#include <stdio.h>

int main()
{
    a = 10;

    printf("The value of a is : %d", a);

    return 0;
}
```

Output



```
input
Compilation failed due to following error(s).

main.c: In function 'main':
main.c:13:5: error: 'a' undeclared (first use in this function)
    a = 10;
    ^
main.c:13:5: note: each undeclared identifier is reported only once for each function it appears in
```

In the above output, we observe that the code throws the error that 'a' is undeclared. This error is nothing but the syntax error only.

There can be another possibility in which the syntax error can exist, i.e., if we make mistakes in the basic construct. Let's understand this scenario through an example.

```
#include <stdio.h>

int main()
{
    int a=2;

    if(.) // syntax error

    printf("a is greater than 1");

    return 0;
}
```

In the above code, we put the (.) instead of condition in 'if', so this generates the syntax error as shown in the below screenshot.

Output



```
input
Compilation failed due to following error(s).

main.c: In function 'main':
main.c:15:6: error: expected expression before '.' token
    if(.
```

Run-time error

Sometimes the errors exist during the execution-time even after the successful compilation known as run-time errors. When the program is running, and it is not able to perform the operation is the main cause of the run-time error. The division by zero is the common example of the run-time error. These errors are very difficult to find, as the compiler does not point to these errors.

Let's understand through an example.

```
#include <stdio.h>

int main()
{
    int a=2;

    int b=2/0;

    printf("The value of b is : %d", b);

    return 0;
}
```

Output

```
main.c:14:12: warning: division by zero [-Wdiv-by-zero]
Floating point exception

...Program finished with exit code 136
Press ENTER to exit console.█
```

In the above output, we observe that the code shows the run-time error, i.e., division by zero.

Linker error

Linker errors are mainly generated when the executable file of the program is not created. This can be happened either due to the wrong function prototyping or usage of the wrong header file. For example, the `main.c` file contains the `sub()` function whose declaration and definition is done in some other file such as `func.c`. During the compilation, the compiler finds the `sub()` function in `func.c` file, so it generates two object files, i.e., `main.o` and `func.o`. At the execution time, if the definition of `sub()` function is not found in the `func.o` file, then the linker error will be thrown. The most common linker error that occurs is that we use `Main()` instead of `main()`.

Let's understand through a simple example.

```
#include <stdio.h>

int Main()

{

    int a=78;

    printf("The value of a is : %d", a);

    return 0;

}
```

Output

```
(.text+0x20): undefined reference to `main'  
collect2: error: ld returned 1 exit status
```

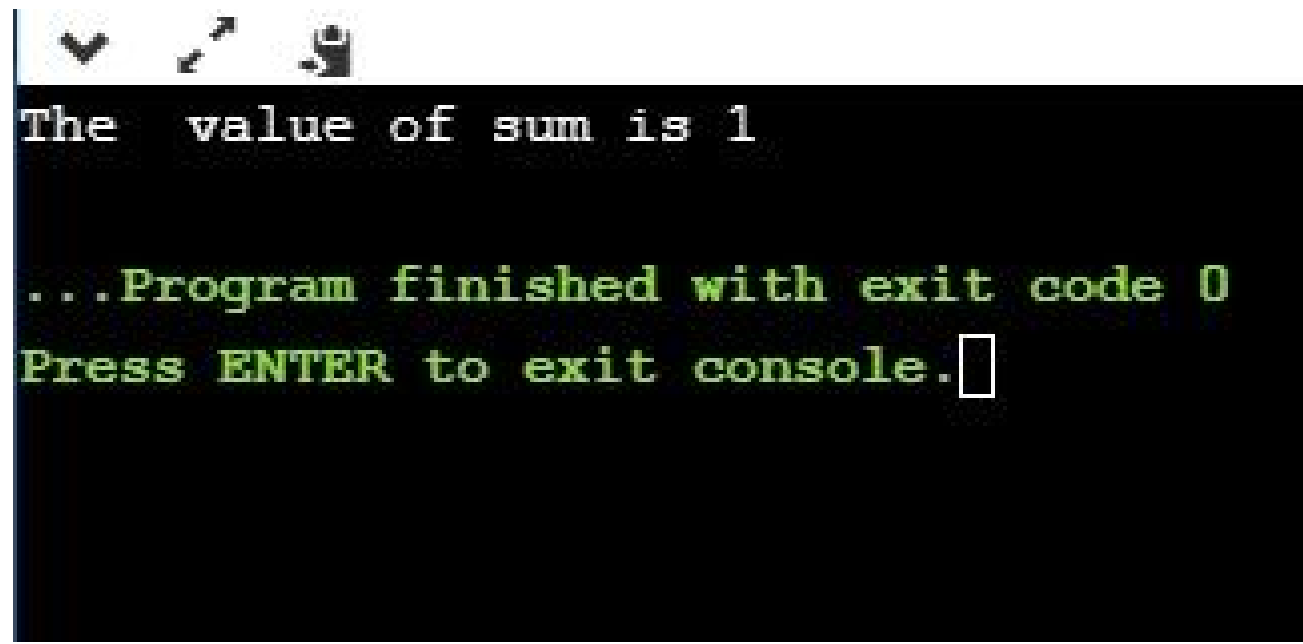
Logical error

The logical error is an error that leads to an undesired output. These errors produce the incorrect output, but they are error-free, known as logical errors. These types of mistakes are mainly done by beginners. The occurrence of these errors mainly depends upon the logical thinking of the developer. If the programmers sound logically good, then there will be fewer chances of these errors.

Let's understand through an example.

```
#include <stdio.h>  
  
int main()  
{  
  
    int sum=0; // variable initialization  
  
    int k=1;  
  
    for(int i=1;i<=10;i++); // logical error, as we put the semicolon after loop  
  
    {  
  
        sum=sum+k;  
  
        k++;  
  
    }  
  
    printf("The value of sum is %d", sum);  
  
    return 0;  
  
}
```

Output

A screenshot of a console window with a black background and white and green text. The text reads: "The value of sum is 1" in white, followed by "...Program finished with exit code 0" and "Press ENTER to exit console." in green. A white cursor is visible at the end of the last line. The window has standard OS icons (minimize, maximize, close) in the top-left corner.

```
The value of sum is 1
...Program finished with exit code 0
Press ENTER to exit console.
```

In the above code, we are trying to print the sum of 10 digits, but we got the wrong output as we put the semicolon (;) after the for loop, so the inner statements of the for loop will not execute. This produces the wrong output.

Semantic error

Semantic errors are the errors that occurred when the statements are not understandable by the compiler.

The following can be the cases for the semantic error:

- Use of a un-initialized variable.

```
int i;
i=i+2;
```

- Type compatibility

```
int b = "javatpoint";
```

- Errors in expressions

```
int a, b, c;
a+b = c;
```

- Array index out of bound

```
int a[10];
a[10] = 34;
```

Let's understand through an example.

```
#include <stdio.h>
```

```

int main()
{
    int a,b,c;

    a=2;

    b=3;

    c=1;

    a+b=c; // semantic error

    return 0;
}

```

In the above code, we use the statement `a+b=c`, which is incorrect as we cannot use the two operands on the left-side.

Output



The screenshot shows a window titled "input" with a dark background. It displays a compilation error message in red text: "Compilation failed due to following error(s)." Below this, it shows the error details: "main.c: In function 'main':" followed by "main.c:17:6: error: lvalue required as left operand of assignment". The code snippet "a+b=c;" is shown with a caret (^) pointing to the equals sign, indicating the location of the error.

Compile time vs Runtime

Compile-time and Runtime are the two programming terms used in the software development. Compile-time is the time at which the source code is converted into an executable code while the run time is the time at which the executable code is started running. Both the compile-time and runtime refer to different types of error.

Compile-time errors

Compile-time errors are the errors that occurred when we write the wrong syntax. If we write the wrong syntax or semantics of any programming language, then the compile-time errors will be thrown by the compiler. The compiler will not allow to run the program until all the errors are removed from the program. When all the errors are removed from the program, then the compiler will generate the executable file.

The compile-time errors can be:

- Syntax errors
- Semantic errors

Syntax errors

When the programmer does not follow the syntax of any programming language, then the compiler will throw the syntax error.

For example,

```
int a, b:
```

The above declaration generates the compile-time error as in C, every statement ends with the semicolon, but we put a colon (:) at the end of the statement.

Semantic errors

The semantic errors exist when the statements are not meaningful to the compiler.

For example,

```
a+b=c;
```

The above statement throws a compile-time errors. In the above statement, we are assigning the value of 'c' to the summation of 'a' and 'b' which is not possible in C programming language as it can contain only one variable on the left of the assignment operator while right of the assignment operator can contain more than one variable.

The above statement can be re-written as:

```
c=a+b;
```

Runtime errors

The runtime errors are the errors that occur during the execution and after compilation. The examples of runtime errors are division by zero, etc. These errors are not easy to detect as the compiler does not point to these errors.

Let's look at the differences between compile-time and runtime:

Compile-time	Runtime
The compile-time errors are the errors which are produced at the compile-time, and they are detected by the compiler.	The runtime errors are the errors which are not generated by the compiler and produce an unpredictable result at the execution time.
In this case, the compiler prevents the code from execution if it detects an error in the program.	In this case, the compiler does not detect the error, so it cannot prevent the code from the execution.
It contains the syntax and semantic errors such as missing semicolon at the end of the statement.	It contains the errors such as division by zero, determining the square root of a negative number.

Example of Compile-time error

```
#include <stdio.h>

int main()

{

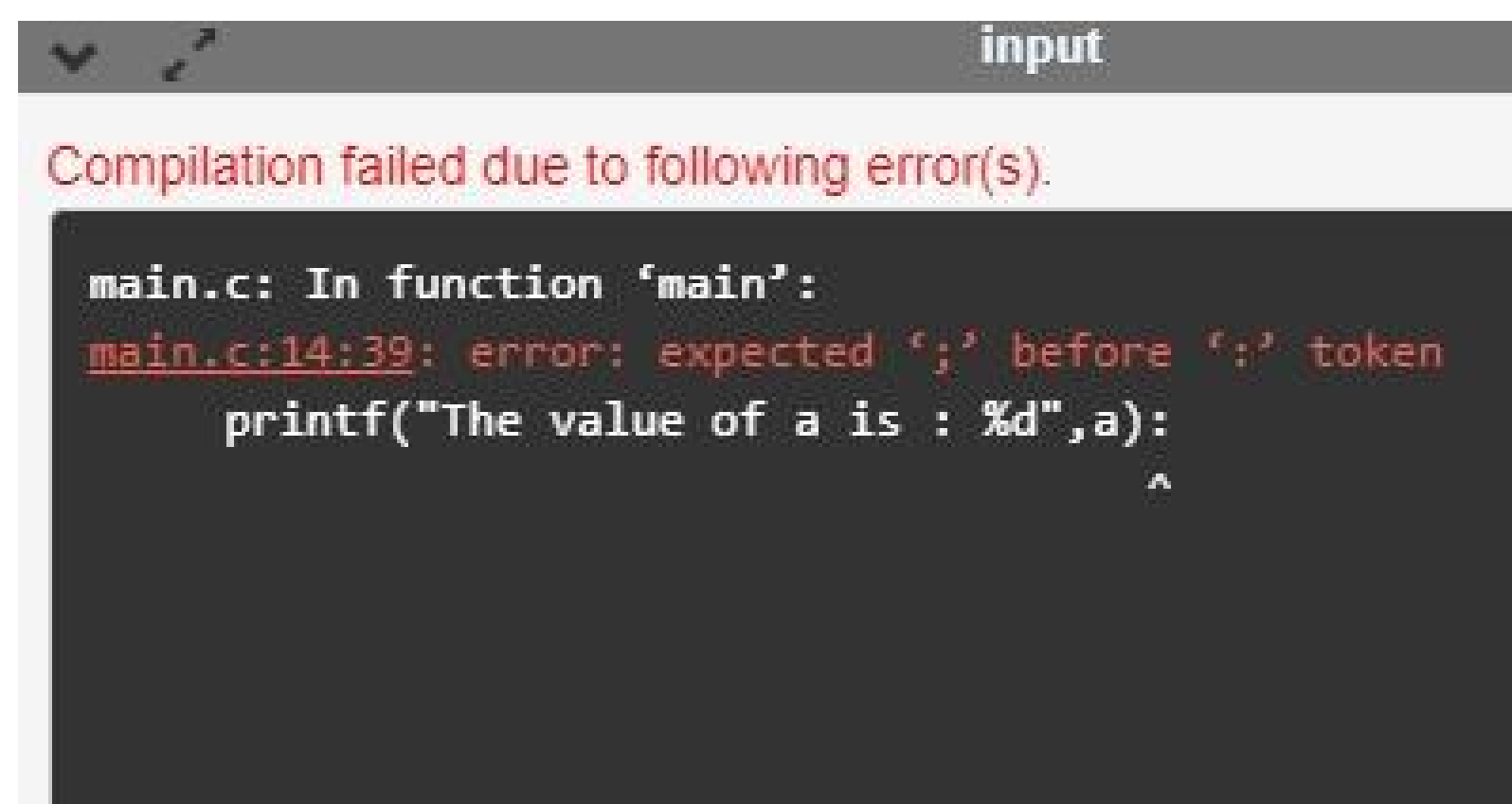
    int a=20;

    printf("The value of a is : %d",a):
```

```
    return 0;
}
```

In the above code, we have tried to print the value of 'a', but it throws an error. We put the colon at the end of the statement instead of a semicolon, so this code generates a compile-time error.

Output



Example of runtime error

```
#include <stdio.h>

int main()
{
    int a=20;

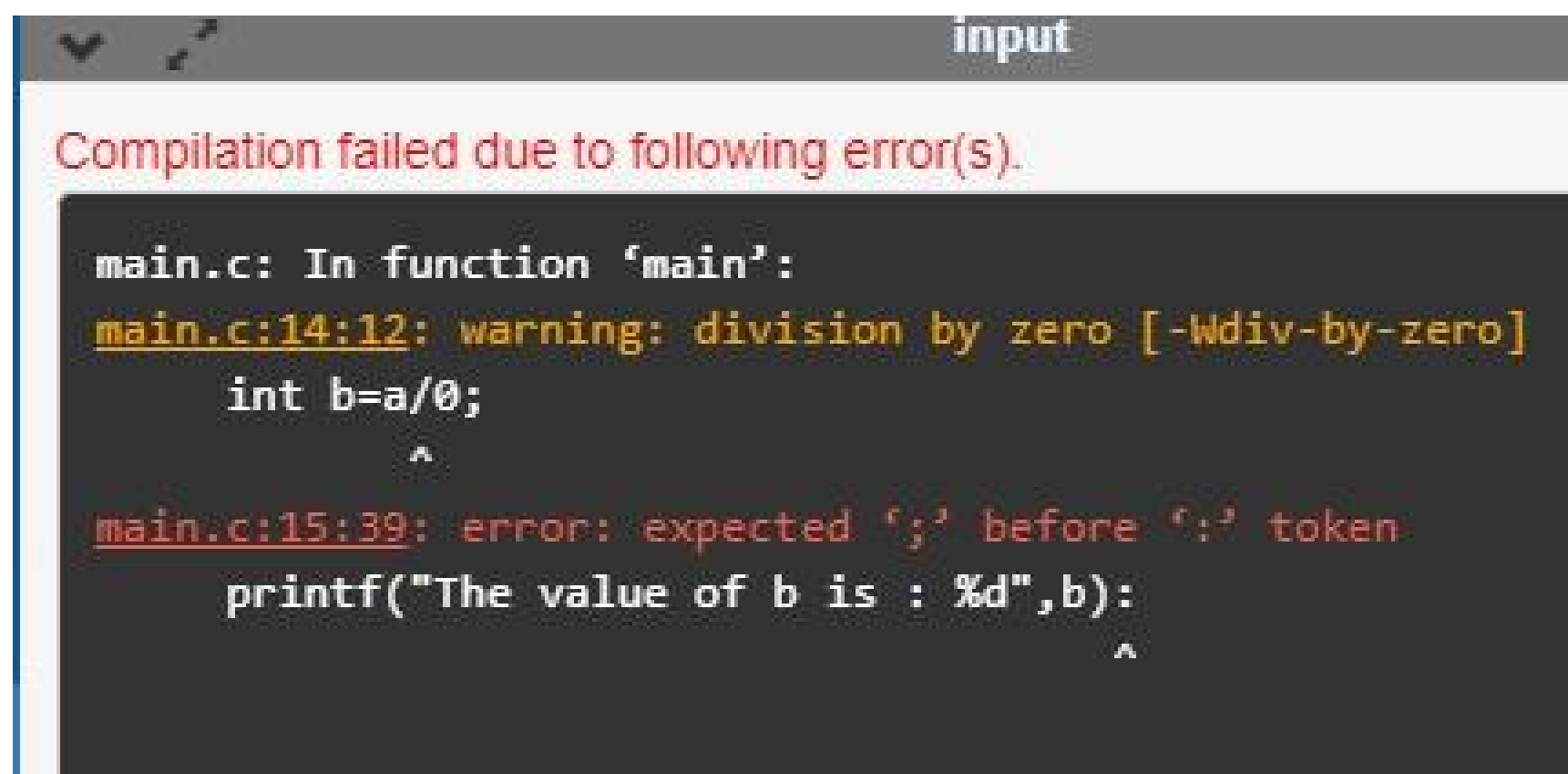
    int b=a/0; // division by zero

    printf("The value of b is : %d",b):

    return 0;
}
```

In the above code, we try to divide the value of 'b' by zero, and this throws a runtime error.

Output



```
input
Compilation failed due to following error(s).

main.c: In function 'main':
main.c:14:12: warning: division by zero [-Wdiv-by-zero]
    int b=a/0;
           ^
main.c:15:39: error: expected ';' before ':' token
    printf("The value of b is : %d",b):
                                   ^
```

Conditional Operator in C

The conditional operator is also known as a **ternary operator**. The conditional statements are the decision-making statements which depends upon the output of the expression. It is represented by two symbols, i.e., '?' and ':'.

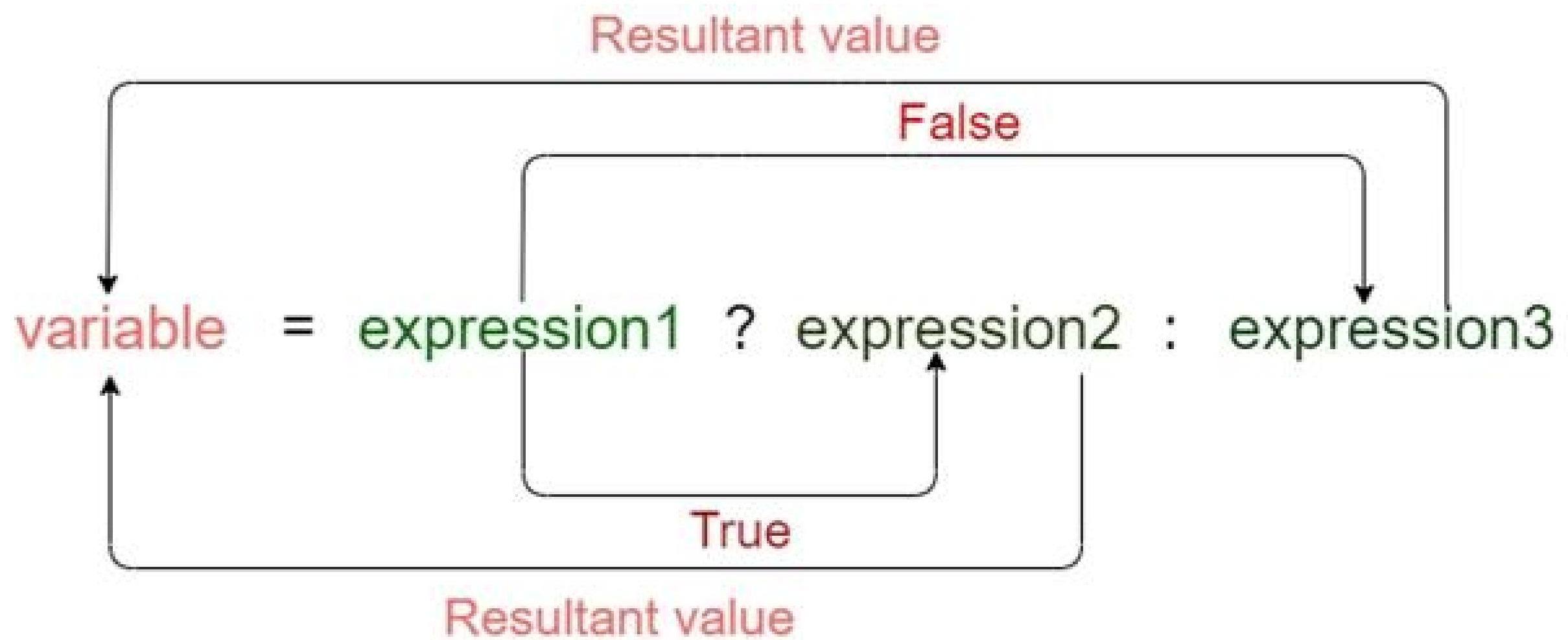
As conditional operator works on three operands, so it is also known as the ternary operator.

The behavior of the conditional operator is similar to the 'if-else' statement as 'if-else' statement is also a decision-making statement.

Syntax of a conditional operator

Expression1? expression2: expression3;

The pictorial representation of the above syntax is shown below:



Meaning of the above syntax.

- In the above syntax, the `expression1` is a Boolean condition that can be either true or false value.
- If the `expression1` results into a true value, then the `expression2` will execute.
- The `expression2` is said to be true only when it returns a non-zero value.
- If the `expression1` returns false value then the `expression3` will execute.
- The `expression3` is said to be false only when it returns zero value.

Let's understand the ternary or conditional operator through an example.

```
#include <stdio.h>

int main()
{
    int age; // variable declaration

    printf("Enter your age");

    scanf("%d",&age); // taking user input for age variable

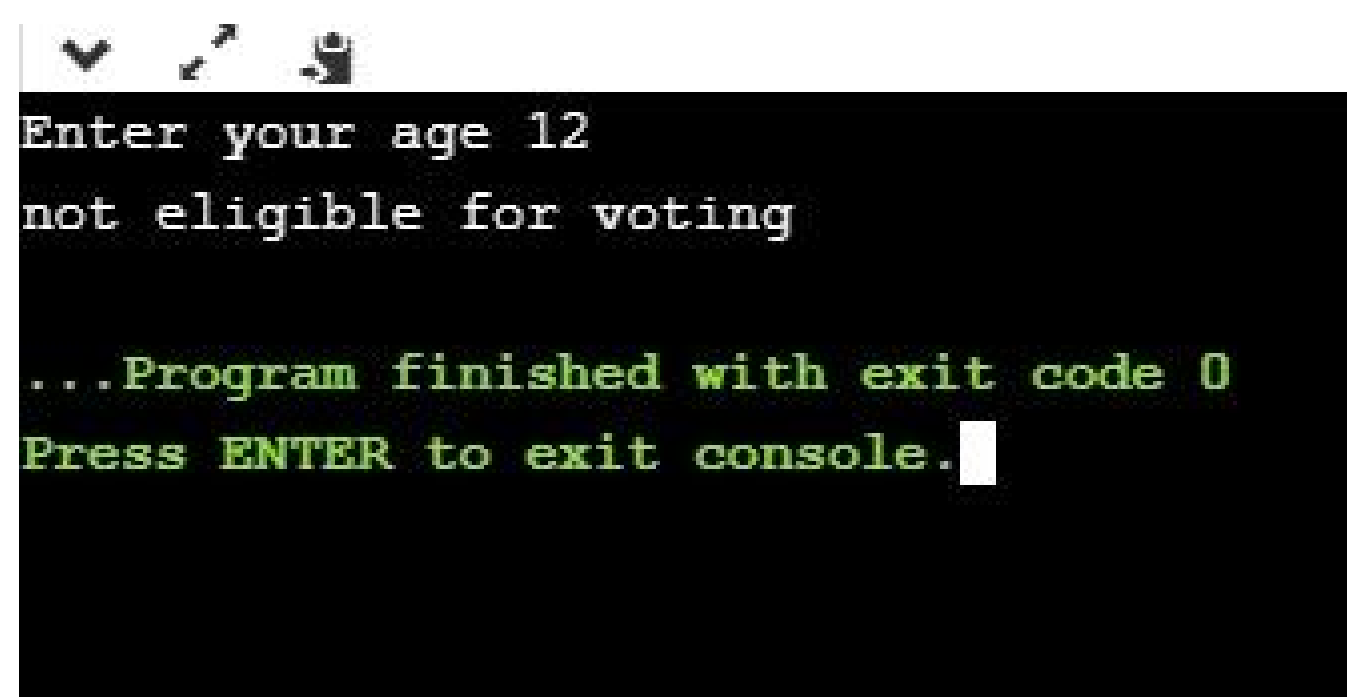
    (age>=18)? (printf("eligible for voting")) : (printf("not eligible for voting")); // conditional operator
```

```
    return 0;  
}
```

In the above code, we are taking input as the 'age' of the user. After taking input, we have applied the condition by using a conditional operator. In this condition, we are checking the age of the user. If the age of the user is greater than or equal to 18, then the statement1 will execute, i.e., (printf("eligible for voting")) otherwise, statement2 will execute, i.e., (printf("not eligible for voting")).

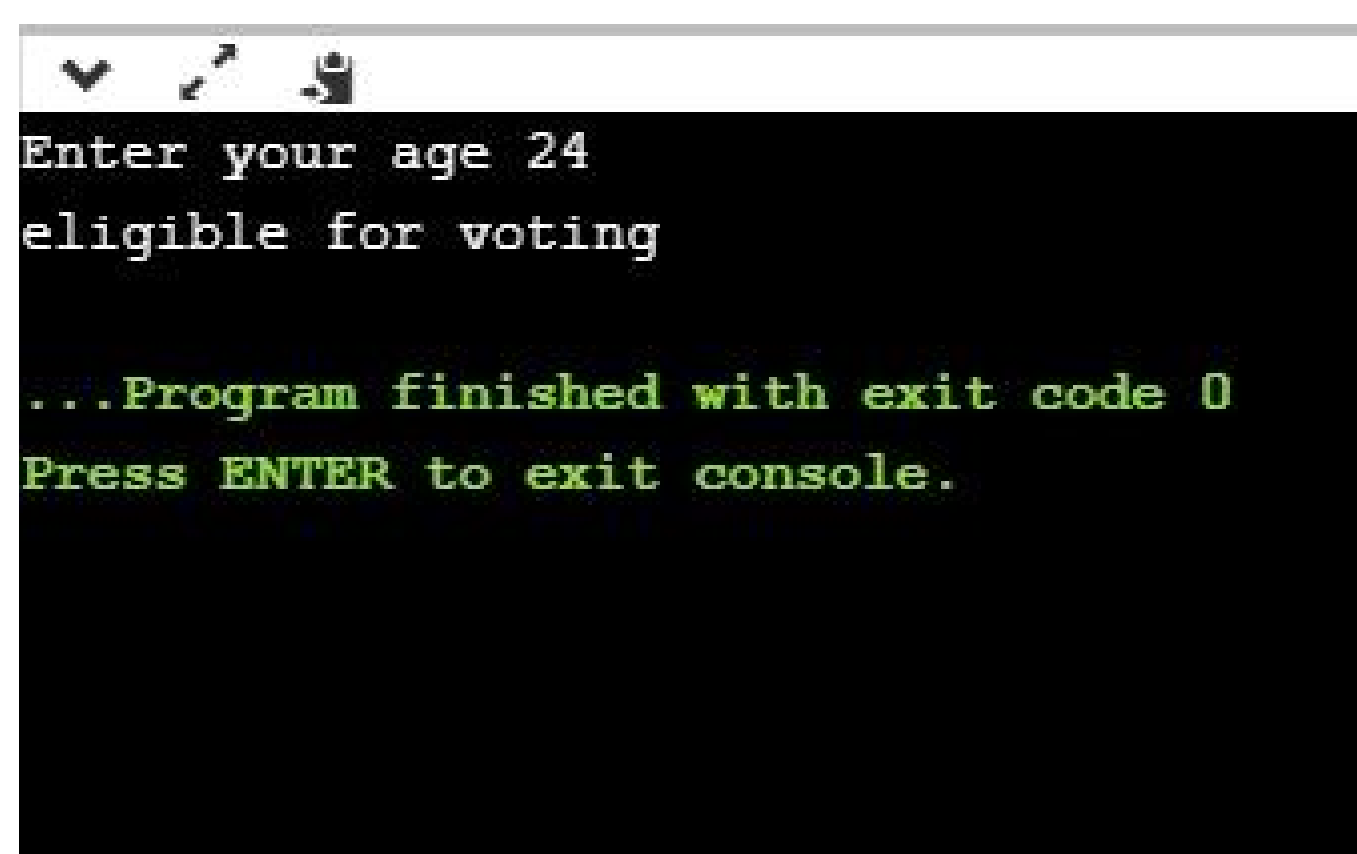
Let's observe the output of the above program.

If we provide the age of user below 18, then the output would be:



```
Enter your age 12  
not eligible for voting  
  
...Program finished with exit code 0  
Press ENTER to exit console.
```

If we provide the age of user above 18, then the output would be:



```
Enter your age 24  
eligible for voting  
  
...Program finished with exit code 0  
Press ENTER to exit console.
```

As we can observe from the above two outputs that if the condition is true, then the statement1 is executed; otherwise, statement2 will be executed.

Till now, we have observed that how conditional operator checks the condition and based on condition, it executes the statements. Now, we will see how a conditional operator is used to assign the value to a variable.

Let's understand this scenario through an example.

```
#include <stdio.h>

int main()
{
    int a=5,b; // variable declaration

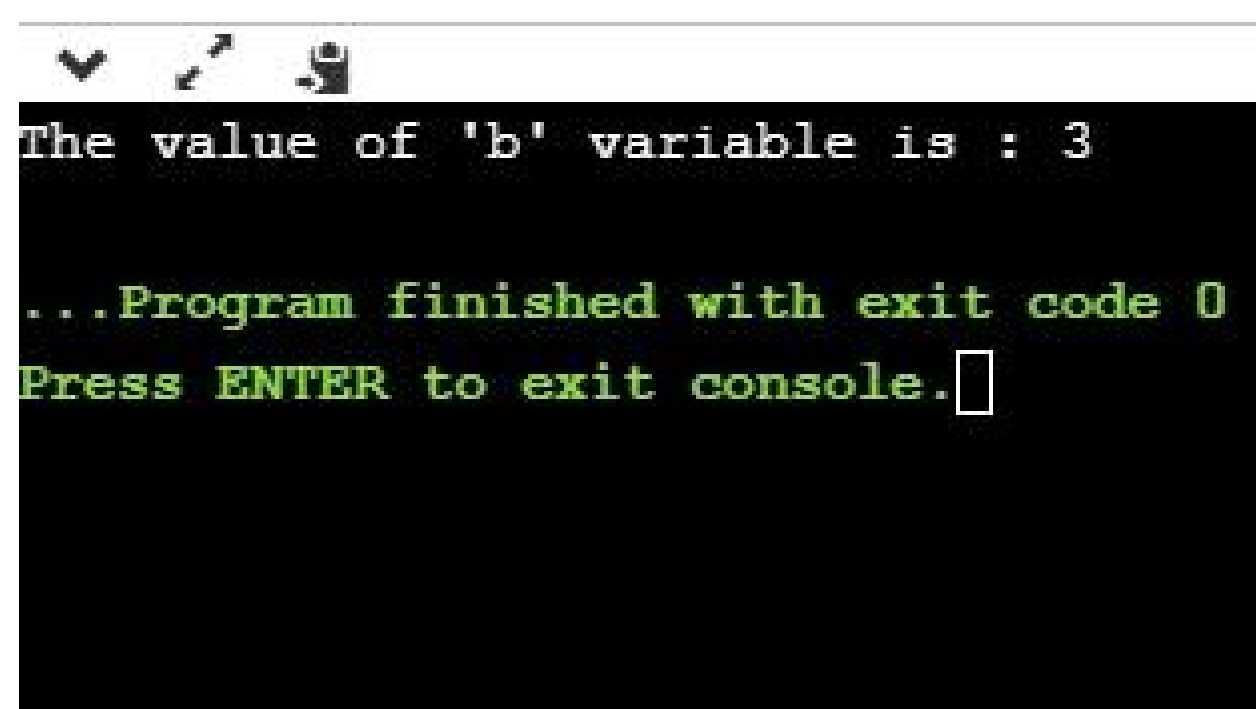
    b=((a==5)?(3):(2)); // conditional operator

    printf("The value of 'b' variable is : %d",b);

    return 0;
}
```

In the above code, we have declared two variables, i.e., 'a' and 'b', and assign 5 value to the 'a' variable. After the declaration, we are assigning value to the 'b' variable by using the conditional operator. If the value of 'a' is equal to 5 then 'b' is assigned with a 3 value otherwise 2.

Output



```
The value of 'b' variable is : 3

...Program finished with exit code 0
Press ENTER to exit console.
```

The above output shows that the value of 'b' variable is 3 because the value of 'a' variable is equal to 5.

As we know that the behavior of conditional operator and 'if-else' is similar but they have some differences. Let's look at their differences.

- A conditional operator is a single programming statement, while the 'if-else' statement is a programming block in which statements come under the parenthesis.
- A conditional operator can also be used for assigning a value to the variable, whereas the 'if-else' statement cannot be used for the assignment purpose.
- It is not useful for executing the statements when the statements are multiple, whereas the 'if-else' statement proves more suitable when executing multiple statements.
- The nested ternary operator is more complex and cannot be easily debugged, while the nested 'if-else' statement is easy to read and maintain.

Bitwise Operator in C

The bitwise operators are the operators used to perform the operations on the data at the bit-level. When we perform the bitwise operations, then it is also known as bit-level programming. It consists of two digits, either 0 or 1. It is mainly used in numerical computations to make the calculations faster.

We have different types of bitwise operators in the C programming language. The following is the list of the bitwise operators:

Operator	Meaning of operator
&	Bitwise AND operator
	Bitwise OR operator

$\wedge$	Bitwise exclusive OR operator
$\sim$	One's complement operator (unary operator)
$\ll$	Left shift operator
$\gg$	Right shift operator

Let's look at the truth table of the bitwise operators.

X	Y	X&Y	X Y	X^Y
0	0	0	0	0
0	1	0	1	1
1	0	0	1	1
1	1	1	1	1

Bitwise AND operator

Bitwise AND operator is denoted by the single ampersand sign (&). Two integer operands are written on both sides of the (&) operator. If the corresponding bits of both the operands are 1, then the output of the bitwise AND operation is 1; otherwise, the output would be 0.

For example,

We have two variables a and b.

a =6;

b=4;

The binary representation of the above two variables are given below:

a = 0110

b = 0100

When we apply the bitwise AND operation in the above two variables, i.e., a&b, the output would be:

Result = 0100

As we can observe from the above result that bits of both the variables are compared one by one. If the bit of both the variables is 1 then the output would be 1, otherwise 0.

Let's understand the bitwise AND operator through the program.

```
#include <stdio.h>

int main()
{
    int a=6, b=14; // variable declarations

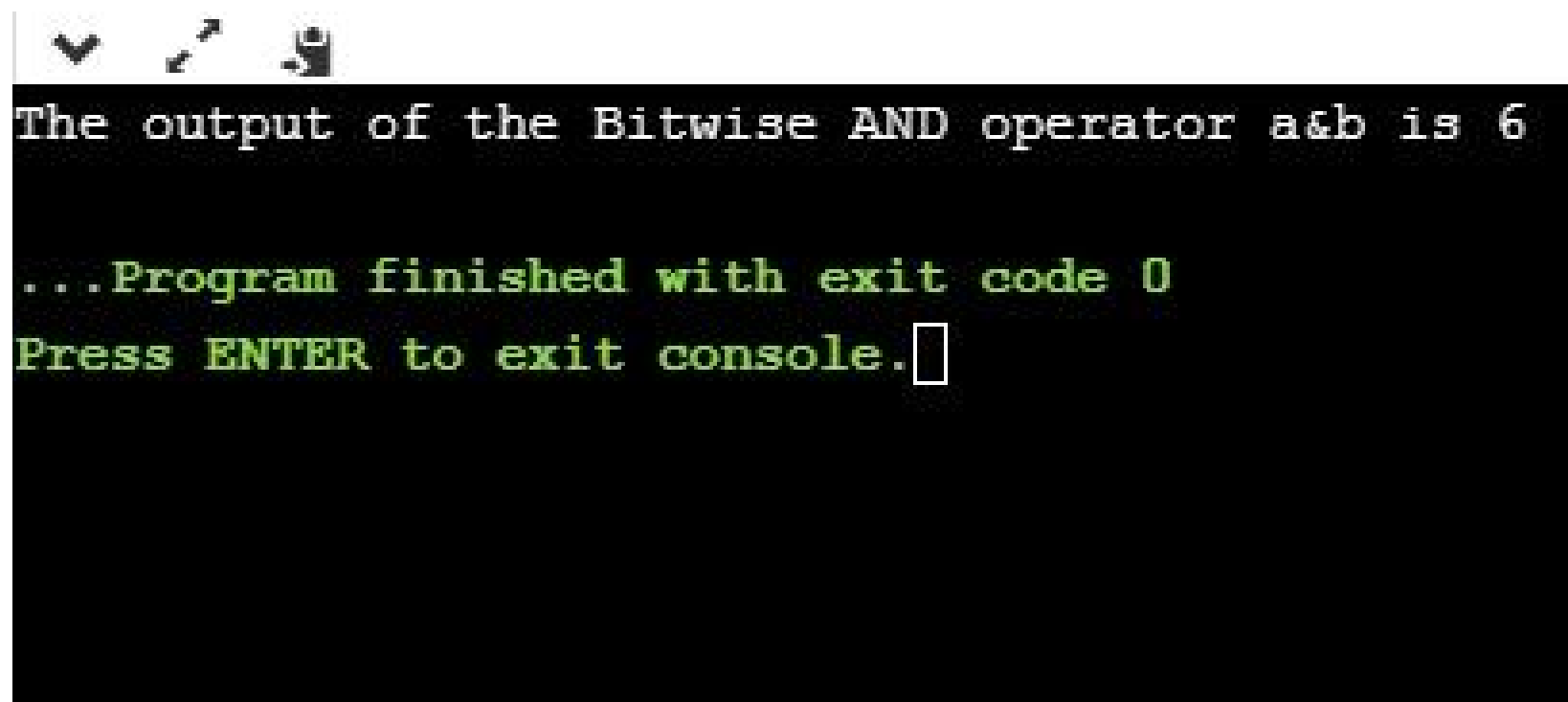
    printf("The output of the Bitwise AND operator a&b is %d",a&b);

    return 0;
}
```

In the above code, we have created two variables, i.e., 'a' and 'b'. The values of 'a' and 'b' are 6 and 14 respectively. The binary value of 'a' and 'b' are 0110 and 1110, respectively. When we apply the AND operator between these two variables,

a AND b = 0110 & 1110 = 0110

Output



```
✓ ↗ 📄  
The output of the Bitwise AND operator a&b is 6  
...Program finished with exit code 0  
Press ENTER to exit console.█
```

Bitwise OR operator

The bitwise OR operator is represented by a single vertical sign ($|$). Two integer operands are written on both sides of the ($|$) symbol. If the bit value of any of the operand is 1, then the output would be 1, otherwise 0.

For example,

We consider two variables,

$a = 23;$

$b = 10;$

The binary representation of the above two variables would be:

$a = 0001\ 0111$

$b = 0000\ 1010$

When we apply the bitwise OR operator in the above two variables, i.e., $a|b$, then the output would be:

Result = $0001\ 1111$

As we can observe from the above result that the bits of both the operands are compared one by one; if the value of either bit is 1, then the output would be 1 otherwise 0.

Let's understand the bitwise OR operator through a program.

```
#include <stdio.h>
```

```

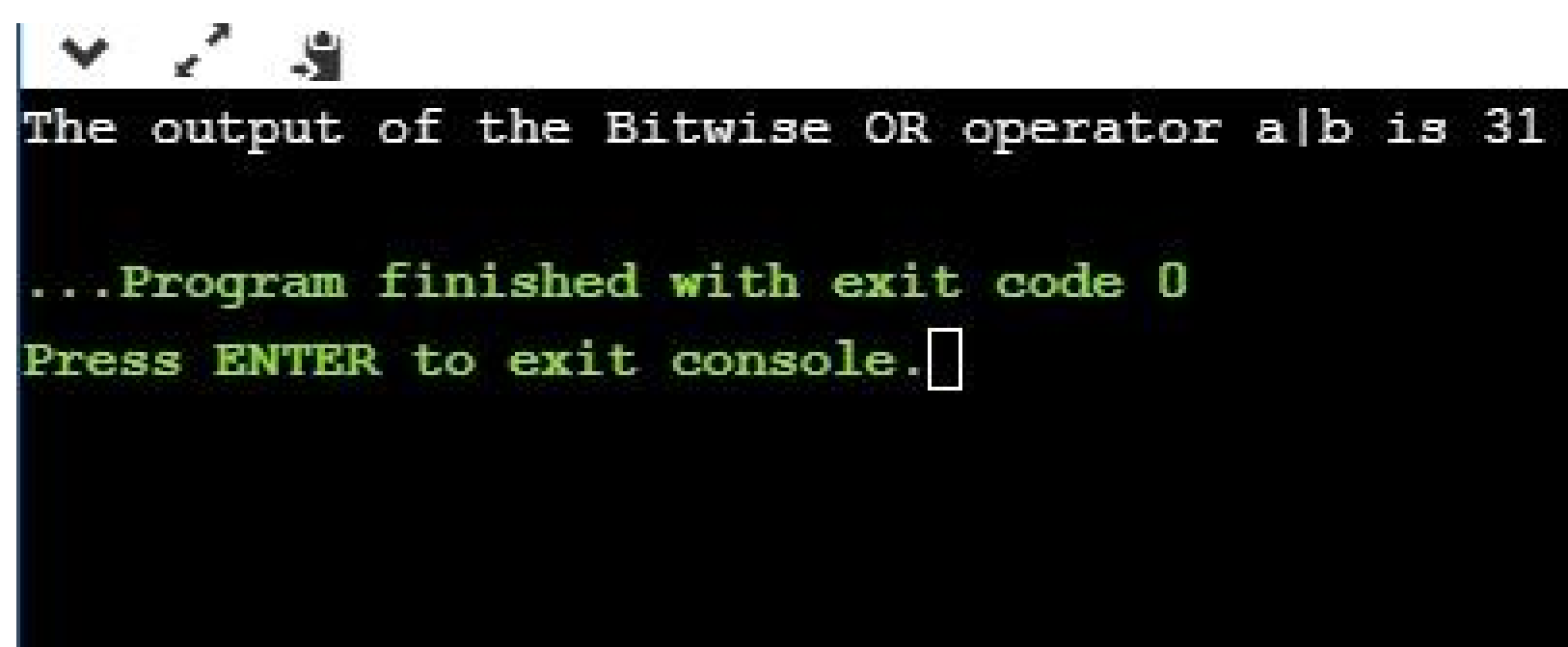
int main()
{
    int a=23,b=10; // variable declarations

    printf("The output of the Bitwise OR operator a|b is %d",a|b);

    return 0;
}

```

Output



```

The output of the Bitwise OR operator a|b is 31

...Program finished with exit code 0
Press ENTER to exit console.

```

Bitwise exclusive OR operator

Bitwise exclusive OR operator is denoted by (^) symbol. Two operands are written on both sides of the exclusive OR operator. If the corresponding bit of any of the operand is 1 then the output would be 1, otherwise 0.

For example,

We consider two variables a and b,

a = 12;

b = 10;

The binary representation of the above two variables would be:

a = 0000 1100

b = 0000 1010

When we apply the bitwise exclusive OR operator in the above two variables ($a \oplus b$), then the result would be:

Result = 0000 1110

As we can observe from the above result that the bits of both the operands are compared one by one; if the corresponding bit value of any of the operand is 1, then the output would be 1 otherwise 0.

Let's understand the bitwise exclusive OR operator through a program.

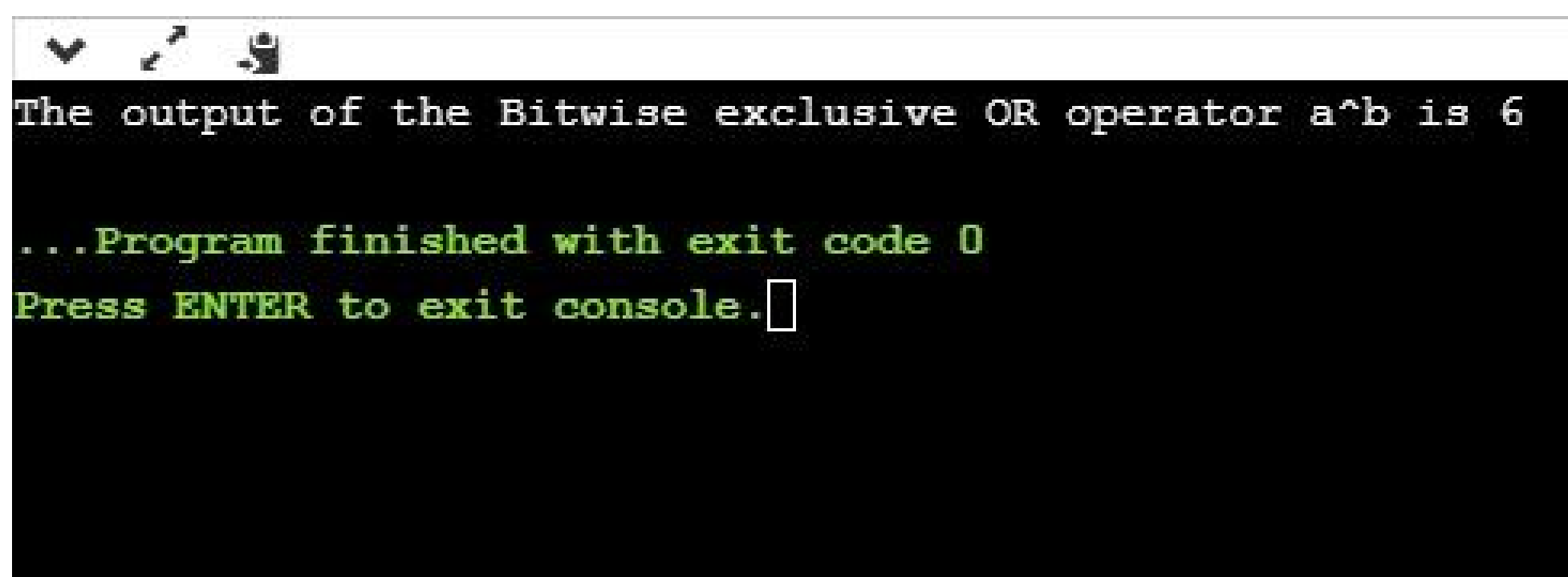
```
#include <stdio.h>

int main()
{
    int a=12,b=10; // variable declarations

    printf("The output of the Bitwise exclusive OR operator a^b is %d",a^b);

    return 0;
}
```

Output

A screenshot of a console window with a black background and white text. The text displays the output of the program: "The output of the Bitwise exclusive OR operator a^b is 6". Below this, it shows "...Program finished with exit code 0" and "Press ENTER to exit console." with a cursor. The window has standard OS icons at the top.

Bitwise complement operator

The bitwise complement operator is also known as one's complement operator. It is represented by the symbol tilde ($\sim$). It takes only one operand or variable and

performs complement operation on an operand. When we apply the complement operation on any bits, then 0 becomes 1 and 1 becomes 0.

For example,

If we have a variable named 'a',

a = 8;

The binary representation of the above variable is given below:

a = 1000

When we apply the bitwise complement operator to the operand, then the output would be:

Result = 0111

As we can observe from the above result that if the bit is 1, then it gets changed to 0 else 1.

Let's understand the complement operator through a program.

```
#include <stdio.h>

int main()
{
    int a=8; // variable declarations

    printf("The output of the Bitwise complement operator ~a is %d",~a);

    return 0;
}
```

Output

```
✓ ↗ 📄  
The output of the Bitwise complement operator ~a is -9  
...Program finished with exit code 0  
Press ENTER to exit console.□
```

Bitwise shift operators

Two types of bitwise shift operators exist in C programming. The bitwise shift operators will shift the bits either on the left-side or right-side. Therefore, we can say that the bitwise shift operator is divided into two categories:

- Left-shift operator
- Right-shift operator

Left-shift operator

It is an operator that shifts the number of bits to the left-side.

Syntax of the left-shift operator is given below:

Operand << n

Where,

Operand is an integer expression on which we apply the left-shift operation.

n is the number of bits to be shifted.

In the case of Left-shift operator, 'n' bits will be shifted on the left-side. The 'n' bits on the left side will be popped out, and 'n' bits on the right-side are filled with 0.

For example,

Suppose we have a statement:

```
int a = 5;
```

The binary representation of 'a' is given below:

a = 0101

If we want to left-shift the above representation by 2, then the statement would be:

a << 2;

0101<<2 = 00010100

Let's understand through a program.

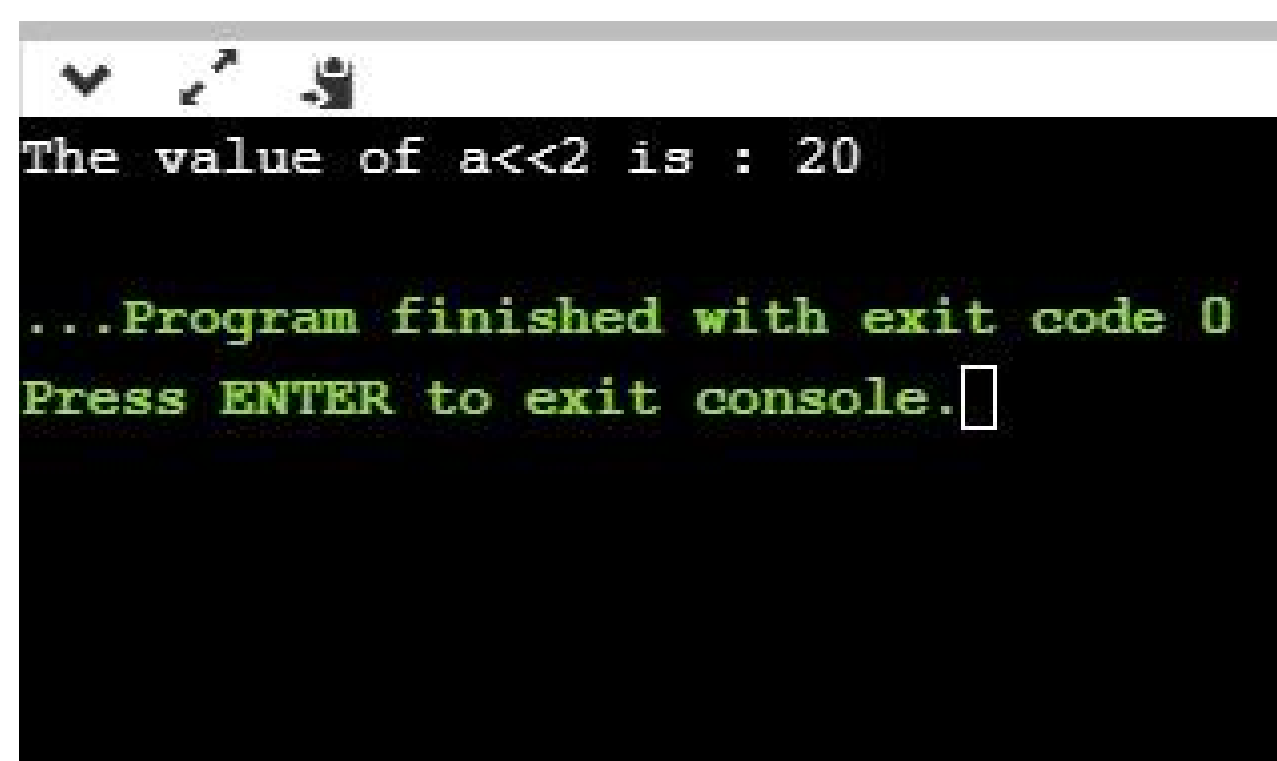
```
#include <stdio.h>

int main()
{
    int a=5; // variable initialization

    printf("The value of a<<2 is : %d ", a<<2);

    return 0;
}
```

Output



```
The value of a<<2 is : 20

...Program finished with exit code 0
Press ENTER to exit console.
```

Right-shift operator

It is an operator that shifts the number of bits to the right side.

Syntax of the right-shift operator is given below:

Operand >> n;

Where,

Operand is an integer expression on which we apply the right-shift operation.

N is the number of bits to be shifted.

In the case of the right-shift operator, 'n' bits will be shifted on the right-side. The 'n' bits on the right-side will be popped out, and 'n' bits on the left-side are filled with 0.

For example,

Suppose we have a statement,

```
int a = 7;
```

The binary representation of the above variable would be:

a = 0111

If we want to right-shift the above representation by 2, then the statement would be:

a>>2;

0000 0111 >> 2 = 0000 0001

Let's understand through a program.

```
#include <stdio.h>

int main()
{
    int a=7; // variable initialization

    printf("The value of a>>2 is : %d ", a>>2);

    return 0;
}
```

Output

```

The value of a>>2 is : 1

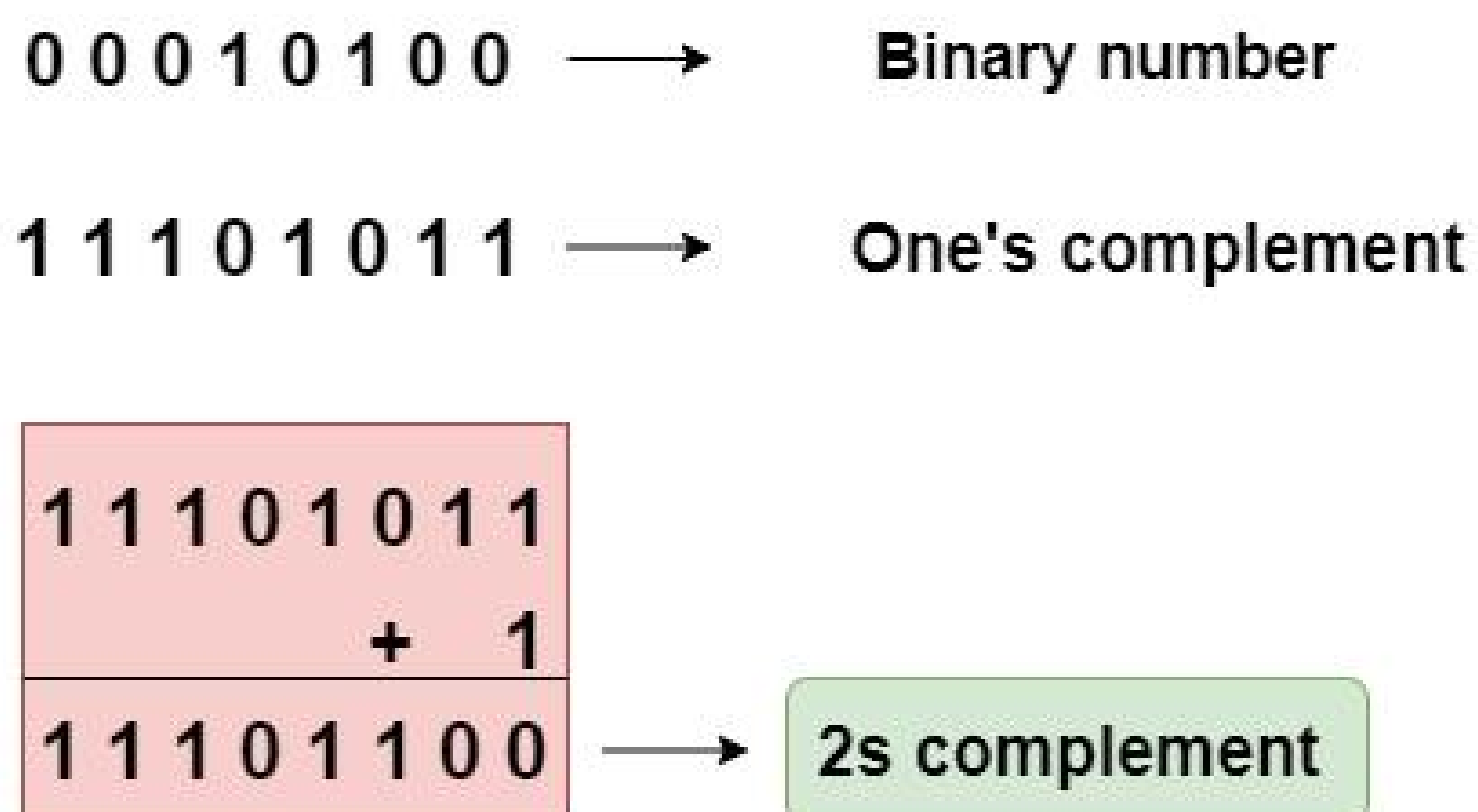
...Program finished with exit code 0
Press ENTER to exit console.

```

2s complement in C?

The 2s complement in C is generated from the 1s complement in C. As we know that the 1s complement of a binary number is created by transforming bit 1 to 0 and 0 to 1; the 2s complement of a binary number is generated by adding one to the 1s complement of a binary number.

In short, we can say that the 2s complement in C is defined as the sum of the one's complement in C and one.



In the above figure, the binary number is equal to 00010100, and its one's complement is calculated by transforming the bit 1 to 0 and 0 to 1 vice versa. Therefore, one's complement becomes 11101011. After calculating one's complement, we calculate the two's complement by adding 1 to the one's complement, and its result is 11101100.

Let's create a program of 2s complement.

```

#include <stdio.h>

int main()
{
    int n; // variable declaration

    printf("Enter the number of bits do you want to enter :");

    scanf("%d",&n);

    char binary[n+1]; // binary array declaration;

    char onescomplement[n+1]; // onescomplement array declaration

    char twoscomplement[n+1]; // twoscomplement array declaration

    int carry=1; // variable initialization

    printf("\nEnter the binary number : ");

    scanf("%s", binary);

    printf("%s", binary);

    printf("\nThe ones complement of the binary number is :");

    // Finding onescomplement in C

    for(int i=0;i<n;i++)
    {
        if(binary[i]=='0')

            onescomplement[i]='1';

        else if(binary[i]=='1')

            onescomplement[i]='0';

    }

    onescomplement[n]='\0';

    printf("%s",onescomplement);

    printf("\nThe twos complement of a binary number is : ");

```

```

// Finding twoscomplement in C

for(int i=n-1; i>=0; i--)
{
    if(onescomplement[i] == '1' && carry == 1)
    {
        twoscomplement[i] = '0';
    }
    else if(onescomplement[i] == '0' && carry == 1)
    {
        twoscomplement[i] = '1';
        carry = 0;
    }
    else
    {
        twoscomplement[i] = onescomplement[i];
    }
}

twoscomplement[n]='\0';

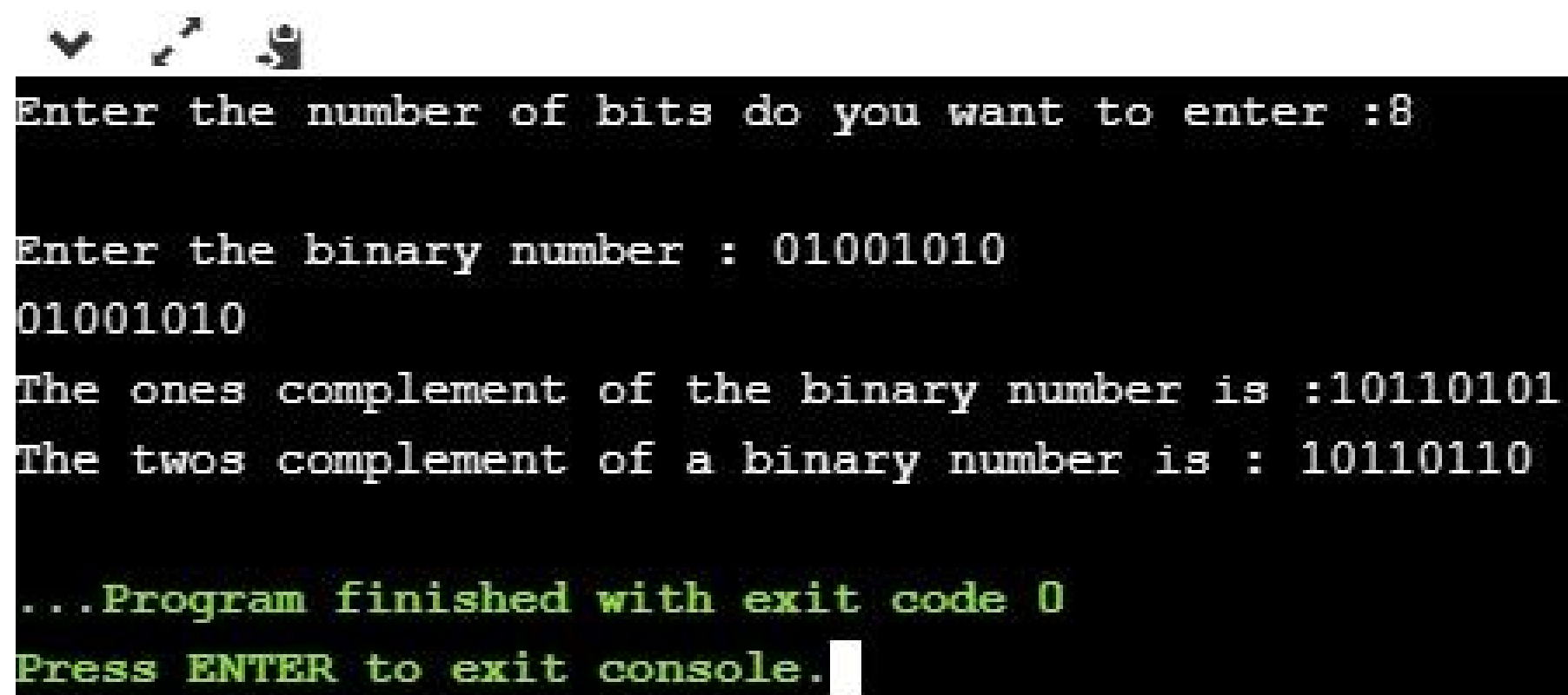
printf("%s",twoscomplement);

return 0;

}

```

Output



```

Enter the number of bits do you want to enter :8

Enter the binary number : 01001010
01001010
The ones complement of the binary number is :10110101
The twos complement of a binary number is : 10110110

...Program finished with exit code 0
Press ENTER to exit console.

```

Analysis of the above program,

- First, we input the number of bits, and it gets stored in the 'n' variable.
- After entering the number of bits, we declare character array, i.e., `char binary[n+1]`, which holds the binary number. The 'n' is the number of bits which we entered in the previous step; it basically defines the size of the array.
- We declare two more arrays, i.e., `onescomplement[n+1]`, and `twoscomplement[n+1]`. The `onescomplement[n+1]` array holds the ones complement of a binary number while the `twoscomplement[n+1]` array holds the two's complement of a binary number.
- Initialize the `carry` variable and assign 1 value to this variable.
- After declarations, we input the binary number.
- Now, we simply calculate the one's complement of a binary number. To do this, we create a **loop** that iterates throughout the binary array, `for(int i=0;i<n;i++)`. In for loop, the condition is checked whether the bit is 1 or 0. If the bit is 1 then `onescomplement[i]=0` else `onescomplement[i]=1`. In this way, one's complement of a binary number is generated.

- After calculating one's complement, we generate the 2s complement of a binary number. To do this, we create a **loop** that iterates from the last element to the starting element. In for loop, we have three conditions:
 - If the bit of onescomplement[i] is 1 and the value of carry is 1 then we put 0 in twocomplement[i].
 - If the bit of onescomplement[i] is 0 and the value of carry is 1 then we put 1 in twocomplement[i] and 0 in carry.
 - If the above two conditions are false, then onescomplement[i] is equal to twocomplement[i].

C Expressions

An expression is a formula in which operands are linked to each other by the use of operators to compute a value. An operand can be a function reference, a variable, an array element or a constant.

Let's see an example:

a-b;

In the above expression, minus character (-) is an operator, and a, and b are the two operands.

There are four types of expressions exist in C:

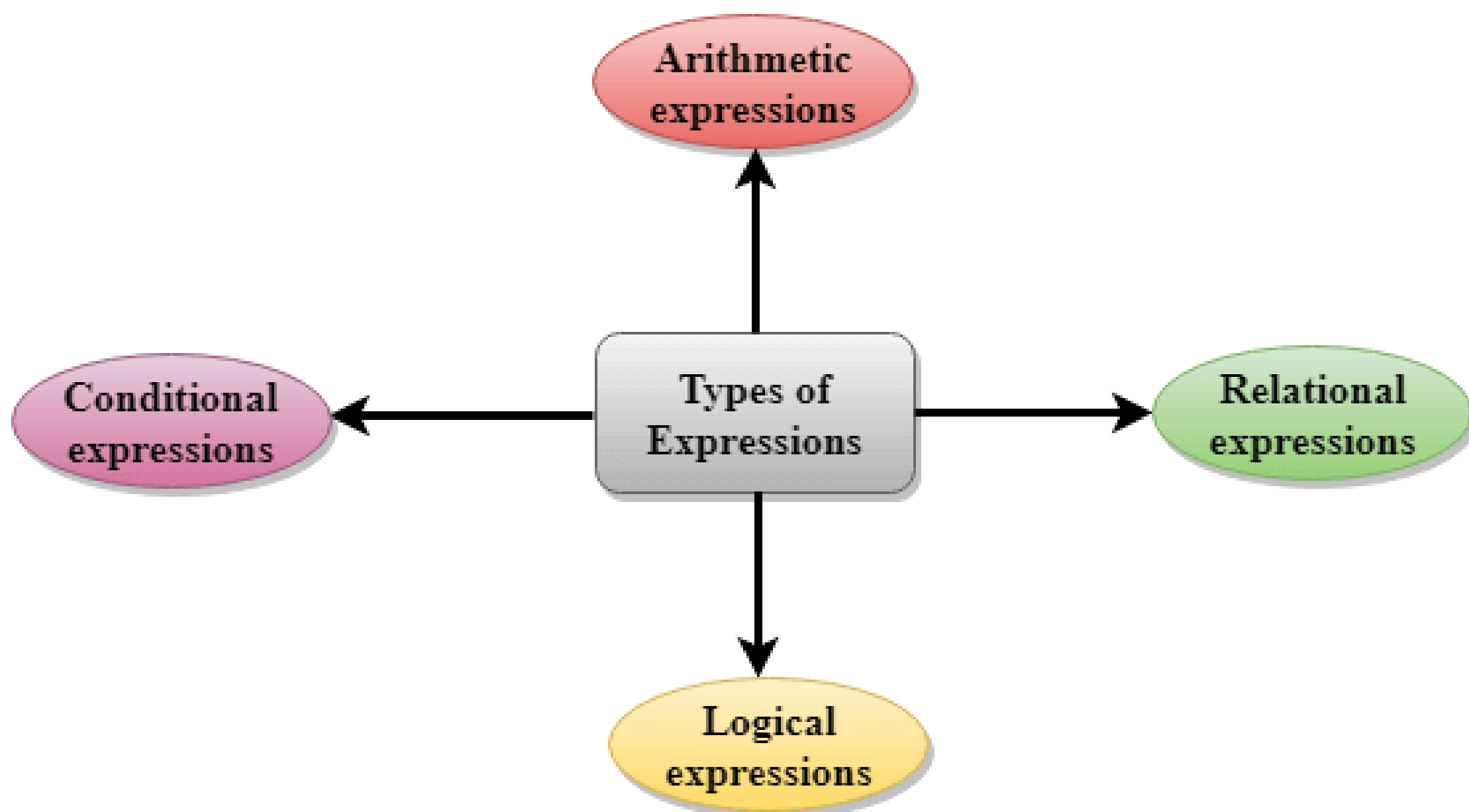
- Arithmetic expressions
- Relational expressions
- Logical expressions
- Conditional expressions

Each type of expression takes certain types of operands and uses a specific set of operators. Evaluation of a particular expression produces a specific value.

For example:

$$x = 9/2 + a-b;$$

The entire above line is a statement, not an expression. The portion after the equal is an expression.



Arithmetic Expressions

An arithmetic expression is an expression that consists of operands and arithmetic operators. An arithmetic expression computes a value of type int, float or double.

When an expression contains only integral operands, then it is known as pure integer expression when it contains only real operands, it is known as pure real expression, and when it contains both integral and real operands, it is known as mixed mode expression.

Evaluation of Arithmetic Expressions

The expressions are evaluated by performing one operation at a time. The

precedence and associativity of operators decide the order of the evaluation of individual operations.

When individual operations are performed, the following cases can be happened:

- When both the operands are of type integer, then arithmetic will be performed, and the result of the operation would be an integer value. For example, $3/2$ will yield 1 not 1.5 as the fractional part is ignored.
- When both the operands are of type float, then arithmetic will be performed, and the result of the operation would be a real value. For example, $2.0/2.0$ will yield 1.0, not 1.
- If one operand is of type integer and another operand is of type real, then the mixed arithmetic will be performed. In this case, the first operand is converted into a real operand, and then arithmetic is performed to produce the real value. For example, $6/2.0$ will yield 3.0 as the first value of 6 is converted into 6.0 and then arithmetic is performed to produce 3.0.

Let's understand through an example.

$$6*2/(2+1*2/3+6)+8*(8/4)$$

Evaluation of expression	Description of each operation
$6*2/(2+1*2/3+6)+8*(8/4)$	An expression is given.
$6*2/(2+2/3+6)+8*(8/4)$	2 is multiplied by 1, giving value 2.
$6*2/(2+0+6)+8*(8/4)$	2 is divided by 3, giving value 0.
$6*2/8+8*(8/4)$	2 is added to 6, giving value 8.

$6 * 2 / 8 + 8 * 2$	8 is divided by 4, giving value 2.
$12 / 8 + 8 * 2$	6 is multiplied by 2, giving value 12.
$1 + 8 * 2$	12 is divided by 8, giving value 1.
$1 + 16$	8 is multiplied by 2, giving value 16.
17	1 is added to 16, giving value 17.

Relational Expressions

- A relational expression is an expression used to compare two operands.
- It is a condition which is used to decide whether the action should be taken or not.
- In relational expressions, a numeric value cannot be compared with the string value.
- The result of the relational expression can be either zero or non-zero value. Here, the zero value is equivalent to a false and non-zero value is equivalent to true.

Relational Expression	Description
$x \% 2 == 0$	This condition is used to check whether the x is an even number or not. The relational expression results in value 1 if x is an even number otherwise results in value 0.
$a != b$	It is used to check whether a is not equal to b. This relational expression results in 1 if a is not

	equal to b otherwise 0.
$a + b == x + y$	It is used to check whether the expression "a+b" is equal to the expression "x+y".
$a \geq 9$	It is used to check whether the value of a is greater than or equal to 9.

Let's see a simple example:

```
#include <stdio.h>

int main()
{
    int x=4;

    if(x%2==0)
    {
        printf("The number x is even");
    }

    else
    printf("The number x is not even");

    return 0;
}
```

Output

```
✓ ↗ 📄
The number x is even

...Program finished with exit code 0
Press ENTER to exit console.
```

Logical Expressions

- A logical expression is an expression that computes either a zero or non-zero value.
- It is a complex test condition to take a decision.

Let's see some example of the logical expressions.

Logical Expressions	Description
$(x > 4) \ \&\& \ (x < 6)$	It is a test condition to check whether the x is greater than 4 and x is less than 6. The result of the condition is true only when both the conditions are true.
$x > 10 \ \ y < 11$	It is a test condition used to check whether x is greater than 10 or y is less than 11. The result of the test condition is true if either of the conditions holds true value.

<code>! (x > 10) && (y == 2)</code>	It is a test condition used to check whether x is not greater than 10 and y is equal to 2. The result of the condition is true if both the conditions are true.
--	---

Let's see a simple program of "&&" operator.

```
#include <stdio.h>

int main()
{
    int x = 4;

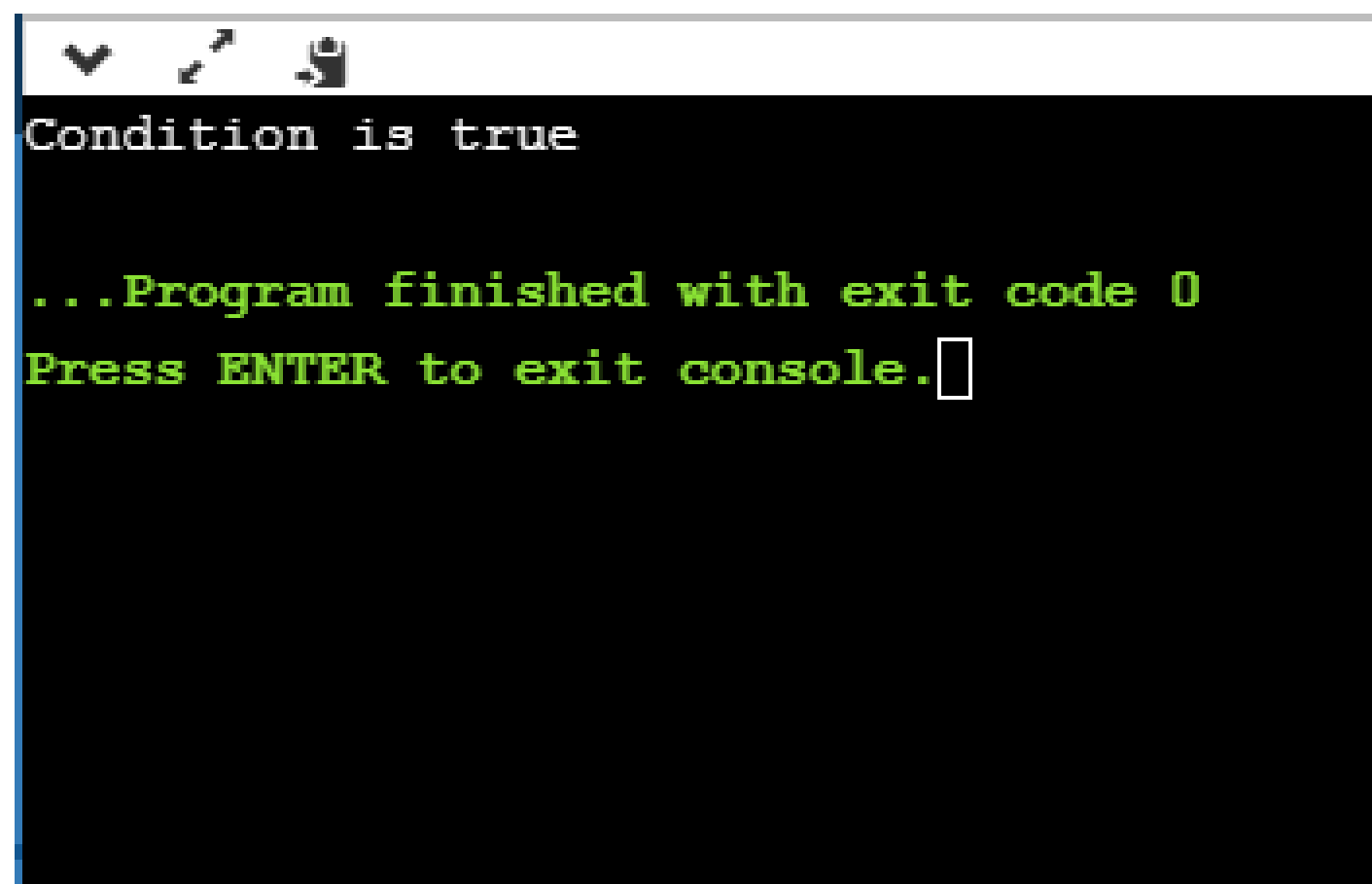
    int y = 10;

    if ( (x < 10) && (y > 5) )
    {
        printf("Condition is true");
    }

    else
        printf("Condition is false");

    return 0;
}
```

Output

A screenshot of a terminal window with a black background and green text. The text displayed is: "Condition is true", followed by "...Program finished with exit code 0", and "Press ENTER to exit console." with a cursor at the end. The terminal has a standard window title bar at the top with icons for minimize, maximize, and close.

```
Condition is true

...Program finished with exit code 0
Press ENTER to exit console.
```

Let's see a simple example of "||" operator

```
#include <stdio.h>

int main()
{
    int x = 4;

    int y = 9;

    if ( (x < 6) || (y > 10) )
    {
        printf("Condition is true");
    }

    else
    printf("Condition is false");

    return 0;
}
```

Output

```
Condition is true

...Program finished with exit code 0
Press ENTER to exit console.
```

Conditional Expressions

- A conditional expression is an expression that returns 1 if the condition is true otherwise 0.
- A conditional operator is also known as a ternary operator.

The Syntax of Conditional operator

Suppose exp1, exp2 and exp3 are three expressions.

`exp1 ? exp2 : exp3`

The above expression is a conditional expression which is evaluated on the basis of the value of the exp1 expression. If the condition of the expression exp1 holds true, then the final conditional expression is represented by exp2 otherwise represented by exp3.

Let's understand through a simple example.

```
#include<stdio.h>

#include<string.h>

int main()
{

    int age = 25;

    char status;

    status = (age>22) ? 'M': 'U';

    if(status == 'M')
```

```

printf("Married");

else

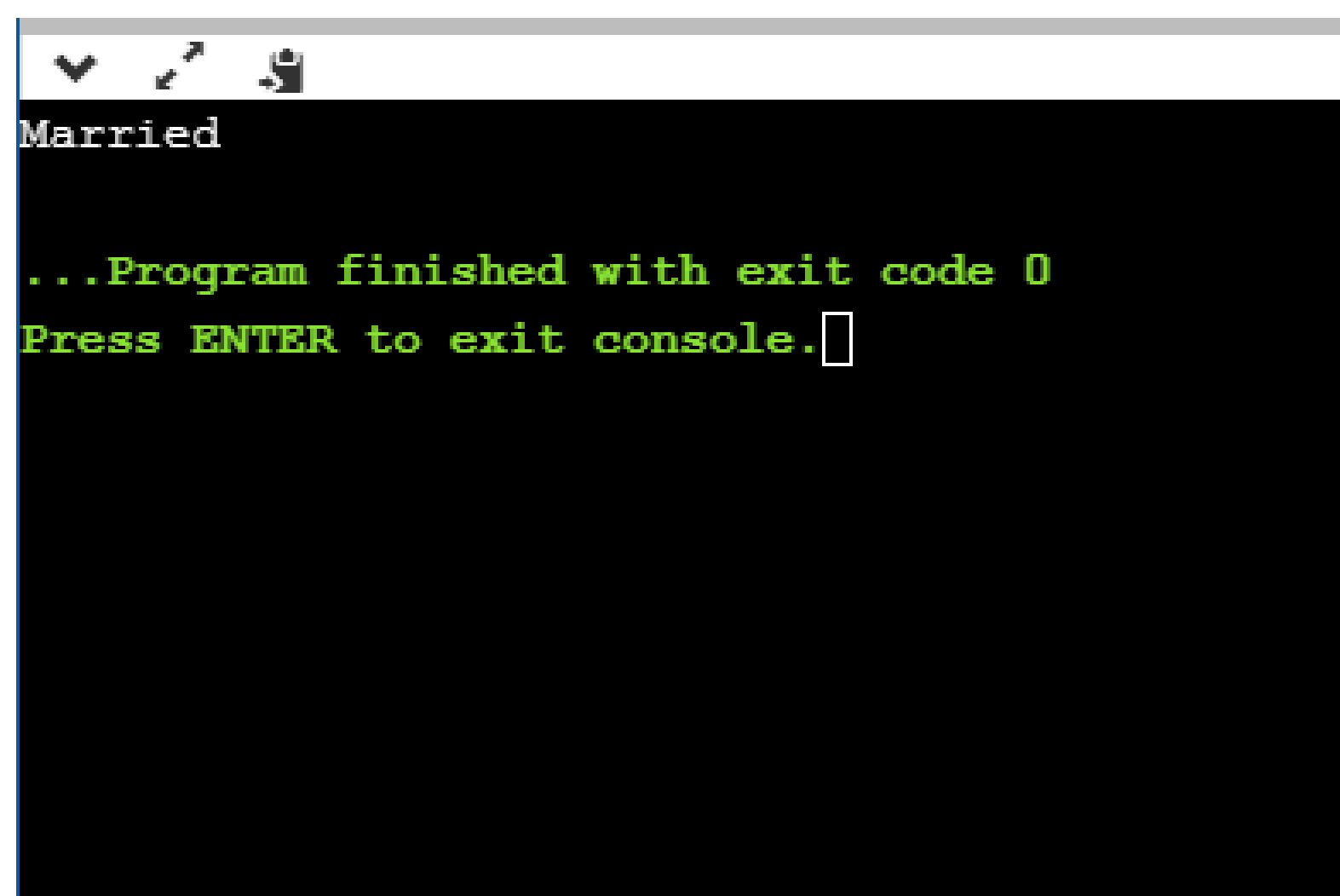
printf("Unmarried");

return 0;

}

```

Output



```

Married

...Program finished with exit code 0
Press ENTER to exit console.

```

C Data Segments

To understand the way our C program works, we need to understand the arrangement of the memory assigned to our program.

All the variables, functions, and data structures are allocated memory into a special memory segment known as Data Segment. The data segment is mainly divided into four different parts which are specifically allocated to different types of data defined in our C program.

Data Segments

Data Area	Code Area	Heap Area	Initialize variable	Non-initialize variable
-----------	-----------	-----------	---------------------	-------------------------

The parts of Data segments are :

1. Data Area

It is the permanent memory area. All static and external variables are stored in the data area. The variables which are stored in the data area exist until the program exits.

2. Code Area

It is the memory area which can only be accessed by the function pointers. The size of the code area is fixed.

3. Heap Area

As we know that C supports dynamic memory allocation. C provides the functions like `malloc()` and `calloc()` which are used to allocate the memory dynamically. Therefore, the heap area is used to store the data structures which are created by using dynamic memory allocation. The size of the heap area is variable and depends upon the free space in the memory.

4. Stack Area

Stack area is divided into two parts namely: initialize and non-initialize. Initialize variables are given priority than non-initialize variables.

1. All the automatic variables get memory into stack area.
2. Constants in c get stored in the stack area.
3. All the local variables of the default storage class get stored in the stack area.
4. Function parameters and return value get stored in the stack area.
5. Stack area is the temporary memory area as the variables stored in the stack area are deleted whenever the program reaches out of scope.

C Conditional Statements

C if else Statement

The if-else statement in C is used to perform the operations based on some specific condition. The operations specified in if block are executed if and only if the given condition is true.

There are the following variants of if statement in C language.

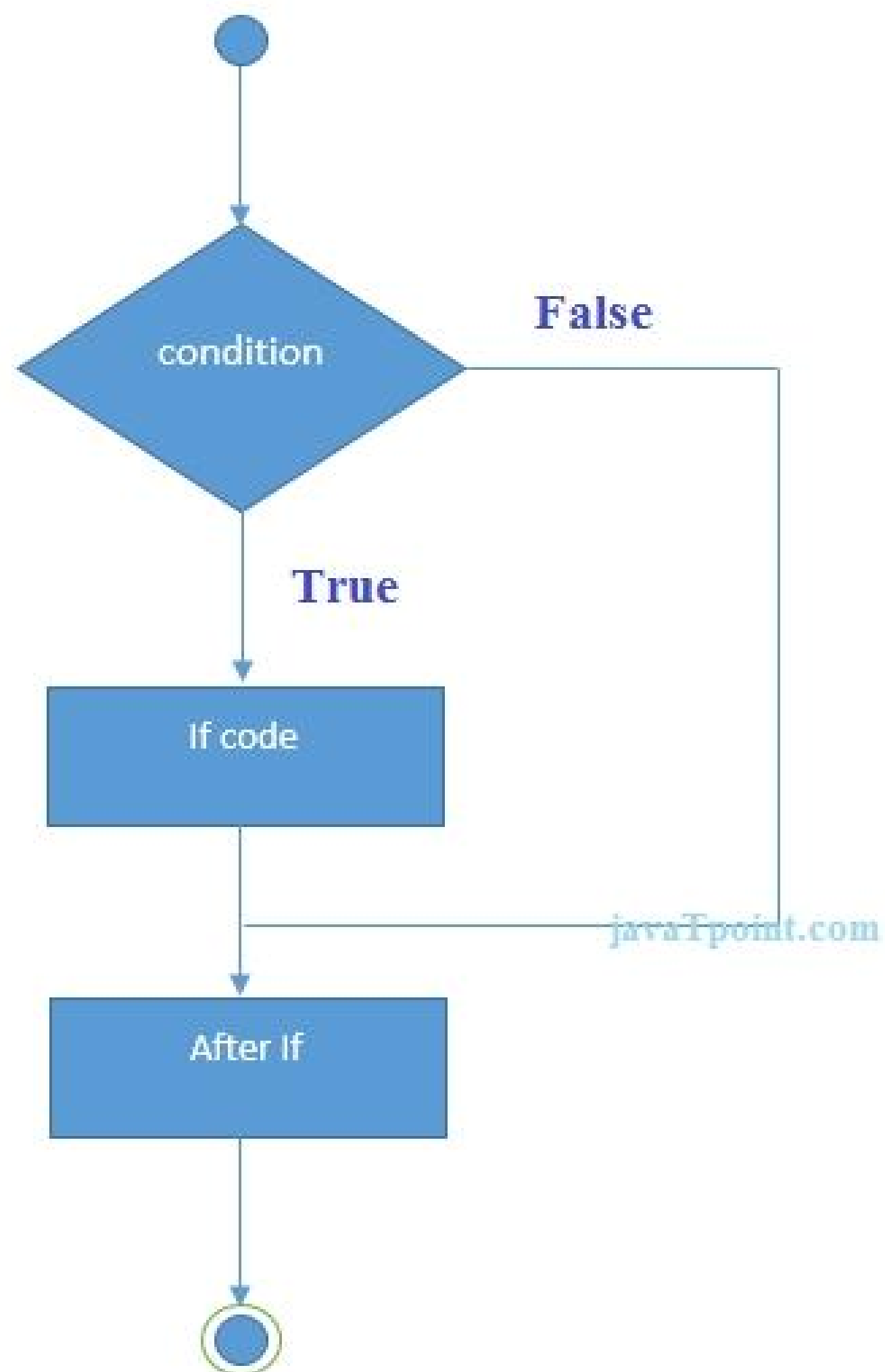
- If statement
- If-else statement
- If else-if ladder
- Nested if

If Statement

The if statement is used to check some given condition and perform some operations depending upon the correctness of that condition. It is mostly used in the scenario where we need to perform the different operations for the different conditions. The syntax of the if statement is given below.

```
if(expression){  
    //code to be executed  
}
```

Flowchart of if statement in C



Let's see a simple example of C language if statement.

```
#include<stdio.h>

int main(){

int number=0;

printf("Enter a number:");

scanf("%d",&number);

if(number%2==0){

printf("%d is even number",number);

}

return 0;
```

```
}
```

Output

Enter a number:4

4 is even number

a number:5

Program to find the largest number of the three.

```
#include <stdio.h>

int main()
{
    int a, b, c;

    printf("Enter three numbers?");

    scanf("%d %d %d",&a,&b,&c);

    if(a>b && a>c)
    {
        printf("%d is largest",a);
    }

    if(b>a && b > c)
    {
        printf("%d is largest",b);
    }

    if(c>a && c>b)
    {
        printf("%d is largest",c);
    }

    if(a == b && a == c)
    {
```

```
        printf("All are equal");  
    }  
}
```

Output

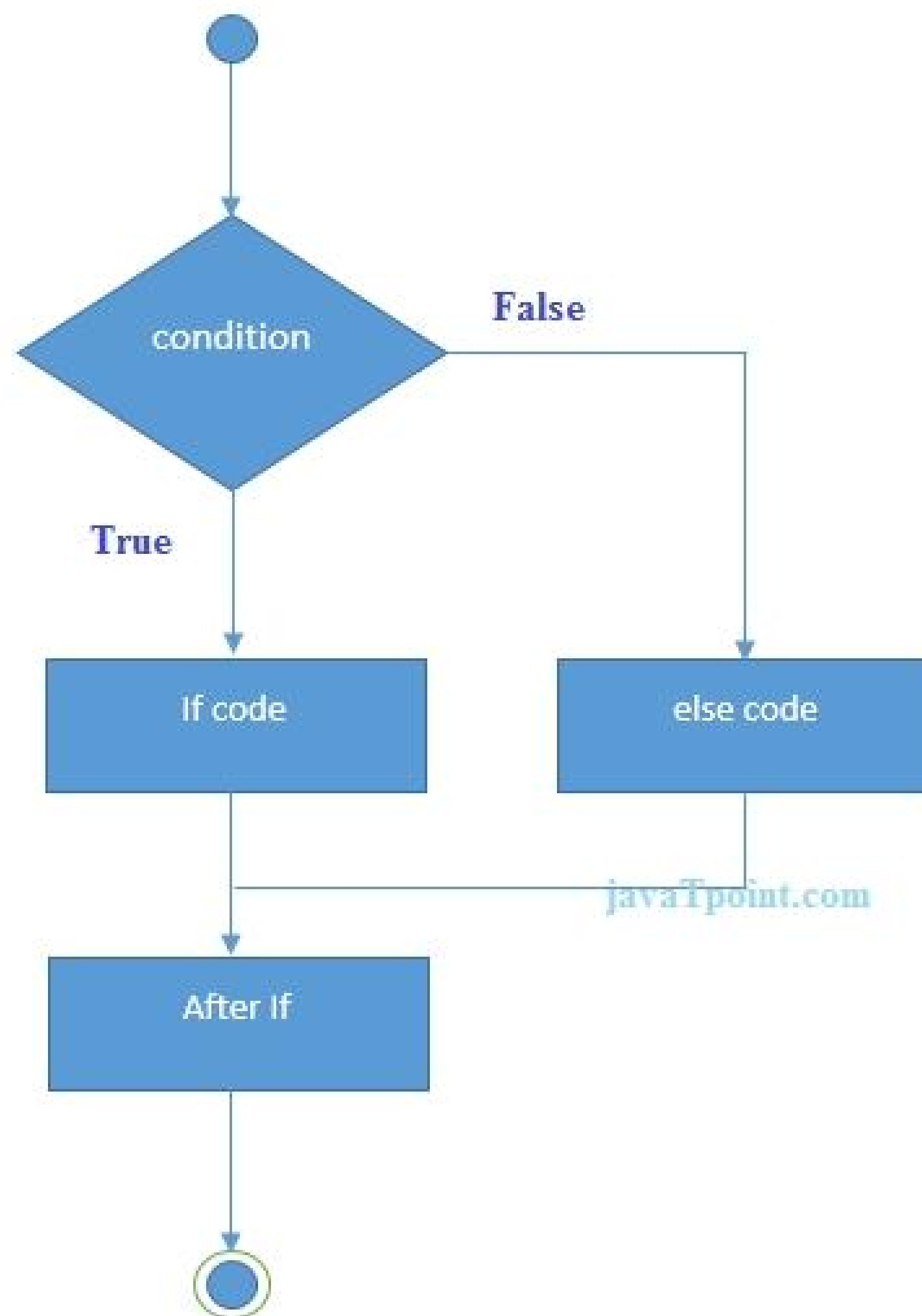
```
Enter three numbers?  
12 23 34  
34 is largest
```

If-else Statement

The if-else statement is used to perform two operations for a single condition. The if-else statement is an extension to the if statement using which, we can perform two different operations, i.e., one is for the correctness of that condition, and the other is for the incorrectness of the condition. Here, we must notice that if and else block cannot be executed simultaneously. Using if-else statement is always preferable since it always invokes an otherwise case with every if condition. The syntax of the if-else statement is given below.

```
if(expression){  
    //code to be executed if condition is true  
}  
else{  
    //code to be executed if condition is false  
}
```

Flowchart of the if-else statement in C



Let's see the simple example to check whether a number is even or odd using if-else statement in C language.

```
#include<stdio.h>

int main(){

int number=0;

printf("enter a number:");

scanf("%d",&number);

if(number%2==0){

printf("%d is even number",number);
```

```

}

else{

printf("%d is odd number",number);

}

return 0;

}

```

Output

```

enter a number:4

4 is even number

enter a number:5

5 is odd number

```

Program to check whether a person is eligible to vote or not.

```

#include <stdio.h>

int main()

{

    int age;

    printf("Enter your age?");

    scanf("%d",&age);

    if(age>=18)

    {

        printf("You are eligible to vote...");

    }

    else

    {

        printf("Sorry ... you can't vote");
    }
}

```

```
    }  
}
```

Output

Enter your age?18

You are eligible to vote...

Enter your age?13

Sorry ... you can't vote

If else-if ladder Statement

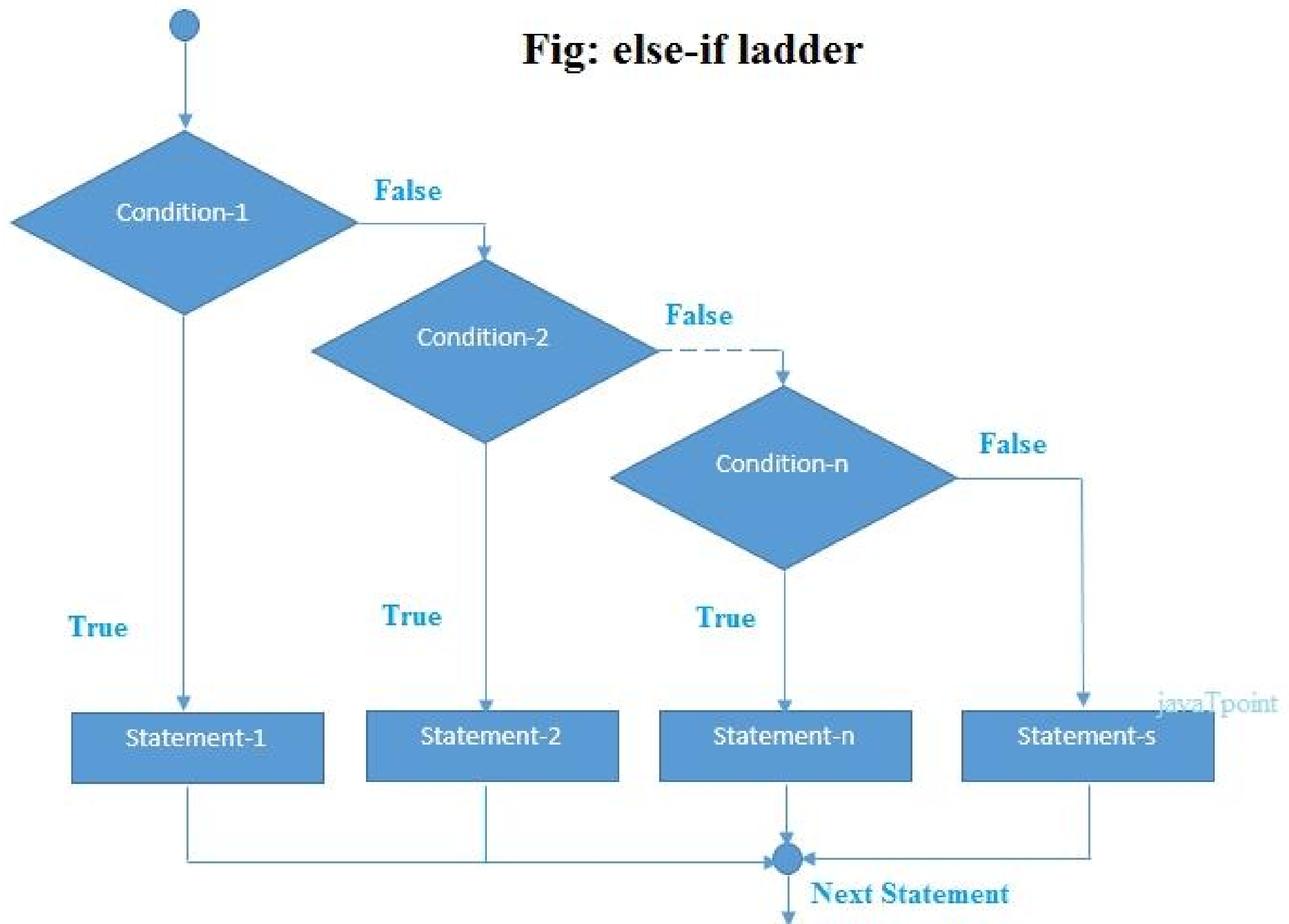
The if-else-if ladder statement is an extension to the if-else statement. It is used in the scenario where there are multiple cases to be performed for different conditions. In if-else-if ladder statement, if a condition is true then the statements defined in the if block will be executed, otherwise if some other condition is true then the statements defined in the else-if block will be executed, at the last if none of the condition is true then the statements defined in the else block will be executed. There are multiple else-if blocks possible. It is similar to the switch case statement where the default is executed instead of else block if none of the cases is matched.

```
if(condition1){  
  
    //code to be executed if condition1 is true  
  
}else if(condition2){  
  
    //code to be executed if condition2 is true  
  
}  
  
else if(condition3){  
  
    //code to be executed if condition3 is true  
  
}  
  
...  
  
else{
```

```
//code to be executed if all the conditions are false
```

```
}
```

Flowchart of else-if ladder statement in C



The example of an if-else-if statement in C language is given below.

```
#include<stdio.h>

int main(){

int number=0;

printf("enter a number:");

scanf("%d",&number);

if(number==10){

printf("number is equals to 10");

}
```

```

else if(number==50){

printf("number is equal to 50");

}

else if(number==100){

printf("number is equal to 100");

}

else{

printf("number is not equal to 10, 50 or 100");

}

return 0;

}

```

Output

```

enter a number:4

number is not equal to 10, 50 or 100

enter a number:50

number is equal to 50

```

Program to calculate the grade of the student according to the specified marks.

```

#include <stdio.h>

int main()

{

    int marks;

    printf("Enter your marks?");

    scanf("%d",&marks);

    if(marks > 85 && marks <= 100)

```

```

{
    printf("Congrats! you scored grade A ...");
}

else if (marks > 60 && marks <= 85)

{
    printf("You scored grade B + ...");
}

else if (marks > 40 && marks <= 60)

{
    printf("You scored grade B ...");
}

else if (marks > 30 && marks <= 40)

{
    printf("You scored grade C ...");
}

else

{
    printf("Sorry you are fail ...");
}

}

```

Output

Enter your marks?10

Sorry you are fail ...

Enter your marks?40

You scored grade C ...

Enter your marks?90

Congrats! you scored grade A ...

C Switch Statement

The switch statement in C is an alternate to if-else-if ladder statement which allows us to execute multiple operations for the different possible values of a single variable called switch variable. Here, We can define various statements in the multiple cases for the different values of a single variable.

The syntax of switch statement in c language is given below:

```
switch(expression){  
  
    case value1:  
  
        //code to be executed;  
  
        break; //optional  
  
    case value2:  
  
        //code to be executed;  
  
        break; //optional  
  
    .....  
  
    default:  
  
        code to be executed if all cases are not matched;  
  
}
```

Rules for switch statement in C language

- The *switch expression* must be of an integer or character type.
- The *case value* must be an integer or character constant.
- The *case value* can be used only inside the switch statement.

- The *break statement* in switch case is not must. It is optional. If there is no break statement found in the case, all the cases will be executed present after the matched case. It is known as *fall through* the state of C switch statement.

Let's try to understand it by the examples. We are assuming that there are following variables.

`int x,y,z;`

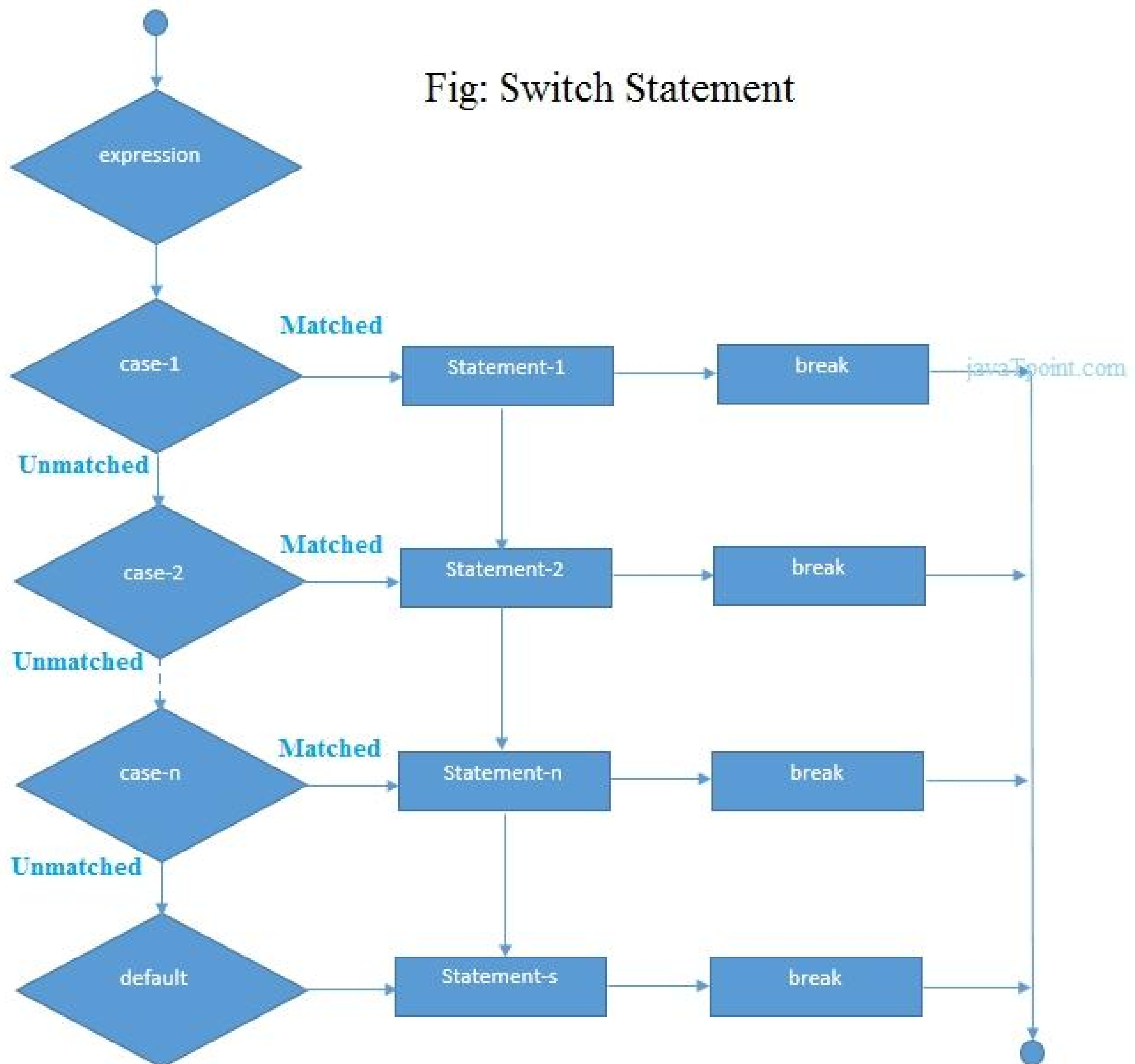
`char a,b;`

`float f;`

Valid Switch	Invalid Switch	Valid Case	Invalid Case
switch(x)	switch(f)	case 3;	case 2.5;
switch(x>y)	switch(x+2.5)	case 'a';	case x;
switch(a+b-2)		case 1+2;	case x+2;
switch(func(x,y))		case 'x'>'y';	case 1,2,3;

Flowchart of switch statement in C

Fig: Switch Statement



Functioning of switch case statement

First, the integer expression specified in the switch statement is evaluated. This value is then matched one by one with the constant values given in the different cases. If a match is found, then all the statements specified in that case are executed along with the all the cases present after that case including the default statement. No two cases can have similar values. If the matched case contains a break statement, then all the cases present after that will be skipped, and the control comes out of the switch. Otherwise, all the cases following the matched case will be executed.

Let's see a simple example of c language switch statement.

```
#include<stdio.h>

int main(){

int number=0;

printf("enter a number:");

scanf("%d",&number);

switch(number){

case 10:

printf("number is equals to 10");

break;

case 50:

printf("number is equal to 50");

break;

case 100:

printf("number is equal to 100");

break;

default:

printf("number is not equal to 10, 50 or 100");

}

return 0;

}
```

Output

enter a number:4

number is not equal to 10, 50 or 100

enter a number:50

number is equal to 50

Switch case example 2

```
#include <stdio.h>

int main()
{
    int x = 10, y = 5;

    switch(x>y && x+y>0)
    {
        case 1:
            printf("hi");

            break;

        case 0:
            printf("bye");

            break;

        default:
            printf(" Hello bye ");
    }
}
```

Output

hi

C Switch statement is fall-through

In C language, the switch statement is fall through; it means if you don't use a break statement in the switch case, all the cases after the matching case will be executed.

Let's try to understand the fall through state of switch statement by the example given below.

```
#include<stdio.h>

int main(){

int number=0;

printf("enter a number:");

scanf("%d",&number);

switch(number){

case 10:

printf("number is equal to 10\n");

case 50:

printf("number is equal to 50\n");

case 100:

printf("number is equal to 100\n");

default:

printf("number is not equal to 10, 50 or 100");

}

return 0;

}
```

Output 1

```
enter a number:10

number is equal to 10

number is equal to 50

number is equal to 100

number is not equal to 10, 50 or 100
```

Output 2

```
enter a number:50
number is equal to 50
number is equal to 100
number is not equal to 10, 50 or 100
```

Nested switch case statement

We can use as many switch statement as we want inside a switch statement. Such type of statements is called nested switch case statements. Consider the following example.

```
#include <stdio.h>

int main () {

    int i = 10;

    int j = 20;

    switch(i) {

        case 10:

            printf("the value of i evaluated in outer switch: %d\n",i);

        case 20:

            switch(j) {

                case 20:

                    printf("The value of j evaluated in nested switch: %d\n",j);

            }

    }

    printf("Exact value of i is: %d\n", i );

    printf("Exact value of j is: %d\n", j );

    return 0;
```

```
}
```

Output

```
the value of i evaluated in outer switch: 10
```

```
The value of j evaluated in nested switch: 20
```

```
Exact value of i is: 10
```

```
Exact value of j is: 20
```

if-else vs switch

What is an if-else statement?

An if-else statement in C programming is a conditional statement that executes a different set of statements based on the condition that is true or false. The 'if' block will be executed only when the specified condition is true, and if the specified condition is false, then the else block will be executed.

Syntax of if-else statement is given below:

```
if(expression)

{

    // statements;

}

else

{

    // statements;

}
```

What is a switch statement?

A switch statement is a conditional statement used in C programming to check the value of a variable and compare it with all the cases. If the value is matched with any

case, then its corresponding statements will be executed. Each case has some name or number known as the identifier. The value entered by the user will be compared with all the cases until the case is found. If the value entered by the user is not matched with any case, then the default statement will be executed.

Syntax of the switch statement is given below:

```
switch(expression)
{
    case constant 1:
        // statements;
        break;

    case constant 2:
        // statements;
        break;

    case constant n:
        // statements;
        break;

    default:
        // statements;
}
```

Similarity b/w if-else and switch

Both the if-else and switch are the decision-making statements. Here, decision-making statements mean that the output of the expression will decide which statements are to be executed.

Differences b/w if-else and switch statement

The following are the differences between if-else and switch statement are:

Definition

if-else

Based on the result of the expression in the 'if-else' statement, the block of statements will be executed. If the condition is true, then the 'if' block will be executed otherwise 'else' block will execute.

Switch statement

The switch statement contains multiple cases or choices. The user will decide the case, which is to execute.

Expression

If-else

It can contain a single expression or multiple expressions for multiple choices. In this, an expression is evaluated based on the range of values or conditions. It checks both equality and logical expressions.

Switch

It contains only a single expression, and this expression is either a single integer object or a string object. It checks only equality expression.

Evaluation

If-else

An if-else statement can evaluate almost all the types of data such as integer, floating-point, character, pointer, or Boolean.

Switch

A switch statement can evaluate either an integer or a character.

Sequence of Execution

If-else

In the case of 'if-else' statement, either the 'if' block or the 'else' block will be executed based on the condition.

Switch

In the case of the 'switch' statement, one case after another will be executed until the **break** keyword is not found, or the default statement is executed.

Default Execution

If-else

If the condition is not true within the 'if' statement, then by default, the else block statements will be executed.

Switch

If the expression specified within the **switch** statement is not matched with any of the cases, then the default statement, if defined, will be executed.

Values

If-else

Values are based on the condition specified inside the 'if' statement. The value will decide either the 'if' or 'else' block is to be executed.

Switch

In this case, value is decided by the user. Based on the choice of the user, the case will be executed.

Use

If-else

It evaluates a condition to be true or false.

Switch

A **switch** statement compares the value of the variable with multiple cases. If the value is matched with any of the cases, then the block of statements associated with this case will be executed.

Editing

If-else

Editing in 'if-else' statement is not easy as if we remove the 'else' statement, then it will create the havoc.

Switch

Editing in **switch** statement is easier as compared to the 'if-else' statement. If we remove any of the cases from the switch, then it will not interrupt the execution of other cases. Therefore, we can say that the **switch** statement is easy to modify and maintain.

Speed

If-else

If the choices are multiple, then the speed of the execution of 'if-else' statements is slow.

Switch

The case constants in the switch statement create a jump table at the compile time. This jump table chooses the path of the execution based on the value of the expression.

If we have a multiple choice, then the execution of the switch statement will be much faster than the equivalent logic of 'if-else' statement.

Let's summarize the above differences in a tabular form.

	If-else	Switch
Definition	Depending on the condition in the 'if' statement, 'if' and 'else' blocks are executed.	The user will decide which statement is to be executed.
Expression	It contains either logical or equality expression.	It contains a single expression which can be either a character or integer variable.
Evaluation	It evaluates all types of data, such as integer, floating-point, character or Boolean.	It evaluates either an integer, or character.
Sequence of execution	First, the condition is checked. If the condition is true then 'if' block is executed otherwise 'else' block	It executes one case after another till the break keyword is not found, or the default statement is executed.
Default execution	If the condition is not true, then by default, else block will be executed.	If the value does not match with any case, then by default, default statement is executed.
Editing	Editing is not easy in the 'if-else' statement.	Cases in a switch statement are easy to maintain and modify. Therefore, we can say that the removal or

		editing of any case will not interrupt the execution of other cases.
Speed	If there are multiple choices implemented through 'if-else', then the speed of the execution will be slow.	If we have multiple choices then the switch statement is the best option as the speed of the execution will be much higher than 'if-else'.

C Loops

The looping can be defined as repeating the same process multiple times until a specific condition satisfies. There are three types of loops used in the C language. In this part of the tutorial, we are going to learn all the aspects of C loops.

Why use loops in C language?

The looping simplifies the complex problems into the easy ones. It enables us to alter the flow of the program so that instead of writing the same code again and again, we can repeat the same code for a finite number of times. For example, if we need to print the first 10 natural numbers then, instead of using the printf statement 10 times, we can print inside a loop which runs up to 10 iterations.

Advantage of loops in C

- It provides code reusability.
- Using loops, we do not need to write the same code again and again.
- Using loops, we can traverse over the elements of data structures (array or linked lists).

Types of C Loops

There are three types of loops in C language that is given below:

- do while
- while
- for

do-while loop in C

The do-while loop continues until a given condition satisfies. It is also called post tested loop. It is used when it is necessary to execute the loop at least once (mostly menu driven programs).

The syntax of do-while loop in c language is given below:

```
do{  
  
    //code to be executed  
  
}while(condition);
```

while loop in C

The while loop in c is to be used in the scenario where we don't know the number of iterations in advance. The block of statements is executed in the while loop until the condition specified in the while loop is satisfied. It is also called a pre-tested loop.

The syntax of while loop in c language is given below:

```
while(condition){  
  
    //code to be executed  
  
}
```

for loop in C

The for loop is used in the case where we need to execute some part of the code until the given condition is satisfied. The for loop is also called as a per-tested loop. It is better to use for loop if the number of iteration is known in advance.

The syntax of for loop in c language is given below:

```
for(initialization;condition;incr/decr){  
  
    //code to be executed  
  
}
```

do while loop in C

The do while loop is a post tested loop. Using the do-while loop, we can repeat the execution of several parts of the statements. The do-while loop is mainly used in the case where we need to execute the loop at least once. The do-while loop is mostly used in menu-driven programs where the termination condition depends upon the end user.

do while loop syntax

The syntax of the C language do-while loop is given below:

```
do{  
  
    //code to be executed  
  
}while(condition);
```

Example 1

```
#include<stdio.h>  
  
#include<stdlib.h>  
  
void main ( )  
  
{  
  
    char c;  
  
    int choice,dummy;
```

```
do{

printf("\n1. Print Hello\n2. Print Javatpoint\n3. Exit\n");

scanf("%d",&choice);

switch(choice)

{

    case 1 :

        printf("Hello");

        break;

    case 2:

        printf("Javatpoint");

        break;

    case 3:

        exit(0);

        break;

    default:

        printf("please enter valid choice");

}

printf("do you want to enter more?");

scanf("%d",&dummy);

scanf("%c",&c);

}while(c=='y');

}
```

Output

1. Print Hello
2. Print Javatpoint

3. Exit

1

Hello

do you want to enter more?

y

1. Print Hello

2. Print Javatpoint

3. Exit

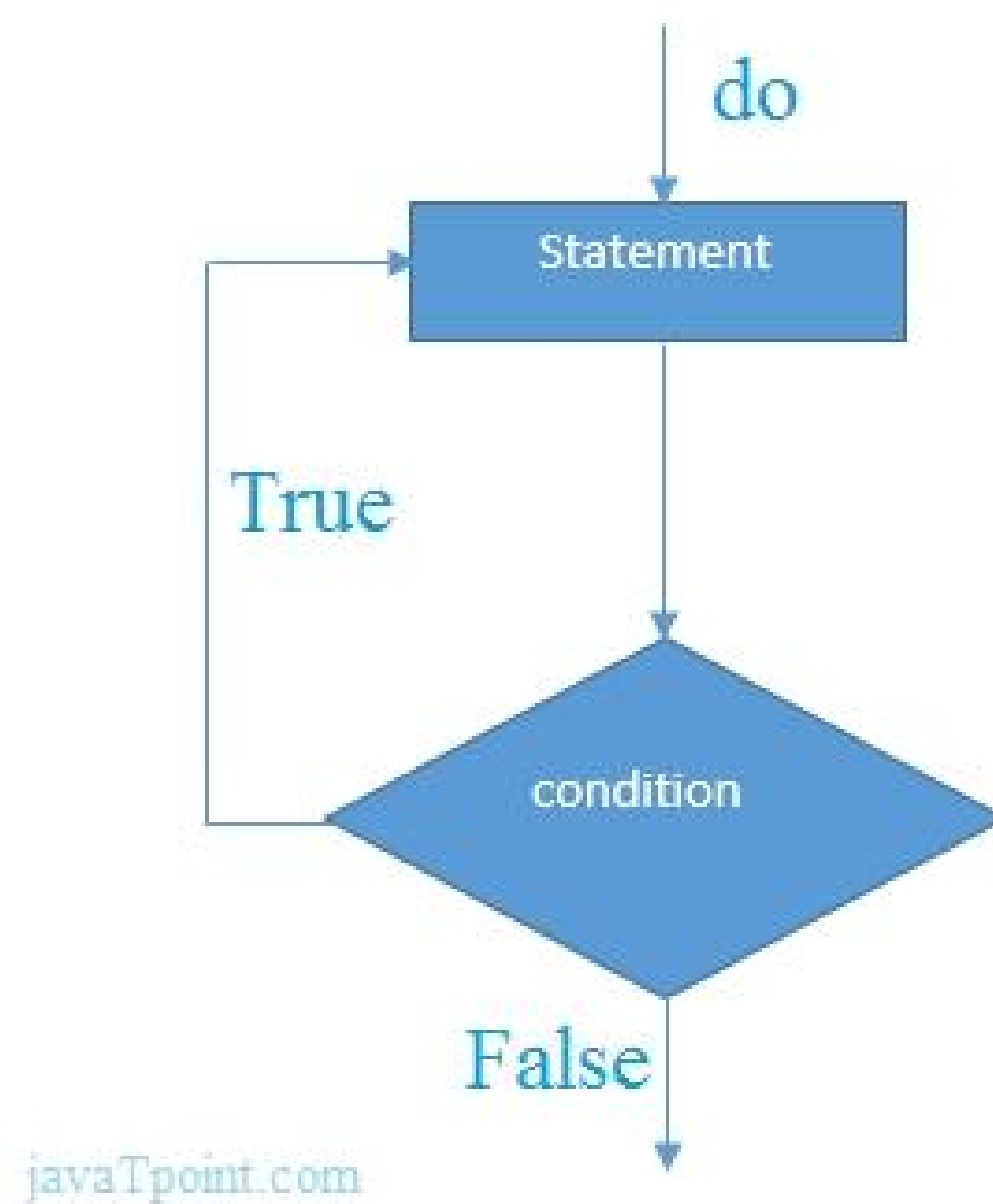
2

Javatpoint

do you want to enter more?

n

Flowchart of do while loop



do while example

There is given the simple program of c language do while loop where we are printing the table of 1.

```
#include<stdio.h>

int main(){

int i=1;

do{

printf("%d \n",i);

i++;

}while(i<=10);

return 0;

}
```

Output

```
1
2
3
4
5
6
7
8
9
10
```

Program to print table for the given number using do while loop

```
#include<stdio.h>
```

```
int main(){  
  
int i=1,number=0;  
  
printf("Enter a number: ");  
  
scanf("%d",&number);  
  
do{  
  
printf("%d \n",(number*i));  
  
i++;  
  
}while(i<=10);  
  
return 0;  
  
}
```

Output

Enter a number: 5

5

10

15

20

25

30

35

40

45

50

Enter a number: 10

10

20
30
40
50
60
70
80
90
100

Infinitive do while loop

The do-while loop will run infinite times if we pass any non-zero value as the conditional expression.

```
do{  
  
    //statement  
  
}while(1);
```

while loop in C

While loop is also known as a pre-tested loop. In general, a while loop allows a part of the code to be executed multiple times depending upon a given boolean condition. It can be viewed as a repeating if statement. The while loop is mostly used in the case where the number of iterations is not known in advance.

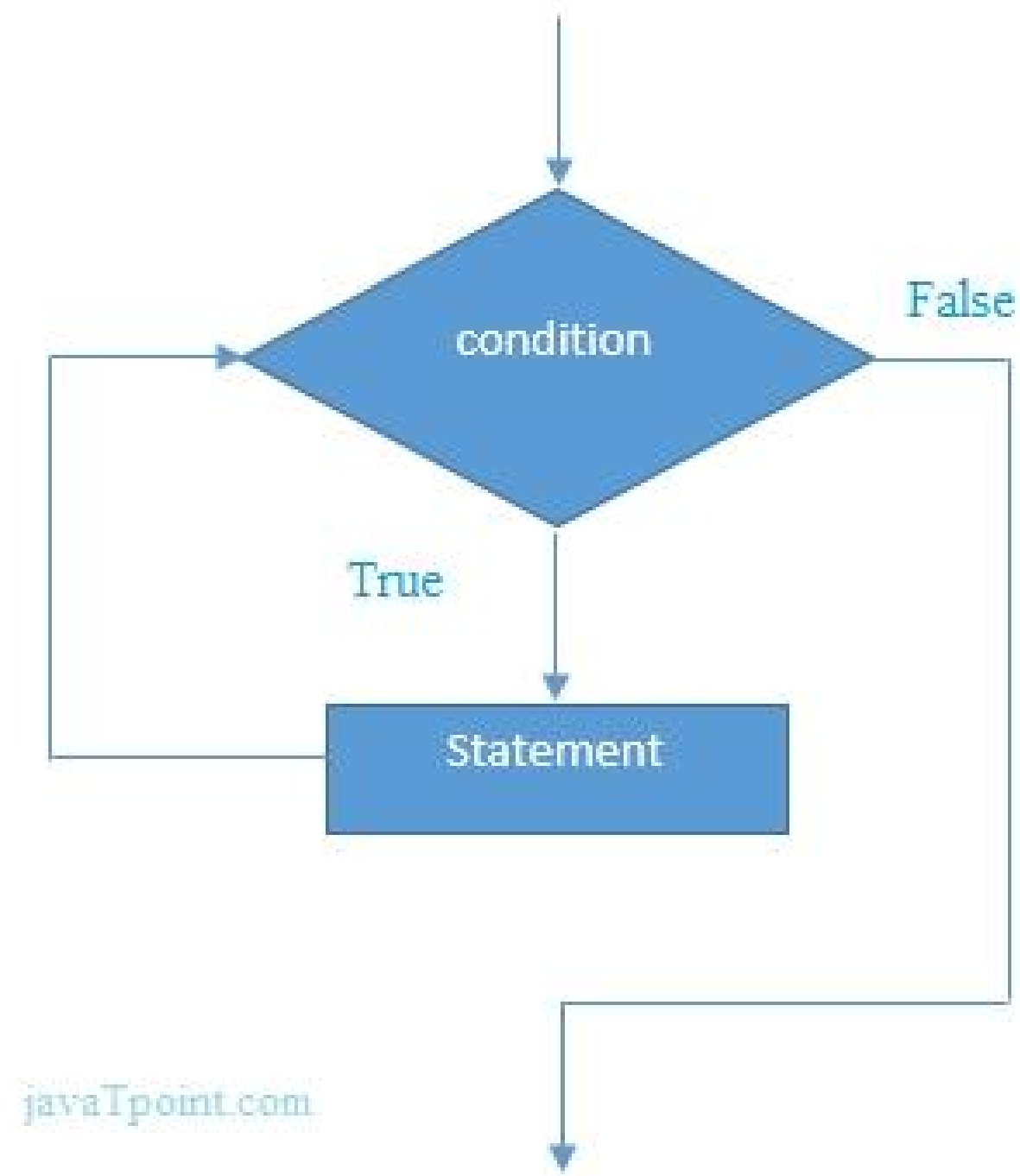
Syntax of while loop in C language

The syntax of while loop in c language is given below:

```
while(condition){  
  
    //code to be executed
```

}

Flowchart of while loop in C



Example of the while loop in C language

Let's see the simple program of while loop that prints table of 1.

```
#include<stdio.h>

int main(){

int i=1;

while(i<=10){

printf("%d \n",i);

i++;

}

return 0;

}
```

Output

1
2
3
4
5
6
7
8
9
10

Program to print table for the given number using while loop in C

```
#include<stdio.h>

int main(){

int i=1,number=0,b=9;

printf("Enter a number: ");

scanf("%d",&number);

while(i<=10){

printf("%d \n",(number*i));

i++;

}

return 0;

}
```

Output

Enter a number: 50
50

100

150

200

250

300

350

400

450

500

Enter a number: 100

100

200

300

400

500

600

700

800

900

1000

Properties of while loop

- A conditional expression is used to check the condition. The statements defined inside the while loop will repeatedly execute until the given condition fails.
- The condition will be true if it returns 0. The condition will be false if it returns any non-zero number.

- In while loop, the condition expression is compulsory.
- Running a while loop without a body is possible.
- We can have more than one conditional expression in while loop.
- If the loop body contains only one statement, then the braces are optional.

Example 1

```
#include<stdio.h>

void main ( )
{
    int j = 1;

    while(j+=2,j<=10)
    {
        printf("%d ",j);
    }

    printf("%d",j);
}
```

Output

3 5 7 9 11

Example 2

```
#include<stdio.h>

void main ( )
{
    while( )
    {
```

```
    printf("hello Javatpoint");  
}  
}
```

Output

compile time error: while loop can't be empty

Example 3

```
#include<stdio.h>  
  
void main ()  
{  
    int x = 10, y = 2;  
    while(x+y-1)  
    {  
        printf("%d %d",x--,y--);  
    }  
}
```

Output

infinite loop

Infinite while loop in C

If the expression passed in while loop results in any non-zero value then the loop will run the infinite number of times.

```
while(1){  
    //statement  
}
```

for loop in C

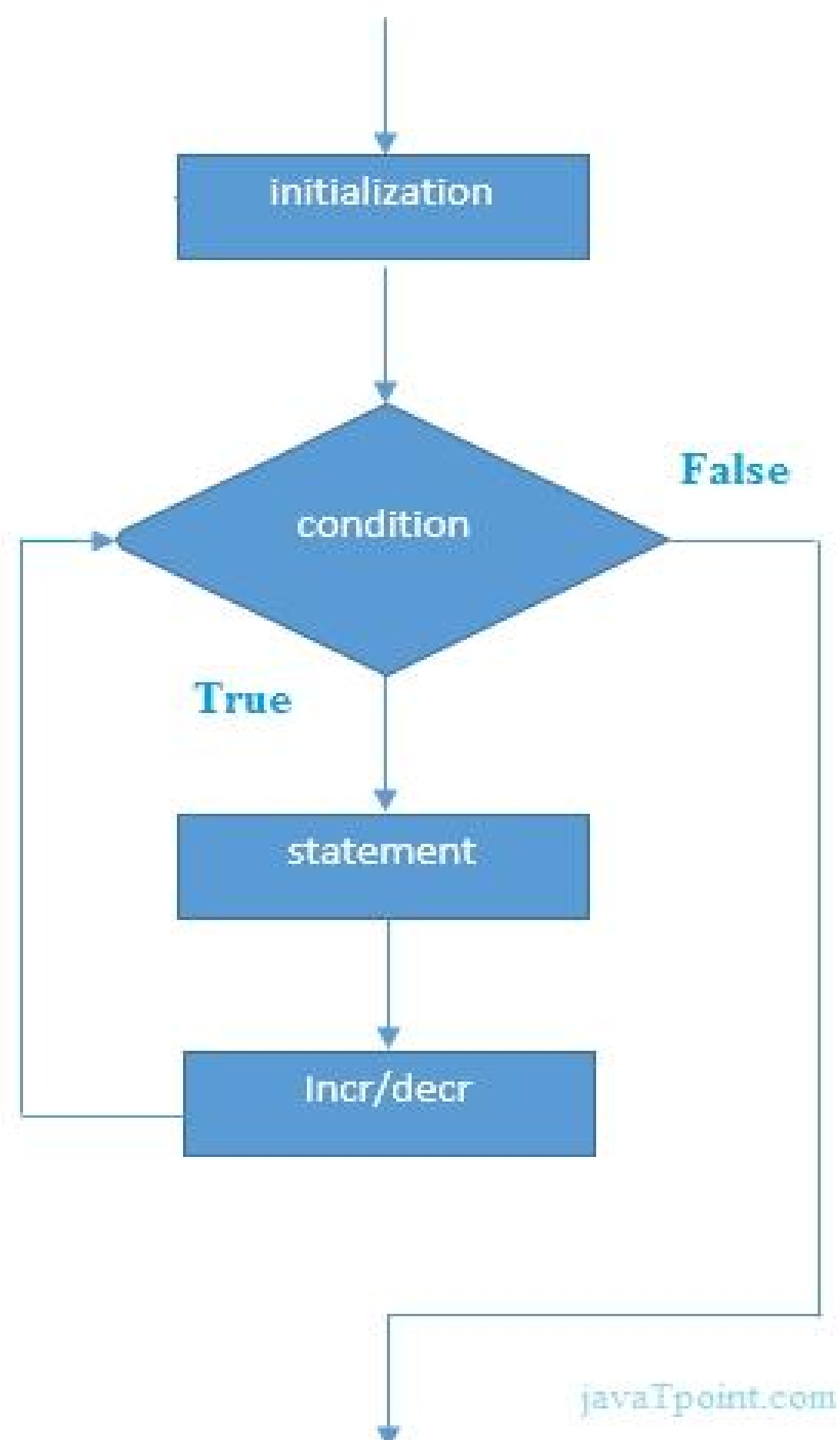
The **for loop in C language** is used to iterate the statements or a part of the program several times. It is frequently used to traverse the data structures like the array and linked list.

Syntax of for loop in C

The syntax of for loop in c language is given below:

```
for(Expression 1; Expression 2; Expression 3){  
    //code to be executed  
}
```

Flowchart of for loop in C



C for loop Examples

Let's see the simple program of for loop that prints table of 1.

```
#include<stdio.h>

int main(){

int i=0;

for(i=1;i<=10;i++){

printf("%d \n",i);

}

return 0;

}
```

Output

```
1
2
3
4
5
6
7
8
9
10
```

C Program: Print table for the given number using C for loop

```
#include<stdio.h>

int main(){

int i=1,number=0;
```

```
printf("Enter a number: ");  
  
scanf("%d",&number);  
  
for(i=1;i<=10;i++){  
  
printf("%d \n",(number*i));  
  
}  
  
return 0;  
  
}
```

Output

Enter a number: 2

2

4

6

8

10

12

14

16

18

20

Enter a number: 1000

1000

2000

3000

4000

5000

6000

7000

8000

9000

10000

Properties of Expression 1

- The expression represents the initialization of the loop variable.
- We can initialize more than one variable in Expression 1.
- Expression 1 is optional.
- In C, we can not declare the variables in Expression 1. However, It can be an exception in some compilers.

Example 1

```
#include <stdio.h>

int main()
{
    int a,b,c;

    for(a=0,b=12,c=23;a<2;a++)
    {
        printf("%d ",a+b+c);
    }
}
```

Output

Example 2

```
#include <stdio.h>

int main()

{

    int i=1;

    for(;i<5;i++)

    {

        printf("%d ",i);

    }

}
```

Output

1 2 3 4

Properties of Expression 2

- Expression 2 is a conditional expression. It checks for a specific condition to be satisfied. If it is not, the loop is terminated.
- Expression 2 can have more than one condition. However, the loop will iterate until the last condition becomes false. Other conditions will be treated as statements.
- Expression 2 is optional.
- Expression 2 can perform the task of expression 1 and expression 3. That is, we can initialize the variable as well as update the loop variable in expression 2 itself.

- We can pass zero or non-zero value in expression 2. However, in C, any non-zero value is true, and zero is false by default.

Example 1

```
#include <stdio.h>

int main()
{
    int i;

    for(i=0;i<=4;i++)
    {
        printf("%d ",i);
    }
}
```

Output

0 1 2 3 4

Example 2

```
#include <stdio.h>

int main()
{
    int i,j,k;

    for(i=0,j=0,k=0;i<4,k<8,j<10;i++)
    {
        printf("%d %d %d\n",i,j,k);

        j+=2;

        k+=3;
    }
}
```

```
}  
}
```

Output

```
0 0 0  
1 2 3  
2 4 6  
3 6 9  
4 8 12
```

Example 3

```
#include <stdio.h>  
  
int main()  
{  
  
    int i;  
  
    for(i=0;;i++)  
    {  
        printf("%d",i);  
    }  
}
```

Output

```
infinite loop
```

Properties of Expression 3

- Expression 3 is used to update the loop variable.
- We can update more than one variable at the same time.

- Expression 3 is optional.

Example 1

```
#include<stdio.h>

void main ( )

{

    int i=0,j=2;

    for(i = 0;i<5;i++,j=j+2)

    {

        printf("%d %d\n",i,j);

    }

}
```

Output

```
0 2
1 4
2 6
3 8
4 10
```

Loop body

The braces {} are used to define the scope of the loop. However, if the loop contains only one statement, then we don't need to use braces. A loop without a body is possible.

The braces work as a block separator, i.e., the value variable declared inside for loop is valid only for that block and not outside. Consider the following example.

```
#include<stdio.h>

void main ( )

{
```

```

int i;

for(i=0;i<10;i++)

{

    int i = 20;

    printf("%d ",i);

}

}

```

Output

20 20 20 20 20 20 20 20 20 20

Infinite for loop in C

To make a for loop infinite, we need not give any expression in the syntax. Instead of that, we need to provide two semicolons to validate the syntax of the for loop. This will work as an infinite for loop.

```

#include<stdio.h>

void main ( )

{

    for(;;)

    {

        printf("welcome to javatpoint");

    }

}

```

If you run this program, you will see above statement infinite times.

Nested Loops in C

C supports nesting of loops in C. **Nesting of loops** is the feature in C that allows the looping of statements inside another loop. Let's observe an example of nesting loops in C.

Any number of loops can be defined inside another loop, i.e., there is no restriction for defining any number of loops. The nesting level can be defined at n times. You can define any type of loop inside another loop; for example, you can define 'while' loop inside a 'for' loop.

Syntax of Nested loop

```
Outer_loop
{
    Inner_loop
    {
        // inner loop statements.
    }
    // outer loop statements.
}
```

Outer_loop and **Inner_loop** are the valid loops that can be a 'for' loop, 'while' loop or 'do-while' loop.

Nested for loop

The nested for loop means any type of loop which is defined inside the 'for' loop.

```
for (initialization; condition; update)
{
    for(initialization; condition; update)
    {
```

```

        // inner loop statements.

    }

    // outer loop statements.

}

```

Example of nested for loop

```

#include <stdio.h>

int main()
{
    int n; // variable declaration

    printf("Enter the value of n :");

    // Displaying the n tables.

    for(int i=1;i<=n;i++) // outer loop
    {
        for(int j=1;j<=10;j++) // inner loop
        {
            printf("%d\t",(i*j)); // printing the value.
        }

        printf("\n");
    }
}

```

Explanation of the above code

- First, the 'i' variable is initialized to 1 and then program control passes to the $i \leq n$.
- The program control checks whether the condition ' $i \leq n$ ' is true or not.
- If the condition is true, then the program control passes to the inner loop.

- The inner loop will get executed until the condition is true.
- After the execution of the inner loop, the control moves back to the update of the outer loop, i.e., $i++$.
- After incrementing the value of the loop counter, the condition is checked again, i.e., $i \leq n$.
- If the condition is true, then the inner loop will be executed again.
- This process will continue until the condition of the outer loop is true.

Output:

```

input
Enter the value of n : 3
1      2      3      4      5      6      7      8      9      10
2      4      6      8      10     12     14     16     18     20
3      6      9      12     15     18     21     24     27     30

...Program finished with exit code 0
Press ENTER to exit console.

```

Nested while loop

The nested while loop means any type of loop which is defined inside the 'while' loop.

```

while(condition)
{
    while(condition)
    {
        // inner loop statements.
    }

    // outer loop statements.
}

```

```
}
```

Example of nested while loop

```
#include <stdio.h>

int main()

{

    int rows; // variable declaration

    int columns; // variable declaration

    int k=1; // variable initialization

    printf("Enter the number of rows :"); // input the number of rows.

    scanf("%d",&rows);

    printf("\nEnter the number of columns :"); // input the number of columns.

    scanf("%d",&columns);

    int a[rows][columns]; //2d array declaration

    int i=1;

    while(i<=rows) // outer loop

    {

        int j=1;

        while(j<=columns) // inner loop

        {

            printf("%d\t",k); // printing the value of k.

            k++; // increment counter

            j++;

        }

        i++;

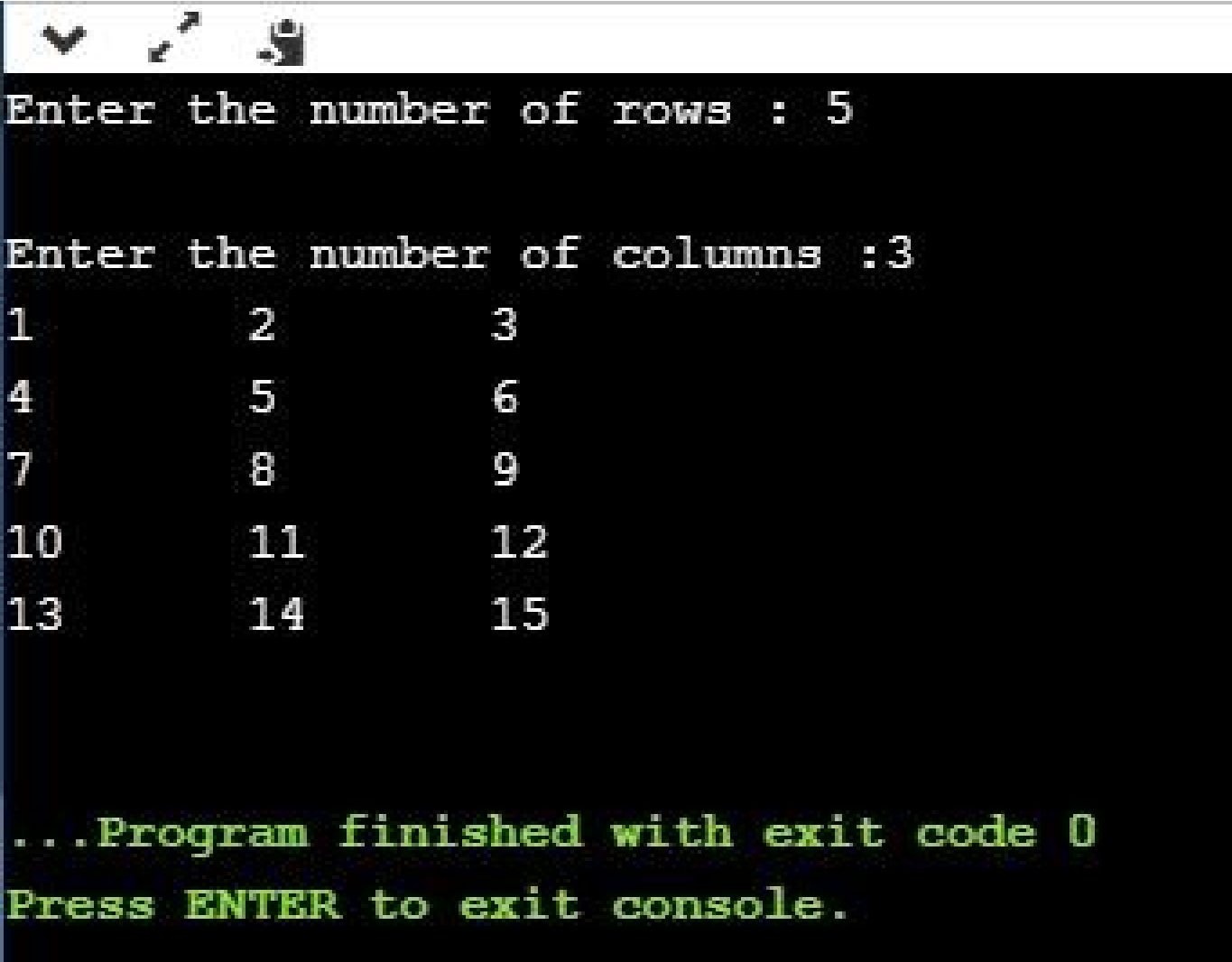
        printf("\n");
```

```
}  
  
}
```

Explanation of the above code.

- We have created the 2d array, i.e., `int a[rows][columns]`.
- The program initializes the 'i' variable by 1.
- Now, control moves to the while loop, and this loop checks whether the condition is true, then the program control moves to the inner loop.
- After the execution of the inner loop, the control moves to the update of the outer loop, i.e., `i++`.
- After incrementing the value of 'i', the condition (`i<=rows`) is checked.
- If the condition is true, the control then again moves to the inner loop.
- This process continues until the condition of the outer loop is true.

Output:



```
Enter the number of rows : 5  
  
Enter the number of columns :3  
1      2      3  
4      5      6  
7      8      9  
10     11     12  
13     14     15  
  
...Program finished with exit code 0  
Press ENTER to exit console.
```

Nested do..while loop

The nested do..while loop means any type of loop which is defined inside the 'do..while' loop.

```
do
{
    do
    {
        // inner loop statements.
    }while(condition);
    // outer loop statements.
}while(condition);
```

Example of nested do..while loop.

```
#include <stdio.h>

int main()
{
    /* printing the pattern
    * * * * *
    * * * * *
    * * * * *
    * * * * * */

    int i=1;

    do // outer loop
    {
        int j=1;

        do // inner loop
        {
```

```

        printf("*");

        j++;

    }while(j<=8);

    printf("\n");

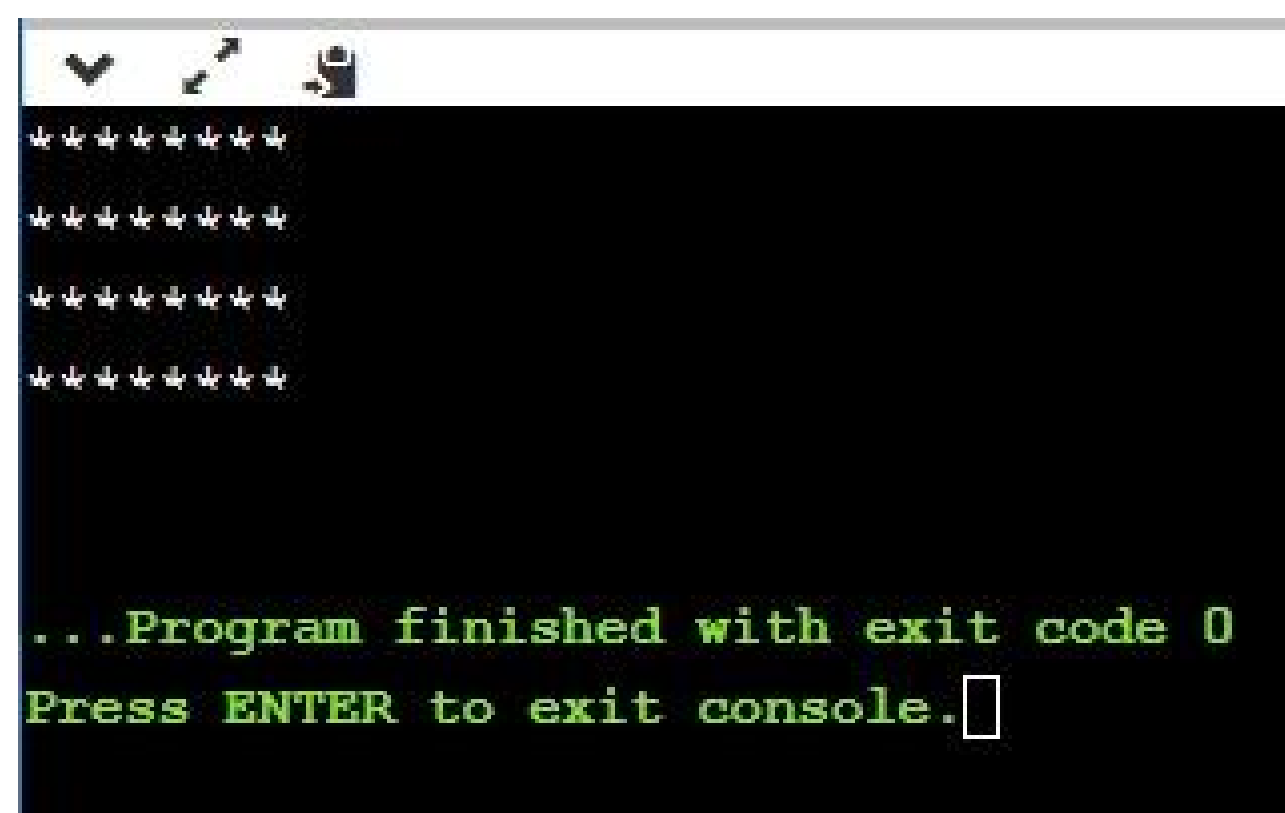
    i++;

    }while(i<=4);

}

```

Output:



```

****
****
****
****

...Program finished with exit code 0
Press ENTER to exit console.

```

Explanation of the above code.

- First, we initialize the outer loop counter variable, i.e., 'i' by 1.
- As we know that the do..while loop executes once without checking the condition, so the inner loop is executed without checking the condition in the outer loop.
- After the execution of the inner loop, the control moves to the update of the $i++$.
- When the loop counter value is incremented, the condition is checked. If the condition in the outer loop is true, then the inner loop is executed.
- This process will continue until the condition in the outer loop is true.

Infinite Loop in C

What is infinite loop?

An infinite loop is a looping construct that does not terminate the loop and executes the loop forever. It is also called an **indefinite** loop or an **endless** loop. It either produces a continuous output or no output.

When to use an infinite loop

An infinite loop is useful for those applications that accept the user input and generate the output continuously until the user exits from the application manually. In the following situations, this type of loop can be used:

- All the operating systems run in an infinite loop as it does not exist after performing some task. It comes out of an infinite loop only when the user manually shuts down the system.
- All the servers run in an infinite loop as the server responds to all the client requests. It comes out of an indefinite loop only when the administrator shuts down the server manually.
- All the games also run in an infinite loop. The game will accept the user requests until the user exits from the game.

We can create an infinite loop through various loop structures. The following are the loop structures through which we will define the infinite loop:

- for loop
- while loop
- do-while loop
- go to statement

- C macros

For loop

Let's see the **infinite** 'for' loop. The following is the definition for the **infinite** for loop:

```
for(;;)
{
    // body of the for loop.
}
```

As we know that all the parts of the 'for' loop are optional, and in the above for loop, we have not mentioned any condition; so, this loop will execute infinite times.

Let's understand through an example.

```
#include <stdio.h>

int main()
{
    for(;;)
    {
        printf("Hello javatpoint");
    }

    return 0;
}
```

In the above code, we run the 'for' loop infinite times, so "Hello javatpoint" will be displayed infinitely.

Output

input

while loop

Now, we will see how to create an infinite loop using a while loop. The following is the definition for the infinite while loop:

```
while(1)
{
    // body of the loop..
}
```

In the above while loop, we put '1' inside the loop condition. As we know that any non-zero integer represents the true condition while '0' represents the false condition.

Let's look at a simple example.

```
#include <stdio.h>

int main()

{

    int i=0;

    while(1)

    {

        i++;
```

```
printf("i is :%d",i);

}

return 0;

}
```

In the above code, we have defined a while loop, which runs infinite times as it does not contain any condition. The value of 'i' will be updated an infinite number of times.

Output

```

input
is :67951i is :67952i is :67953i is :67954i is :67955i is :67956i is :67957i is :67958i is :67959i is :67960i is :67961i is :67962i is :67963i is :67964i is :67965i is :67966i is :67967i is :67968i is :67969i is :67970i is :67971i is :67972i is :67973i is :67974i is :67975i is :67976i is :67977i is :67978i is :67979i is :67980i is :67981i is :67982i is :67983i is :67984i is :67985i is :67986i is :67987i is :67988i is :67989i is :67990i is :67991i is :67992i is :67993i is :67994i is :67995i is :67996i is :67997i is :67998i is :67999i is :68000i is :68001i is :68002i is :68003i is :68004i is :68005i is :68006i is :68007i is :68008i is :68009i is :68010i is :68011i is :68012i is :68013i is :68014i is :68015i is :68016i is :68017i is :68018i is :68019i is :68020i is :68021i is :68022i is :68023i is :68024i is :68025i is :68026i is :68027i is :68028i is :68029i is :68030i is :68031i is :68032i is :68033i is :68034i is :68035i is :68036i is :68037i is :68038i is :68039i is :68040i is :68041i is :68042i is :68043i is :68044i is :68045i is :68046i is :68047i is :68048i is :68049i is :68050i is :68051i is :68052i is :68053i is :68054i is :68055i is :68056i is :68057i is :68058i is :68059i is :68060i is :68061i is :68062i is :68063i is :68064i is :68065i is :68066i is :68067i is :68068i is :68069i is :68070i is :68071i is :68072i is :68073i is :68074i is :68075i is :68076i is :68077i is :68078i is :68079i is :68080i is :68081i is :68082i is :68083i is :68084i is :68085i is :68086i is :68087i is :68088i is :68089i is :68090i is :68091i is :68092i is :68093i is :68094i is :68095i is :68096i is :68097i is :68098i is :68099i is :68100i is :68101i is :68102i is :68103i is :68104i is :68105i is :68106i is :68107i is :68108i is :68109i is :68110i is :68111i is :68112i is :68113i is :68114i is :68115i is :68116i is :68117i is :68118i is :68119i is :68120i is :68121i i

```

do..while loop

The **do..while** loop can also be used to create the infinite loop. The following is the syntax to create the infinite **do..while** loop.

```
do
{
    // body of the loop..
}while(1);
```

The above do..while loop represents the infinite condition as we provide the '1' value inside the loop condition. As we already know that non-zero integer represents the true condition, so this loop will run infinite times.

goto statement

We can also use the goto statement to define the infinite loop.

```
infinite_loop;  
  
// body statements.  
  
goto infinite_loop;
```

In the above code, the goto statement transfers the control to the infinite loop.

Macros

We can also create the infinite loop with the help of a macro constant. Let's understand through an example.

```
#include <stdio.h>  
  
#define infinite for(;;)  
  
int main()  
{  
  
    infinite  
  
    {  
  
        printf("hello");  
  
    }  
  
    return 0;  
}
```

In the above code, we have defined a macro named as 'infinite', and its value is 'for(;;)'.

Whenever the word 'infinite' comes in a program then it will be replaced with a 'for(;;)'.

Output

```
input
lohellohellohellohellohellohellohellohellohellohellohellohellohellohellohellohellohellohellohellohellohello
hellohellohellohellohellohellohellohellohellohellohellohellohellohellohellohellohellohellohellohellohellohel
llohellohellohellohellohellohellohellohellohellohellohellohellohellohellohellohellohellohellohellohellohello
ohellohellohellohellohellohellohellohellohellohellohellohellohellohellohellohellohellohellohellohellohellohe
ellohellohellohellohellohellohellohellohellohellohellohellohellohellohellohellohellohellohellohellohellohell
lohellohellohellohellohellohellohellohellohellohellohellohellohellohellohellohellohellohellohellohellohell
hellohellohellohellohellohellohellohellohellohellohellohellohellohellohellohellohellohellohellohellohell
llohellohellohellohellohellohellohellohellohellohellohellohellohellohellohellohellohellohellohellohellohell
ohellohellohellohellohellohellohellohellohellohellohellohellohellohellohellohellohellohellohellohellohellohe
ellohellohellohellohellohellohellohellohellohellohellohellohellohellohellohellohellohellohellohellohellohell
lohellohellohellohellohellohellohellohellohellohellohellohellohellohellohellohellohellohellohellohellohell
hellohellohellohellohellohellohellohellohellohellohellohellohellohellohellohellohellohellohellohellohell
llohellohellohellohellohellohellohellohellohellohellohellohellohellohellohellohellohellohellohellohellohell
lohellohellohellohellohellohellohellohellohellohellohellohellohellohellohellohellohellohellohellohellohell
```

Till now, we have seen various ways to define an infinite loop. However, we need some approach to come out of the infinite loop. In order to come out of the infinite loop, we can use the break statement.

Let's understand through an example.

```
#include <stdio.h>

int main()

{

    char ch;

    while(1)

    {

        ch=getchar();

        if(ch=='n')

        {

            break;

        }

        printf("hello");

    }

    return 0;
```

```
}
```

In the above code, we have defined the while loop, which will execute an infinite number of times until we press the key 'n'. We have added the 'if' statement inside the while loop. The 'if' statement contains the break keyword, and the break keyword brings control out of the loop.

Unintentional infinite loops

Sometimes the situation arises where unintentional infinite loops occur due to the bug in the code. If we are the beginners, then it becomes very difficult to trace them. Below are some measures to trace an unintentional infinite loop:

- We should examine the semicolons carefully. Sometimes we put the semicolon at the wrong place, which leads to the infinite loop.

```
#include <stdio.h>

int main()
{
    int i=1;
    while(i<=10);
    {
        printf("%d", i);
        i++;
    }
    return 0;
}
```

In the above code, we put the semicolon after the condition of the while loop which leads to the infinite loop. Due to this semicolon, the internal body of the while loop will not execute.

- We should check the logical conditions carefully. Sometimes by mistake, we place the assignment operator (=) instead of a relational operator (==).

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
char ch='n';
```

```
while(ch='y')
```

```
{
```

```
printf("hello");
```

```
}
```

```
return 0;
```

```
}
```

In the above code, we use the assignment operator (ch='y') which leads to the execution of loop infinite number of times.

- We use the wrong loop condition which causes the loop to be executed indefinitely.

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
for(int i=1;i>=1;i++)
```

```
{
```

```
printf("hello");
```

```
}
```

```
return 0;
```

```
}
```

The above code will execute the 'for loop' infinite number of times. As we put the condition ($i \geq 1$), which will always be true for every condition, it means that "hello" will be printed infinitely.

- We should be careful when we are using the **break** keyword in the nested loop because it will terminate the execution of the nearest loop, not the entire loop.

```
#include <stdio.h>

int main()
{
    while(1)
    {
        for(int i=1;i<=10;i++)
        {
            if(i%2==0)
            {
                break;
            }
        }
    }

    return 0;
}
```

In the above code, the while loop will be executed an infinite number of times as we use the break keyword in an inner loop. This break keyword will bring the control out of the inner loop, not from the outer loop.

- We should be very careful when we are using the floating-point value inside the loop as we cannot underestimate the floating-point errors.

```
#include <stdio.h>

int main()

{

    float x = 3.0;

    while (x != 4.0) {

        printf("x = %f\n", x);

        x += 0.1;

    }

    return 0;

}
```

In the above code, the loop will run infinite times as the computer represents a floating-point value as a real value. The computer will represent the value of 4.0 as 3.999999 or 4.000001, so the condition ($x \neq 4.0$) will never be false. The solution to this problem is to write the condition as ($k \leq 4.0$).

C break statement

The break is a keyword in C which is used to bring the program control out of the loop. The break statement is used inside loops or switch statement. The break statement breaks the loop one by one, i.e., in the case of nested loops, it breaks the inner loop first and then proceeds to outer loops. The break statement in C can be used in the following two scenarios:

- With switch case
- With loop

Syntax:

```
//loop or switch case

break;
```

Flowchart of break in c

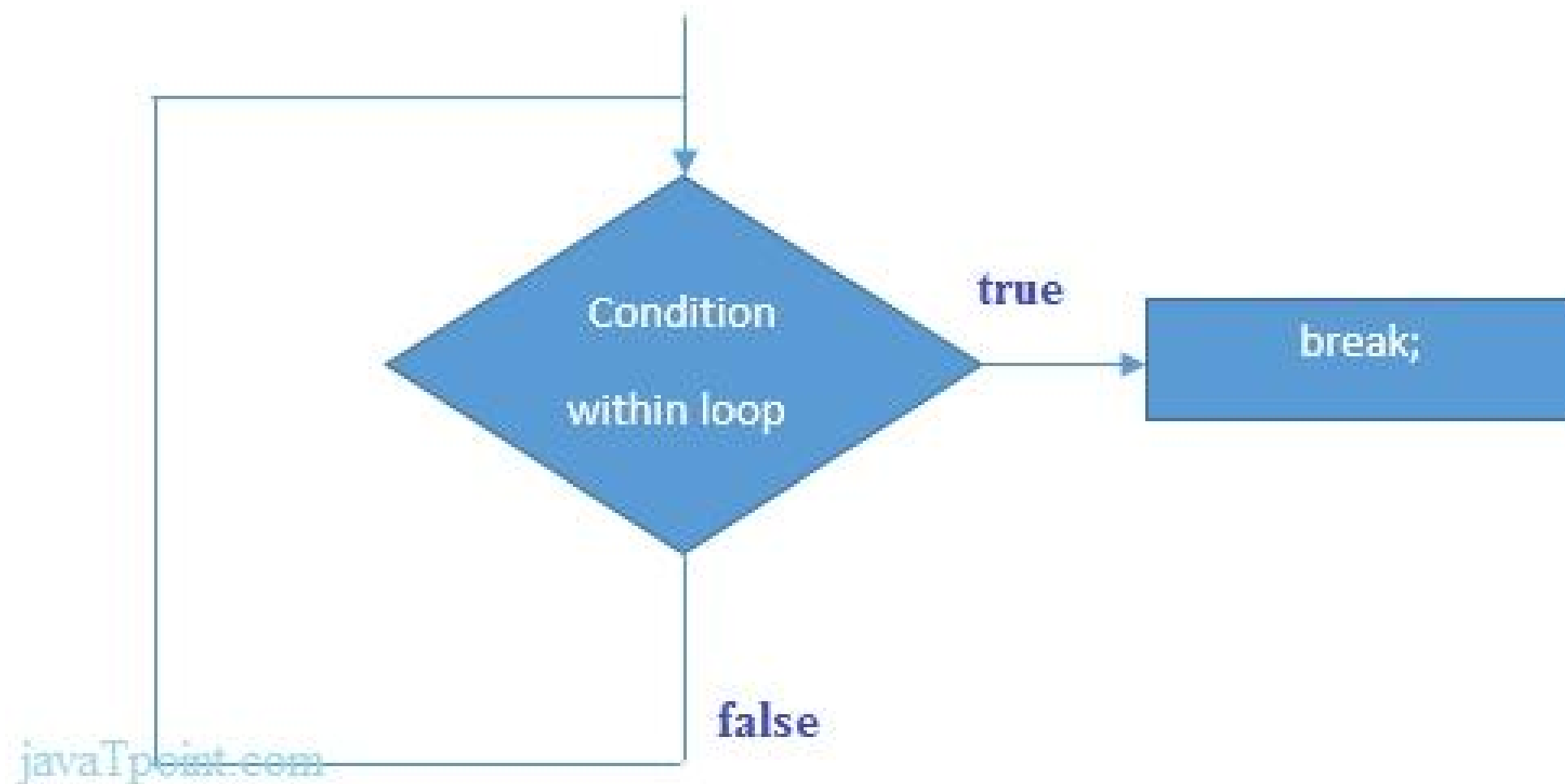


Figure: Flowchart of break statement

Example

```
#include<stdio.h>

#include<stdlib.h>

void main ( )
{
    int i;

    for(i = 0; i<10; i++)
    {
        printf("%d ",i);

        if(i == 5)
        {
            break;
        }

        printf("came outside of loop i = %d",i);
```

```
}
```

Output

```
0 1 2 3 4 5 came outside of loop i = 5
```

C break statement with the nested loop

In such case, it breaks only the inner loop, but not outer loop.

```
#include<stdio.h>

int main(){

int i=1,j=1; //initializing a local variable

for(i=1;i<=3;i++){

for(j=1;j<=3;j++){

printf("%d &d\n",i,j);

if(i==2 && j==2){

break; //will break loop of j only

}

} //end of for loop

return 0;

}
```

Output

```
1 1
1 2
1 3
2 1
2 2
3 1
```

3 2

3 3

As you can see the output on the console, 2 3 is not printed because there is a break statement after printing $i==2$ and $j==2$. But 3 1, 3 2 and 3 3 are printed because the break statement is used to break the inner loop only.

break statement with while loop

Consider the following example to use break statement inside while loop.

```
#include<stdio.h>

void main ( )
{
    int i = 0;

    while(1)
    {
        printf("%d ",i);

        i++;

        if(i == 10)
            break;
    }

    printf("came out of while loop");
}
```

Output

0 1 2 3 4 5 6 7 8 9 came out of while loop

break statement with do-while loop

Consider the following example to use the break statement with a do-while loop.

```

#include<stdio.h>

void main ( )
{
    int n=2,i,choice;

    do
    {
        i=1;

        while(i<=10)
        {
            printf("%d X %d = %d\n",n,i,n*i);

            i++;
        }

        printf("do you want to continue with the table of %d , enter any non-zero value to continue.
",n+1);

        scanf("%d",&choice);

        if(choice == 0)
        {
            break;
        }

        n++;
    }while(1);
}

```

Output

2 X 1 = 2

2 X 2 = 4

2 X 3 = 6

2 X 4 = 8

2 X 5 = 10

2 X 6 = 12

2 X 7 = 14

2 X 8 = 16

2 X 9 = 18

2 X 10 = 20

do you want to continue with the table of 3 , enter any non-zero value to continue.1

3 X 1 = 3

3 X 2 = 6

3 X 3 = 9

3 X 4 = 12

3 X 5 = 15

3 X 6 = 18

3 X 7 = 21

3 X 8 = 24

3 X 9 = 27

3 X 10 = 30

do you want to continue with the table of 4 , enter any non-zero value to continue.0

C continue statement

The **continue statement** in C language is used to bring the program control to the beginning of the loop. The continue statement skips some lines of code inside the loop

and continues with the next iteration. It is mainly used for a condition so that we can skip some code for a particular condition.

Syntax:

```
//loop statements

continue;

//some lines of the code which is to be skipped
```

Continue statement example 1

```
#include<stdio.h>

void main ( )

{

    int i = 0;

    while(i!=10)

    {

        printf("%d", i);

        continue;

        i++;

    }

}
```

Output

infinite loop

Continue statement example 2

```
#include<stdio.h>

int main(){

    int i=1;//initializing a local variable
```

```

//starting a loop from 1 to 10

for(i=1;i<=10;i++){

if(i==5){ //if value of i is equal to 5, it will continue the loop

continue;

}

printf("%d \n",i);

} //end of for loop

return 0;

}

```

Output

```

1
2
3
4
6
7
8
9
10

```

As you can see, 5 is not printed on the console because loop is continued at `i==5`.

C continue statement with inner loop

In such case, C continue statement continues only inner loop, but not outer loop.

```

#include<stdio.h>

int main(){

int i=1,j=1; //initializing a local variable

```

```

for(i=1;i<=3;i++){

for(j=1;j<=3;j++){

if(i==2 && j==2){

continue;//will continue loop of j only

}

printf("%d %d\n",i,j);

}

}

}

return 0;

}

```

Output

```

1 1
1 2
1 3
2 1
2 3
3 1
3 2
3 3

```

As you can see, 2 2 is not printed on the console because inner loop is continued at i==2 and j==2.

C goto statement

The goto statement is known as jump statement in C. As the name suggests, goto is used to transfer the program control to a predefined label. The goto statement can be used to repeat some part of the code for a particular condition. It can also be used to

break the multiple loops which can't be done by using a single break statement. However, using goto is avoided these days since it makes the program less readable and complicated.

Syntax:

```
label:  
  
//some part of the code;  
  
goto label;
```

goto example

Let's see a simple example to use goto statement in C language.

```
#include <stdio.h>  
  
int main()  
{  
  
    int num,i=1;  
  
    printf("Enter the number whose table you want to print?");  
  
    scanf("%d",&num);  
  
    table:  
  
    printf("%d x %d = %d\n",num,i,num*i);  
  
    i++;  
  
    if(i<=10)  
  
        goto table;  
  
}
```

Output:

```
Enter the number whose table you want to print?10  
  
10 x 1 = 10  
  
10 x 2 = 20
```

10 x 3 = 30

10 x 4 = 40

10 x 5 = 50

10 x 6 = 60

10 x 7 = 70

10 x 8 = 80

10 x 9 = 90

10 x 10 = 100

The only condition in which using goto is preferable is when we need to break the multiple loops using a single statement at the same time. Consider the following example.

```
#include <stdio.h>

int main()
{
    int i, j, k;

    for(i=0;i<10;i++)
    {
        for(j=0;j<5;j++)
        {
            for(k=0;k<3;k++)
            {
                printf("%d %d %d\n",i,j,k);

                if(j == 3)
                {
                    goto out;
                }
            }
        }
    }
}
```

```

        }

    }

}

out:

printf("came out of the loop");

}

```

Output:

```

0 0 0
0 0 1
0 0 2
0 1 0
0 1 1
0 1 2
0 2 0
0 2 1
0 2 2
0 3 0

came out of the loop

```

Type Casting in C

Typecasting allows us to convert one data type into other. In C language, we use cast operator for typecasting which is denoted by (type).

Syntax:

```
(type)value;
```

Note: It is always recommended to convert the lower value to higher for avoiding data loss.

Without Type Casting:

```
int f= 9/4;

printf("f : %d\n", f );//Output: 2
```

With Type Casting:

```
float f=( float) 9/4;

printf("f : %f\n", f );//Output: 2.250000
```

Type Casting example

Let's see a simple example to cast int value into the float.

```
#include<stdio.h>

int main(){

float f= (float)9/4;

printf("f : %f\n", f );

return 0;

}
```

Output:

```
f : 2.250000
```

C Functions

What is Function

In c, we can divide a large program into the basic building blocks known as function. The function contains the set of programming statements enclosed by {}. A function can be called multiple times to provide reusability and modularity to the C program. In

other words, we can say that the collection of functions creates a program. The function is also known as *procedure* or *subroutine* in other programming languages.

Advantage of functions in C

There are the following advantages of C functions.

- By using functions, we can avoid rewriting same logic/code again and again in a program.
- We can call C functions any number of times in a program and from any place in a program.
- We can track a large C program easily when it is divided into multiple functions.
- Reusability is the main achievement of C functions.
- However, Function calling is always a overhead in a C program.

Function Aspects

There are three aspects of a C function.

- **Function declaration** A function must be declared globally in a c program to tell the compiler about the function name, function parameters, and return type.
- **Function call** Function can be called from anywhere in the program. The parameter list must not differ in function calling and function declaration. We must pass the same number of functions as it is declared in the function declaration.
- **Function definition** It contains the actual statements which are to be executed. It is the most important aspect to which the control comes when the function is called. Here, we must notice that only one value can be returned from the function.

SN	C function aspects	Syntax
1	Function declaration	return_type function_name (argument list);
2	Function call	function_name (argument_list)
3	Function definition	return_type function_name (argument list) {function body;}

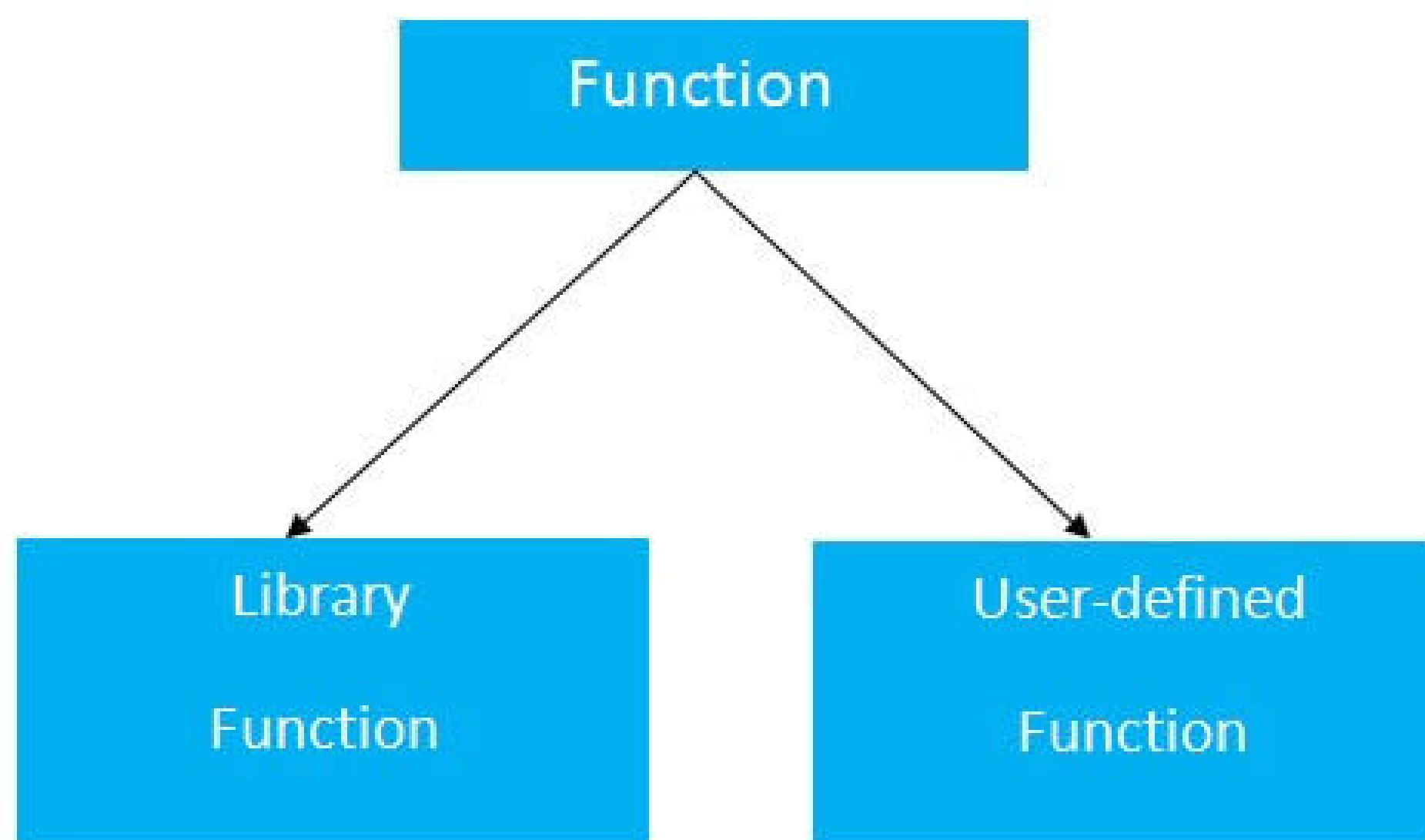
The syntax of creating function in c language is given below:

```
return_type function_name(data_type parameter...){  
  
    //code to be executed  
  
}
```

Types of Functions

There are two types of functions in C programming:

- 1. **Library Functions:** are the functions which are declared in the C header files such as scanf(), printf(), gets(), puts(), ceil(), floor() etc.
- 2. **User-defined functions:** are the functions which are created by the C programmer, so that he/she can use it many times. It reduces the complexity of a big program and optimizes the code.



Return Value

A C function may or may not return a value from the function. If you don't have to return any value from the function, use void for the return type.

Let's see a simple example of C function that doesn't return any value from the function.

Example without return value:

```
void hello(){  
    printf("hello c");  
}
```

If you want to return any value from the function, you need to use any data type such as int, long, char, etc. The return type depends on the value to be returned from the function.

Let's see a simple example of C function that returns int value from the function.

Example with return value:

```
int get(){
```

```
return 10;

}
```

In the above example, we have to return 10 as a value, so the return type is int. If you want to return floating-point value (e.g., 10.2, 3.1, 54.5, etc), you need to use float as the return type of the method.

```
float get(){

return 10.2;

}
```

Now, you need to call the function, to get the value of the function.

Different aspects of function calling

A function may or may not accept any argument. It may or may not return any value. Based on these facts, There are four different aspects of function calls.

- function without arguments and without return value
- function without arguments and with return value
- function with arguments and without return value
- function with arguments and with return value

Example for Function without argument and without return value

Example 1

```
#include<stdio.h>

void printName();

void main ( )

{

printf("Hello ");
```

```
        printName();
    }

    void printName()
    {
        printf("Javatpoint");
    }
}
```

Output

```
Hello Javatpoint
```

Example 2

```
#include<stdio.h>

void sum();

void main()
{
    printf("\nGoing to calculate the sum of two numbers:");

    sum();
}

void sum()
{
    int a,b;

    printf("\nEnter two numbers");

    scanf("%d %d",&a,&b);

    printf("The sum is %d",a+b);
}
```

Output

```
Going to calculate the sum of two numbers:
```

```
Enter two numbers 10
```

```
24
```

```
The sum is 34
```

Example for Function without argument and with return value

Example 1

```
#include<stdio.h>

int sum( );

void main()
{
    int result;

    printf("\nGoing to calculate the sum of two numbers:");

    result = sum( );

    printf("%d",result);
}

int sum( )
{
    int a,b;

    printf("\nEnter two numbers");

    scanf("%d %d",&a,&b);

    return a+b;
}
```

Output

```
Going to calculate the sum of two numbers:
```

```
Enter two numbers 10
```

```
24
```

The sum is 34

Example 2: program to calculate the area of the square

```
#include<stdio.h>

int square();

void main()

{

    printf("Going to calculate the area of the square\n");

    float area = square( );

    printf("The area of the square: %f\n",area);

}

int square()

{

    float side;

    printf("Enter the length of the side in meters: ");

    scanf("%f",&side);

    return side * side;

}
```

Output

Going to calculate the area of the square

Enter the length of the side in meters: 10

The area of the square: 100.000000

Example for Function with argument and without return value

Example 1

```
#include<stdio.h>
```

```

void sum(int, int);

void main()
{
    int a,b,result;

    printf("\nGoing to calculate the sum of two numbers:");

    printf("\nEnter two numbers:");

    scanf("%d %d",&a,&b);

    sum(a,b);

}

void sum(int a, int b)
{
    printf("\nThe sum is %d",a+b);

}

```

Output

```

Going to calculate the sum of two numbers:

Enter two numbers 10

24

The sum is 34

```

Example 2: program to calculate the average of five numbers.

```

#include<stdio.h>

void average(int, int, int, int, int);

void main()
{
    int a,b,c,d,e;

    printf("\nGoing to calculate the average of five numbers:");

```

```

printf("\nEnter five numbers:");

scanf("%d %d %d %d %d",&a,&b,&c,&d,&e);

average(a,b,c,d,e);

}

void average(int a, int b, int c, int d, int e)

{

    float avg;

    avg = (a+b+c+d+e)/5;

    printf("The average of given five numbers : %f",avg);

}

```

Output

```

Going to calculate the average of five numbers:
Enter five numbers:10
20
30
40
50

The average of given five numbers : 30.000000

```

Example for Function with argument and with return value

Example 1

```

#include<stdio.h>

int sum(int, int);

void main()

{

    int a,b,result;

```

```

printf("\nGoing to calculate the sum of two numbers:");

printf("\nEnter two numbers:");

scanf("%d %d",&a,&b);

result = sum(a,b);

printf("\nThe sum is : %d",result);

}

int sum(int a, int b)

{

    return a+b;

}

```

Output

```

Going to calculate the sum of two numbers:

Enter two numbers:10

20

The sum is : 30

```

Example 2: Program to check whether a number is even or odd

```

#include<stdio.h>

int even_odd(int);

void main()

{

    int n,flag=0;

    printf("\nGoing to check whether a number is even or odd");

    printf("\nEnter the number: ");

    scanf("%d",&n);

    flag = even_odd(n);

```

```

if(flag == 0)
{
    printf("\nThe number is odd");
}

else

{
    printf("\nThe number is even");
}
}

int even_odd(int n)
{
    if(n%2 == 0)
    {
        return 1;
    }

    else

    {
        return 0;
    }
}

```

Output

Going to check whether a number is even or odd

Enter the number: 100

The number is even

C Library Functions

Library functions are the inbuilt function in C that are grouped and placed at a common place called the library. Such functions are used to perform some specific operations. For example, printf is a library function used to print on the console. The library functions are created by the designers of compilers. All C standard library functions are defined inside the different header files saved with the extension .h. We need to include these header files in our program to make use of the library functions defined in such header files. For example, To use the library functions such as printf/scanf we need to include stdio.h in our program which is a header file that contains all the library functions regarding standard input/output.

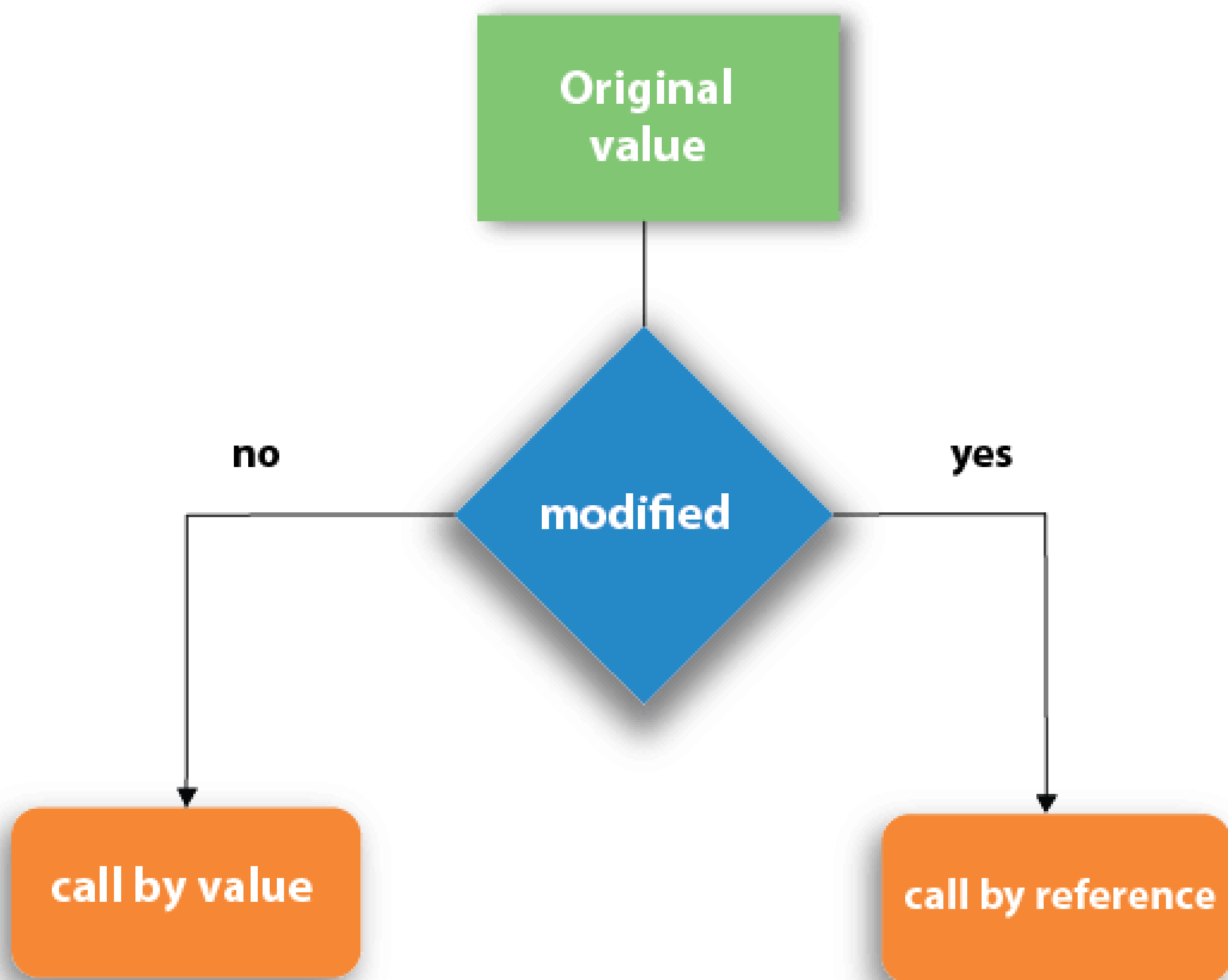
The list of mostly used header files is given in the following table.

SN	Header file	Description
1	stdio.h	This is a standard input/output header file. It contains all the library functions regarding standard input/output.
2	conio.h	This is a console input/output header file.
3	string.h	It contains all string related library functions like gets(), puts(),etc.
4	stdlib.h	This header file contains all the general library functions like malloc(), calloc(), exit(), etc.
5	math.h	This header file contains all the math operations related functions like sqrt(), pow(), etc.
6	time.h	This header file contains all the time-related functions.

7	ctype.h	This header file contains all character handling functions.
8	stdarg.h	Variable argument functions are defined in this header file.
9	signal.h	All the signal handling functions are defined in this header file.
10	setjmp.h	This file contains all the jump functions.
11	locale.h	This file contains locale functions.
12	errno.h	This file contains error handling functions.
13	assert.h	This file contains diagnostics functions.

Call by value and Call by reference in C

There are two methods to pass the data into the function in C language, i.e., *call by value* and *call by reference*.



Let's understand call by value and call by reference in c language one by one.

Call by value in C

- In call by value method, the value of the actual parameters is copied into the formal parameters. In other words, we can say that the value of the variable is used in the function call in the call by value method.
- In call by value method, we can not modify the value of the actual parameter by the formal parameter.
- In call by value, different memory is allocated for actual and formal parameters since the value of the actual parameter is copied into the formal parameter.

- The actual parameter is the argument which is used in the function call whereas formal parameter is the argument which is used in the function definition.

Let's try to understand the concept of call by value in c language by the example given below:

```
#include<stdio.h>

void change(int num) {

    printf("Before adding value inside function num=%d \n",num);

    num=num+100;

    printf("After adding value inside function num=%d \n", num);

}

int main() {

    int x=100;

    printf("Before function call x=%d \n", x);

    change(x); //passing value in function

    printf("After function call x=%d \n", x);

    return 0;

}
```

Output

```
Before function call x=100
Before adding value inside function num=100
After adding value inside function num=200
After function call x=100
```

Call by Value Example: Swapping the values of the two variables

```
#include <stdio.h>

void swap(int , int); //prototype of the function
```

```

int main()
{
    int a = 10;

    int b = 20;

    printf("Before swapping the values in main a = %d, b = %d\n",a,b); // printing the value of a and b in main

    swap(a,b);

    printf("After swapping values in main a = %d, b = %d\n",a,b); // The value of actual parameters do not change by changing the formal parameters in call by value, a = 10, b = 20
}

void swap (int a, int b)
{
    int temp;

    temp = a;

    a=b;

    b=temp;

    printf("After swapping values in function a = %d, b = %d\n",a,b); // Formal parameters, a = 20, b = 10
}

```

Output

Before swapping the values in main a = 10, b = 20

After swapping values in function a = 20, b = 10

After swapping values in main a = 10, b = 20

Call by reference in C

- In call by reference, the address of the variable is passed into the function call as the actual parameter.
- The value of the actual parameters can be modified by changing the formal parameters since the address of the actual parameters is passed.
- In call by reference, the memory allocation is similar for both formal parameters and actual parameters. All the operations in the function are performed on the value stored at the address of the actual parameters, and the modified value gets stored at the same address.

Consider the following example for the call by reference.

```
#include<stdio.h>

void change(int *num) {

    printf("Before adding value inside function num=%d \n",*num);

    (*num) += 100;

    printf("After adding value inside function num=%d \n", *num);

}

int main() {

    int x=100;

    printf("Before function call x=%d \n", x);

    change( &x); //passing reference in function

    printf("After function call x=%d \n", x);

    return 0;

}
```

Output

Before function call x=100

Before adding value inside function num=100

After adding value inside function num=200

After function call x=200

Call by reference Example: Swapping the values of the two variables

```
#include <stdio.h>

void swap(int *, int *); //prototype of the function

int main()
{
    int a = 10;

    int b = 20;

    printf("Before swapping the values in main a = %d, b = %d\n",a,b); // printing the value of a and b in main

    swap( &a,&b);

    printf("After swapping values in main a = %d, b = %d\n",a,b); // The values of actual parameters do change in call by reference, a = 10, b = 20
}

void swap (int *a, int *b)
{
    int temp;

    temp = *a;

    *a=*b;

    *b=temp;

    printf("After swapping values in function a = %d, b = %d\n",*a,*b); // Formal parameters, a = 20, b = 10
}
```

Output

Before swapping the values in main a = 10, b = 20

After swapping values in function a = 20, b = 10

After swapping values in main a = 20, b = 10

Difference between call by value and call by reference in c

No.	Call by value	Call by reference
1	A copy of the value is passed into the function	An address of value is passed into the function
2	Changes made inside the function is limited to the function only. The values of the actual parameters do not change by changing the formal parameters.	Changes made inside the function validate outside of the function also. The values of the actual parameters do change by changing the formal parameters.
3	Actual and formal arguments are created at the different memory location	Actual and formal arguments are created at the same memory location

Recursion in C

Recursion is the process which comes into existence when a function calls a copy of itself to work on a smaller problem. Any function which calls itself is called recursive function, and such function calls are called recursive calls. Recursion involves several numbers of recursive calls. However, it is important to impose a termination condition of recursion. Recursion code is shorter than iterative code however it is difficult to understand.

Recursion cannot be applied to all the problem, but it is more useful for the tasks that can be defined in terms of similar subtasks. For Example, recursion may be applied to sorting, searching, and traversal problems.

Generally, iterative solutions are more efficient than recursion since function call is always overhead. Any problem that can be solved recursively, can also be solved iteratively. However, some problems are best suited to be solved by the recursion, for example, tower of Hanoi, Fibonacci series, factorial finding, etc.

In the following example, recursion is used to calculate the factorial of a number.

```
#include <stdio.h>

int fact (int);

int main()
{
    int n,f;

    printf("Enter the number whose factorial you want to calculate?");

    scanf("%d",&n);

    f = fact(n);

    printf("factorial = %d",f);
}

int fact(int n)
{
    if (n==0)
    {
        return 0;
    }

    else if ( n == 1)
    {
```

```

        return 1;
    }

    else
    {
        return n*fact(n-1);
    }
}

```

Output

```

Enter the number whose factorial you want to calculate?5
factorial = 120

```

We can understand the above program of the recursive method call by the figure given below:

```

return 5 * factorial(4) = 120
└─ return 4 * factorial(3) = 24
    └─ return 3 * factorial(2) = 6
        └─ return 2 * factorial(1) = 2
            └─ return 1 * factorial(0) = 1

```

javaTpoint.com

$1 * 2 * 3 * 4 * 5 = 120$

Fig: Recursion

Recursive Function

A recursive function performs the tasks by dividing it into the subtasks. There is a termination condition defined in the function which is satisfied by some specific subtask. After this, the recursion stops and the final result is returned from the function.

The case at which the function doesn't recur is called the base case whereas the instances where the function keeps calling itself to perform a subtask, is called the recursive case. All the recursive functions can be written using this format.

Pseudocode for writing any recursive function is given below.

```
if (test_for_base)
{
    return some_value;
}

else if (test_for_another_base)
{
    return some_another_value;
}

else
{
    // Statements;

    recursive call;
}
```

Example of recursion in C

Let's see an example to find the nth term of the Fibonacci series.

```
#include<stdio.h>

int fibonacci(int);
```

```

void main ( )
{
    int n,f;

    printf("Enter the value of n?");

    scanf("%d",&n);

    f = fibonacci(n);

    printf("%d",f);
}

int fibonacci (int n)
{
    if (n==0)
    {
        return 0;
    }

    else if (n == 1)
    {
        return 1;
    }

    else
    {
        return fibonacci(n-1)+fibonacci(n-2);
    }
}

```

Output

Enter the value of n?12

Memory allocation of Recursive method

Each recursive call creates a new copy of that method in the memory. Once some data is returned by the method, the copy is removed from the memory. Since all the variables and other stuff declared inside function get stored in the stack, therefore a separate stack is maintained at each recursive call. Once the value is returned from the corresponding function, the stack gets destroyed. Recursion involves so much complexity in resolving and tracking the values at each recursive call. Therefore we need to maintain the stack and track the values of the variables defined in the stack.

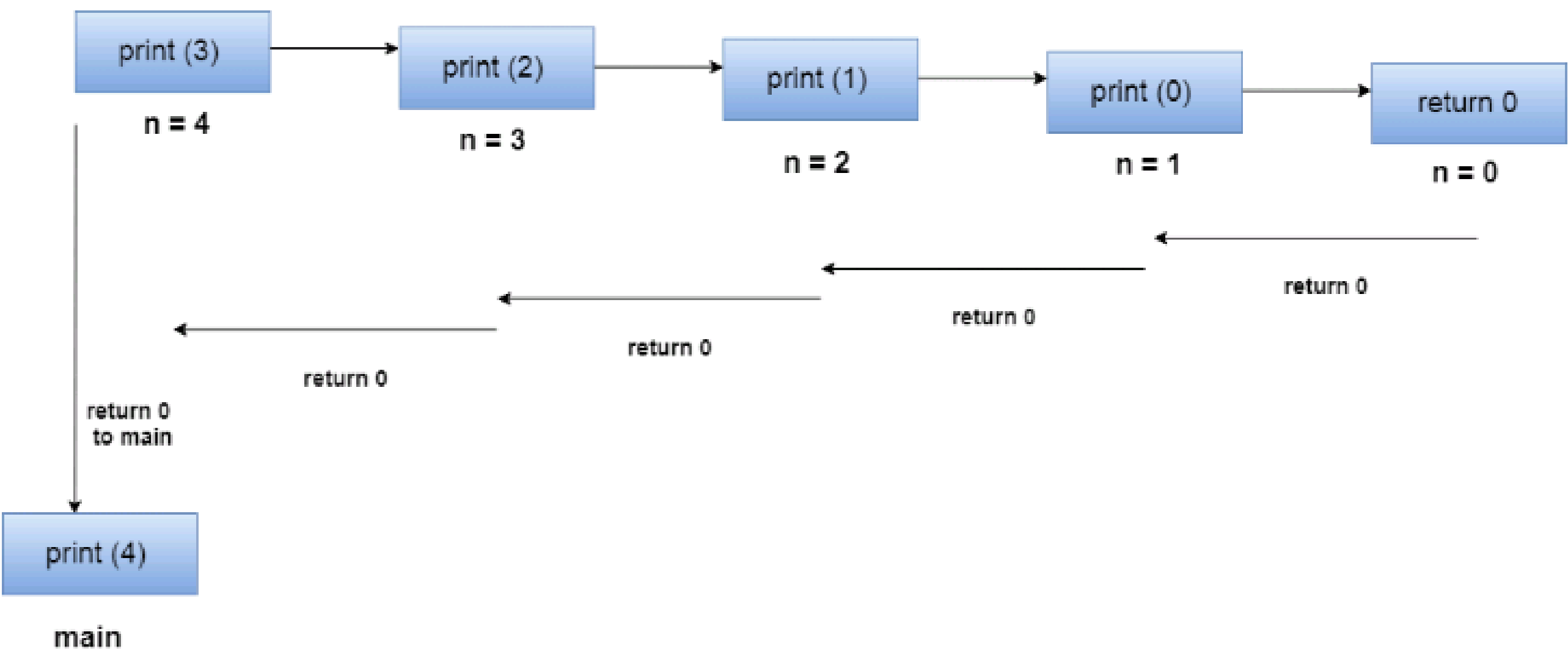
Let us consider the following example to understand the memory allocation of the recursive functions.

```
int display (int n)
{
    if(n == 0)
        return 0; // terminating condition
    else
    {
        printf("%d",n);
        return display(n-1); // recursive call
    }
}
```

Explanation

Let us examine this recursive function for $n = 4$. First, all the stacks are maintained which prints the corresponding value of n until n becomes 0, Once the termination condition is reached, the stacks get destroyed one by one by returning 0 to its calling

stack. Consider the following image for more information regarding the stack trace for the recursive functions.



Stack tracing for recursive function call

Storage Classes in C

Storage classes in C are used to determine the lifetime, visibility, memory location, and initial value of a variable. There are four types of storage classes in C

- Automatic
- External
- Static
- Register

Storage Classes	Storage Place	Default Value	Scope	Lifetime
auto	RAM	Garbage Value	Local	Within function

extern	RAM	Zero	Global	Till the end of the main program Maybe declared anywhere in the program
static	RAM	Zero	Local	Till the end of the main program, Retains value between multiple functions call
register	Register	Garbage Value	Local	Within the function

Automatic

- Automatic variables are allocated memory automatically at runtime.
- The visibility of the automatic variables is limited to the block in which they are defined.
- The scope of the automatic variables is limited to the block in which they are defined.
- The automatic variables are initialized to garbage by default.

- The memory assigned to automatic variables gets freed upon exiting from the block.
- The keyword used for defining automatic variables is `auto`.
- Every local variable is automatic in C by default.

Example 1

```
#include <stdio.h>

int main()
{
    int a; //auto
    char b;
    float c;

    printf("%d %c %f",a,b,c); // printing initial default value of automatic variables a, b, and c.

    return 0;
}
```

Output:

garbage garbage garbage

Example 2

```
#include <stdio.h>

int main()
{
    int a = 10,i;

    printf("%d ",++a);

    {
        int a = 20;
```

```

for (i=0;i<3;i++)
{
    printf("%d ",a); // 20 will be printed 3 times since it is the local value of a
}

}

printf("%d ",a); // 11 will be printed since the scope of a = 20 is ended.
}

```

Output:

11 20 20 20 11

Static

- The variables defined as static specifier can hold their value between the multiple function calls.
- Static local variables are visible only to the function or the block in which they are defined.
- A same static variable can be declared many times but can be assigned at only one time.
- Default initial value of the static integral variable is 0 otherwise null.
- The visibility of the static global variable is limited to the file in which it has declared.
- The keyword used to define static variable is static.

Example 1

```

#include<stdio.h>

static char c;

```

```

static int i;

static float f;

static char s[100];

void main ( )

{

printf("%d %d %f %s",c,i,f); // the initial default value of c, i, and f will be printed.

}

```

Output:

```

0 0 0.000000 (null)

```

Example 2

```

#include<stdio.h>

void sum()

{

static int a = 10;

static int b = 24;

printf("%d %d \n",a,b);

a++;

b++;

}

void main()

{

int i;

for(i = 0; i< 3; i++)

{

sum(); // The static variables holds their value between multiple function calls.

```

```
}  
  
}
```

Output:

```
10 24  
  
11 25  
  
12 26
```

Register

- The variables defined as the register is allocated the memory into the CPU registers depending upon the size of the memory remaining in the CPU.
- We can not dereference the register variables, i.e., we can not use &operator for the register variable.
- The access time of the register variables is faster than the automatic variables.
- The initial default value of the register local variables is 0.
- The register keyword is used for the variable which should be stored in the CPU register. However, it is compiler's choice whether or not; the variables can be stored in the register.
- We can store pointers into the register, i.e., a register can store the address of a variable.
- Static variables can not be stored into the register since we can not use more than one storage specifier for the same variable.

Example 1

```
#include <stdio.h>  
  
int main()
```

```

{

register int a; // variable a is allocated memory in the CPU register. The initial default value of a is 0.

printf("%d",a);

}

```

Output:

0

Example 2

```

#include <stdio.h>

int main()

{

register int a = 0;

printf("%u",&a); // This will give a compile time error since we can not access the address of a register variable.

}

```

Output:

```

main.c:5:5: error: address of register variable 'a' requested
printf("%u",&a);
      ^~~~~~

```

External

- The external storage class is used to tell the compiler that the variable defined as extern is declared with an external linkage elsewhere in the program.
- The variables declared as extern are not allocated any memory. It is only declaration and intended to specify that the variable is declared elsewhere in the program.

- The default initial value of external integral type is 0 otherwise null.
- We can only initialize the extern variable globally, i.e., we can not initialize the external variable within any block or method.
- An external variable can be declared many times but can be initialized at only once.
- If a variable is declared as external then the compiler searches for that variable to be initialized somewhere in the program which may be extern or static. If it is not, then the compiler will show an error.

Example 1

```
#include <stdio.h>

int main()

{

extern int a;

printf("%d",a);

}
```

Output

```
main.c:(.text+0x6): undefined reference to `a'
collect2: error: ld returned 1 exit status
```

Example 2

```
#include <stdio.h>

int a;

int main()

{

extern int a; // variable a is defined globally, the memory will not be allocated to a
```

```
printf("%d",a);  
}
```

Output

0

Example 3

```
#include <stdio.h>  
  
int a;  
  
int main()  
{  
  
    extern int a = 0; // this will show a compiler error since we can not use extern and initializer at  
    same time  
  
    printf("%d",a);  
}
```

Output

```
compile time error  
main.c: In function ?main?:  
main.c:5:16: error: ?a? has both ?extern? and initializer  
extern int a = 0;
```

Example 4

```
#include <stdio.h>  
  
int main()  
{  
  
    extern int a; // Compiler will search here for a variable a defined and initialized somewhere in  
    the pogram or not.  
  
    printf("%d",a);  
}
```

```
}  
  
int a = 20;
```

Output

20

Example 5

```
extern int a;  
  
int a = 10;  
  
#include <stdio.h>  
  
int main()  
{  
  
printf("%d",a);  
  
}  
  
int a = 20; // compiler will show an error at this line
```

Output

compile time error

C Array

1-D Array in C

An array is defined as the collection of similar type of data items stored at contiguous memory locations. Arrays are the derived data type in C programming language which can store the primitive type of data such as int, char, double, float, etc. It also has the capability to store the collection of derived data types, such as pointers, structure, etc. The array is the simplest data structure where each data element can be randomly accessed by using its index number.

C array is beneficial if you have to store similar elements. For example, if we want to store the marks of a student in 6 subjects, then we don't need to define different variables for the marks in the different subject. Instead of that, we can define an array which can store the marks in each subject at the contiguous memory locations.

By using the array, we can access the elements easily. Only a few lines of code are required to access the elements of the array.

Properties of Array

The array contains the following properties.

- Each element of an array is of same data type and carries the same size, i.e., int = 4 bytes.
- Elements of the array are stored at contiguous memory locations where the first element is stored at the smallest memory location.
- Elements of the array can be randomly accessed since we can calculate the address of each element of the array with the given base address and the size of the data element.

Advantage of C Array

- 1) **Code Optimization:** Less code to access the data.
- 2) **Ease of traversing:** By using the for loop, we can retrieve the elements of an array easily.
- 3) **Ease of sorting:** To sort the elements of the array, we need a few lines of code only.
- 4) **Random Access:** We can access any element randomly using the array.

Disadvantage of C Array

- 1) **Fixed Size:** Whatever size, we define at the time of declaration of the array, we can't exceed the limit. So, it doesn't grow the size dynamically like LinkedList which we will learn later.

Declaration of C Array

We can declare an array in the c language in the following way.

```
data_type array_name[array_size];
```

Now, let us see the example to declare the array.

```
int marks[5];
```

Here, int is the *data_type*, marks are the *array_name*, and 5 is the *array_size*.

Initialization of C Array

The simplest way to initialize an array is by using the index of each element. We can initialize each element of the array by using the index. Consider the following example.

```
marks[0]=80; // initialization of array
```

```
marks[1]=60;
```

```
marks[2]=70;
```

```
marks[3]=85;
```

```
marks[4]=75;
```

80	60	70	85	75
marks[0]	marks[1]	marks[2]	marks[3]	marks[4]

Initialization of Array

C array example

```
#include<stdio.h>
```

```
int main(){

int i=0;

int marks[5]; //declaration of array

marks[0]=80; //initialization of array

marks[1]=60;

marks[2]=70;

marks[3]=85;

marks[4]=75;

//traversal of array

for(i=0;i<5;i++){

printf("%d \n",marks[i]);

} //end of for loop

return 0;

}
```

Output

```
80
60
70
85
75
```

C Array: Declaration with Initialization

We can initialize the c array at the time of declaration. Let's see the code.

```
int marks[5]={20,30,40,50,60};
```

In such case, there is **no requirement to define the size**. So it may also be written as the following code.

```
int marks[]={20,30,40,50,60};
```

Let's see the C program to declare and initialize the array in C.

```
#include<stdio.h>

int main(){

int i=0;

int marks[5]={20,30,40,50,60}; //declaration and initialization of array

//traversal of array

for(i=0;i<5;i++){

printf("%d \n",marks[i]);

}

return 0;

}
```

Output

```
20
30
40
50
60
```

C Array Example: Sorting an array

In the following program, we are using bubble sort method to sort the array in ascending order.

```
#include<stdio.h>

void main ( )

{

int i, j,temp;
```

```

int a[10] = { 10, 9, 7, 101, 23, 44, 12, 78, 34, 23};

for(i = 0; i<10; i++)
{
    for(j = i+1; j<10; j++)
    {
        if(a[j] > a[i])
        {
            temp = a[i];
            a[i] = a[j];
            a[j] = temp;
        }
    }
}

printf("Printing Sorted Element List ...\n");

for(i = 0; i<10; i++)
{
    printf("%d\n",a[i]);
}
}

```

Program to print the largest and second largest element of the array.

```

#include<stdio.h>

void main ( )

{

    int arr[100],i,n,largest,sec_largest;

    printf("Enter the size of the array?");

```

```

scanf("%d",&n);

printf("Enter the elements of the array?");

for(i = 0; i<n; i++)

{

    scanf("%d",&arr[i]);

}

largest = arr[0];

sec_largest = arr[1];

for(i=0;i<n;i++)

{

    if(arr[i]>largest)

    {

        sec_largest = largest;

        largest = arr[i];

    }

    else if (arr[i]>sec_largest && arr[i]!=largest)

    {

        sec_largest=arr[i];

    }

}

printf("largest = %d, second largest = %d",largest,sec_largest);

}

```

2-D Array in C

The two-dimensional array can be defined as an array of arrays. The 2D array is organized as matrices which can be represented as the collection of rows and columns. However, 2D arrays are created to implement a relational database lookalike data structure. It provides ease of holding the bulk of data at once which can be passed to any number of functions wherever required.

Declaration of two dimensional Array in C

The syntax to declare the 2D array is given below.

```
data_type array_name[rows][columns];
```

Consider the following example.

```
int twodimen[4][3];
```

Here, 4 is the number of rows, and 3 is the number of columns.

Initialization of 2D Array in C

In the 1D array, we don't need to specify the size of the array if the declaration and initialization are being done simultaneously. However, this will not work with 2D arrays. We will have to define at least the second dimension of the array. The two-dimensional array can be declared and defined in the following way.

```
int arr[4][3]={{1,2,3},{2,3,4},{3,4,5},{4,5,6}};
```

Two-dimensional array example in C

```
#include<stdio.h>

int main(){

int i=0,j=0;

int arr[4][3]={{1,2,3},{2,3,4},{3,4,5},{4,5,6}};

//traversing 2D array
```

```

for(i=0;i<4;i++){

    for(j=0;j<3;j++){

        printf("arr[%d] [%d] = %d \n",i,j,arr[i][j]);

    } //end of j

} //end of i

return 0;

}

```

Output

```

arr[0][0] = 1
arr[0][1] = 2
arr[0][2] = 3
arr[1][0] = 2
arr[1][1] = 3
arr[1][2] = 4
arr[2][0] = 3
arr[2][1] = 4
arr[2][2] = 5
arr[3][0] = 4
arr[3][1] = 5
arr[3][2] = 6

```

C 2D array example: Storing elements in a matrix and printing it.

```

#include <stdio.h>

void main ()

{

    int arr[3][3],i,j;

```

```

for (i=0;i<3;i++)
{
    for (j=0;j<3;j++)
    {
        printf("Enter a[ %d][ %d]: ",i,j);

        scanf("%d",&arr[i][j]);
    }
}

printf("\n printing the elements ....\n");

for(i=0;i<3;i++)
{
    printf("\n");

    for (j=0;j<3;j++)
    {
        printf("%d\t",arr[i][j]);
    }
}
}

```

Output

```

Enter a[0][0]: 56
Enter a[0][1]: 10
Enter a[0][2]: 30
Enter a[1][0]: 34
Enter a[1][1]: 21
Enter a[1][2]: 34

```

Enter a[2][0]: 45

Enter a[2][1]: 56

Enter a[2][2]: 78

printing the elements

56	10	30
34	21	34
45	56	78

Return an Array in C

What is an Array?

An array is a type of data structure that stores a fixed-size of a homogeneous collection of data. In short, we can say that array is a collection of variables of the same type.

For example, if we want to declare 'n' number of variables, n1, n2...n., if we create all these variables individually, then it becomes a very tedious task. In such a case, we create an array of variables having the same type. Each element of an array can be accessed using an index of the element.

Let's first see how to pass a single-dimensional array to a function.

Passing array to a function

```
#include <stdio.h>

void getarray(int arr[])

{

    printf("Elements of array are : ");
```

```

    for(int i=0;i<5;i++)
    {
        printf("%d ", arr[i]);
    }
}

int main()
{
    int arr[5]={45,67,34,78,90};

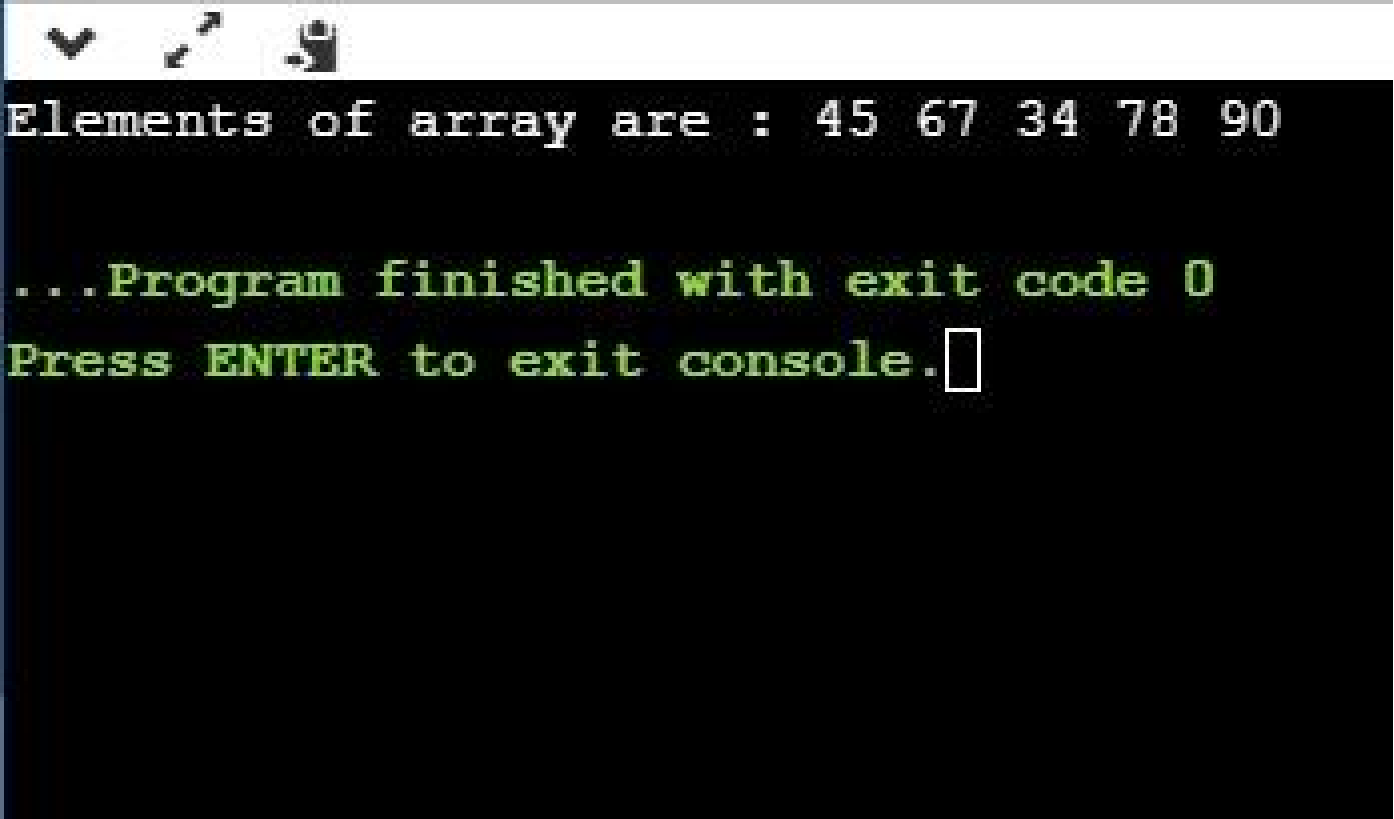
    getarray(arr);

    return 0;
}

```

In the above program, we have first created the array `arr[]` and then we pass this array to the function `getarray()`. The `getarray()` function prints all the elements of the array `arr[]`.

Output



```

▼ ↗ 📄
Elements of array are : 45 67 34 78 90

...Program finished with exit code 0
Press ENTER to exit console.

```

Passing array to a function as a pointer

Now, we will see how to pass an array to a function as a pointer.

```
#include <stdio.h>
```

```

void printarray(char *arr)
{
    printf("Elements of array are : ");

    for(int i=0;i<5;i++)
    {
        printf("%c ", arr[i]);
    }
}

int main()
{
    char arr[5]={'A','B','C','D','E'};

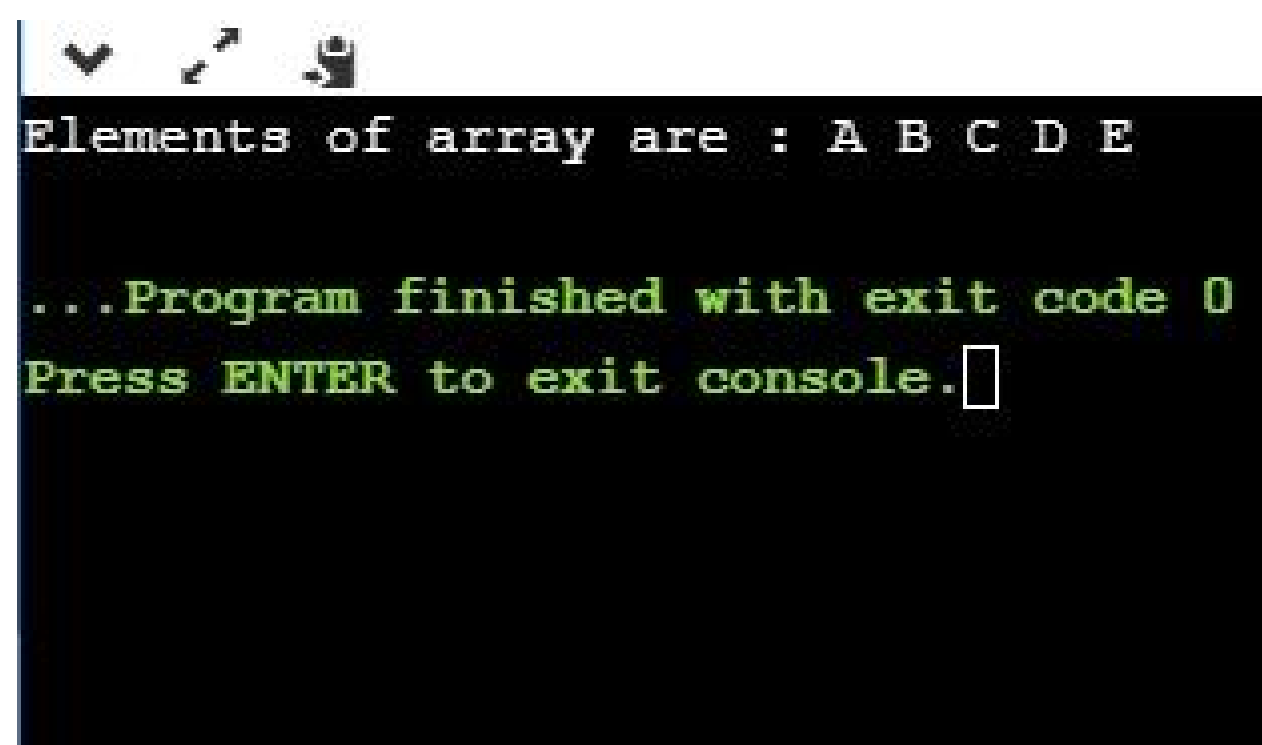
    printarray(arr);

    return 0;
}

```

In the above code, we have passed the array to the function as a pointer. The function `printarray()` prints the elements of an array.

Output



```

Elements of array are : A B C D E

...Program finished with exit code 0
Press ENTER to exit console.

```

Note: From the above examples, we observe that array is passed to a function as a reference which means that array also persist outside the function.

How to return an array from a function

Returning pointer pointing to the array

```
#include <stdio.h>

int *getarray()

{

    int arr[5];

    printf("Enter the elements in an array : ");

    for(int i=0;i<5;i++)

    {

        scanf("%d", &arr[i]);

    }

    return arr;

}

int main()

{

    int *n;

    n=getarray();

    printf("\nElements of array are :");

    for(int i=0;i<5;i++)

    {

        printf("%d", n[i]);

    }

    return 0;

}
```

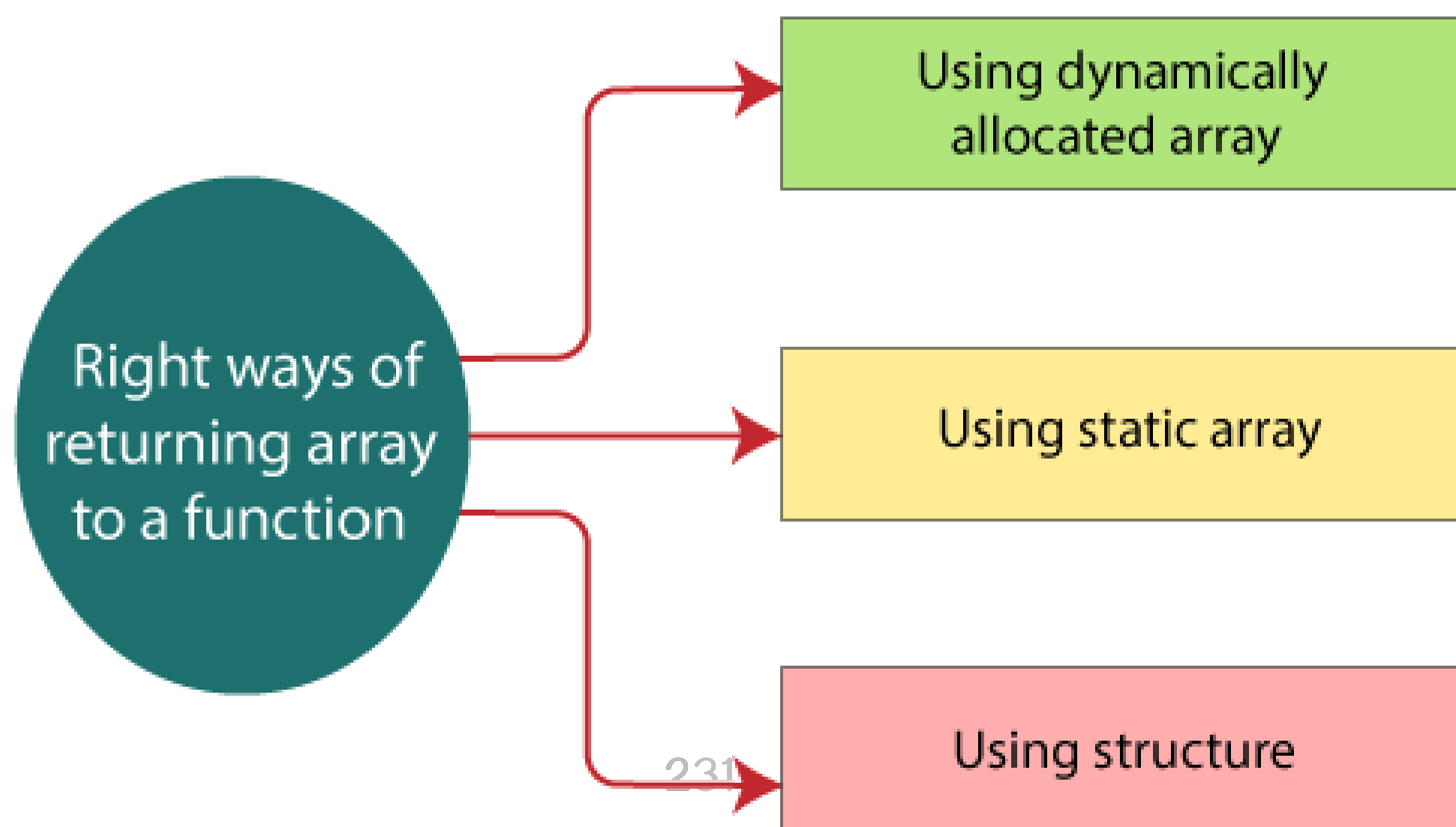
In the above program, `getarray()` function returns a variable 'arr'. It returns a local variable, but it is an illegal memory location to be returned, which is allocated within a function in the stack. Since the program control comes back to the `main()` function, and all the variables in a stack are freed. Therefore, we can say that this program is returning memory location, which is already de-allocated, so the output of the program is a **segmentation fault**.

Output

```
input
main.c:27:12: warning: function returns address of local variable [-Wreturn-local-addr]
Array inside function: 1
2
3
4
5
Array outside function:
Segmentation fault (core dumped)
```

There are three right ways of returning an array to a function:

- Using dynamically allocated array
- Using static array
- Using structure



Returning array by passing an array which is to be returned as a parameter to the function.

```
#include <stdio.h>

int *getarray(int *a)

{

    printf("Enter the elements in an array : ");

    for(int i=0;i<5;i++)

    {

        scanf("%d", &a[i]);

    }

    return a;

}

int main()

{

    int *n;

    int a[5];

    n=getarray(a);

    printf("\nElements of array are :");

    for(int i=0;i<5;i++)

    {

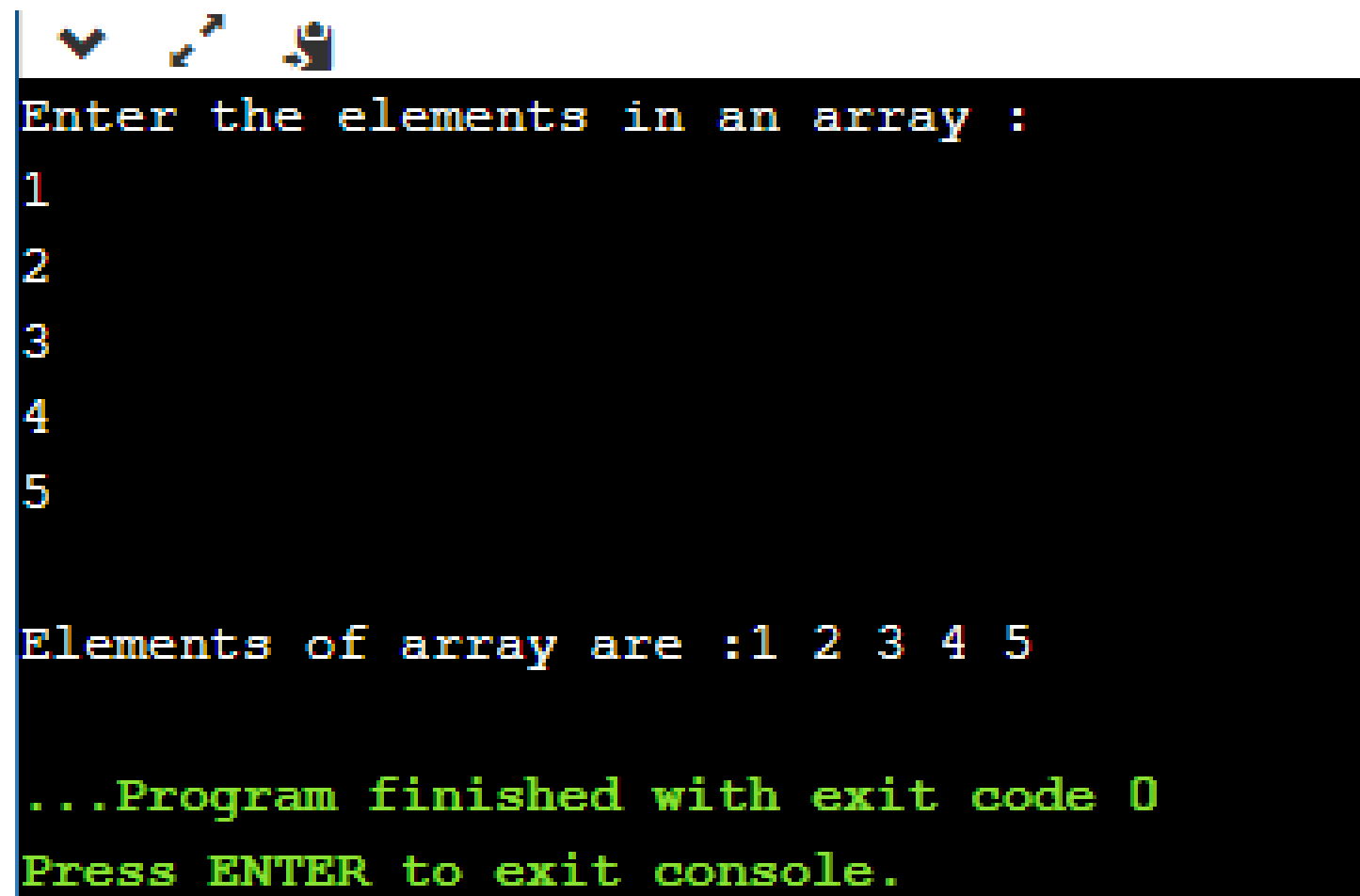
        printf("%d", n[i]);

    }

    return 0;

}
```

Output



```
Enter the elements in an array :
1
2
3
4
5

Elements of array are :1 2 3 4 5

...Program finished with exit code 0
Press ENTER to exit console.
```

Returning array using malloc() function.

```
#include <stdio.h>

#include<malloc.h>

int *getarray()

{

    int size;

    printf("Enter the size of the array : ");

    scanf("%d",&size);

    int *p= malloc(sizeof(size));

    printf("\nEnter the elements in an array");

    for(int i=0;i<size;i++)

    {

        scanf("%d",&p[i]);

    }

    return p;

}
```

```

int main()

{

    int *ptr;

    ptr=getarray();

    int length=sizeof(*ptr);

    printf("Elements that you have entered are : ");

    for(int i=0;ptr[i]!='\0';i++)

    {

        printf("%d ", ptr[i]);

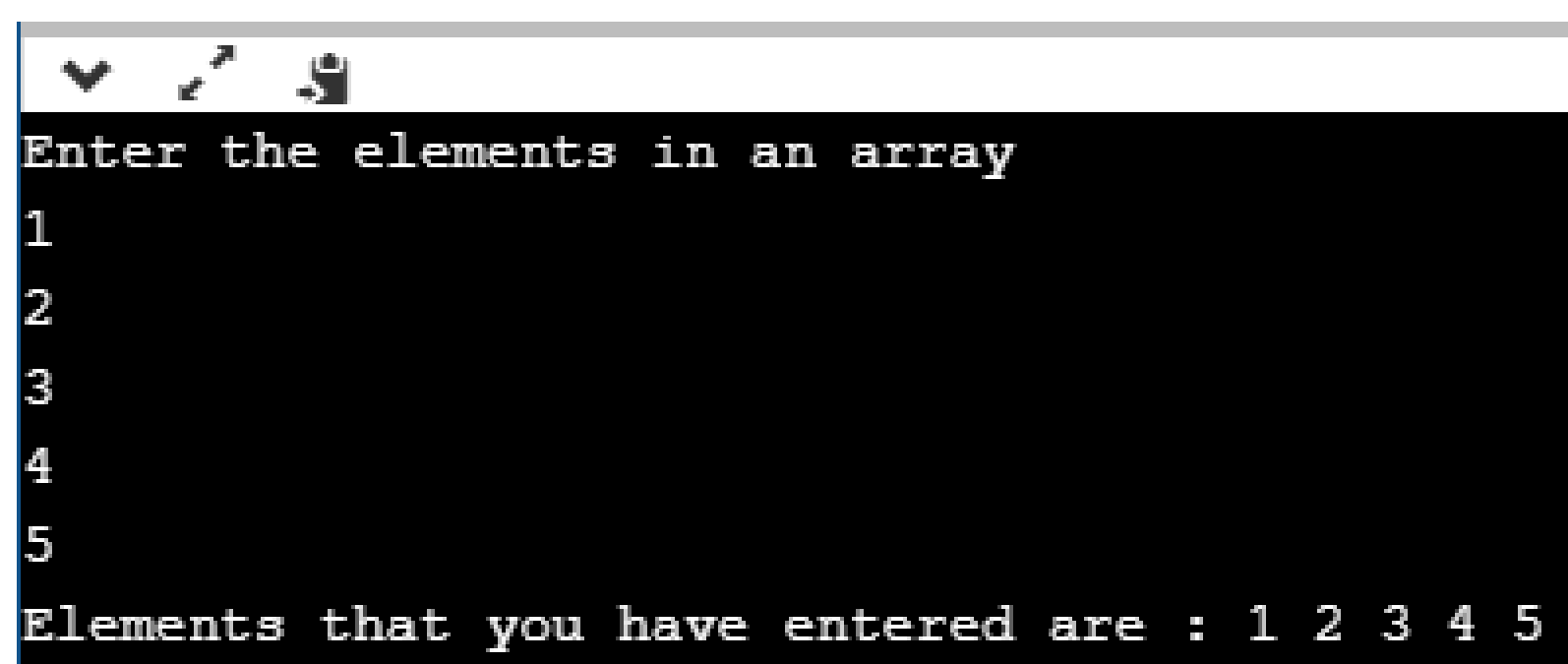
    }

    return 0;

}

```

Output



```

Enter the elements in an array
1
2
3
4
5
Elements that you have entered are : 1 2 3 4 5

```

Using Static Variable

```

#include <stdio.h>

int *getarray()

{

    static int arr[7];

    printf("Enter the elements in an array : ");

```

```

    for(int i=0;i<7;i++)

    {

        scanf("%d",&arr[i]);

    }

    return arr;

}

int main()

{

    int * ptr;

    ptr=getarray();

    printf("\nElements that you have entered are :");

    for(int i=0;i<7;i++)

    {

        printf("%d ", ptr[i]);

    }

}

```

In the above code, we have created the variable `arr[]` as static in `getarray()` function, which is available throughout the program. Therefore, the function `getarray()` returns the actual memory location of the variable 'arr'.

Output

```
> ./main
Enter the elements in an array :
10
20
30
40
50
60
70

Elements that you have entered are :10 20 30 40 50 60 70
```

Using Structure

The structure is a user-defined data type that can contain a collection of items of different types. Now, we will create a program that returns an array by using structure.

```
#include <stdio.h>

#include<malloc.h>

struct array

{

    int arr[8];

};

struct array getarray( )

{

    struct array y;

    printf("Enter the elements in an array : ");

    for(int i=0;i<8;i++)

    {

        scanf("%d",&y.arr[i]);

    }

    return y;
```

```

}

int main()

{

    struct array x=getarray();

    printf("Elements that you have entered are :");

    for(int i=0;x.arr[i]!='\0';i++)

    {

        printf("%d ", x.arr[i]);

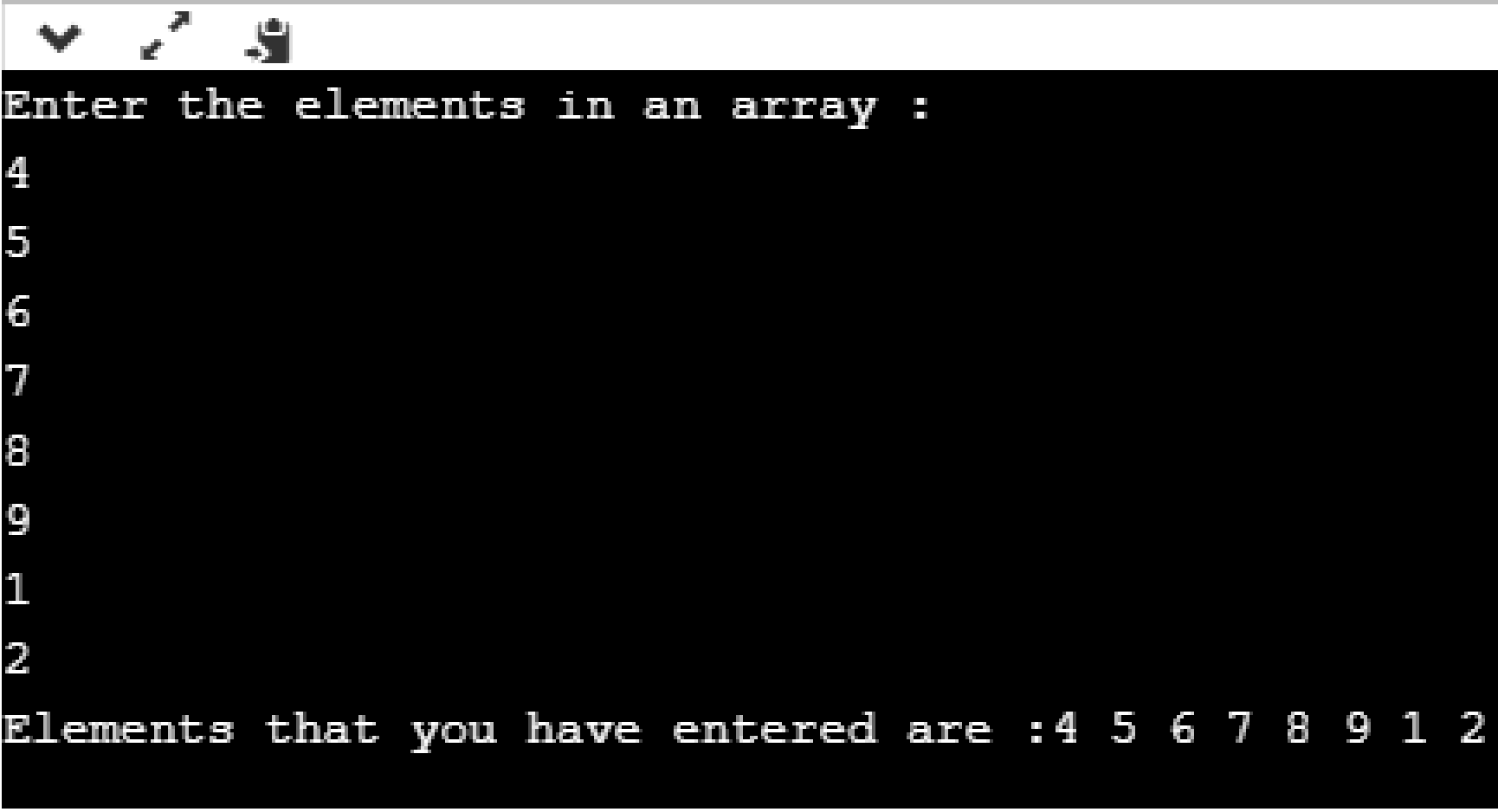
    }

    return 0;

}

```

Output



```

Enter the elements in an array :
4
5
6
7
8
9
1
2
Elements that you have entered are :4 5 6 7 8 9 1 2

```

Passing Array to Function in C

In C, there are various general problems which requires passing more than one variable of the same type to a function. For example, consider a function which sorts the 10 elements in ascending order. Such a function requires 10 numbers to be passed as the actual parameters from the main function. Here, instead of declaring 10

different numbers and then passing into the function, we can declare and initialize an array and pass that into the function. This will resolve all the complexity since the function will now work for any number of values.

As we know that the `array_name` contains the address of the first element. Here, we must notice that we need to pass only the name of the array in the function which is intended to accept an array. The array defined as the formal parameter will automatically refer to the array specified by the array name defined as an actual parameter.

Consider the following syntax to pass an array to the function.

```
functionname(arrayname); // passing array
```

Methods to declare a function that receives an array as an argument

There are 3 ways to declare the function which is intended to receive an array as an argument.

First way:

```
return_type function(type arrayname[])
```

Declaring blank subscript notation `[]` is the widely used technique.

Second way:

```
return_type function(type arrayname[SIZE])
```

Optionally, we can define size in subscript notation `[]`.

Third way:

```
return_type function(type *arrayname)
```

You can also use the concept of a pointer. In pointer chapter, we will learn about it.

C language passing an array to function example

```

#include<stdio.h>

int minarray(int arr[],int size){

int min=arr[0];

int i=0;

for(i=1;i<size;i++){

if(min>arr[i]){

min=arr[i];

}

} //end of for

return min;

} //end of function

int main(){

int i=0,min=0;

int numbers[]={4,5,7,3,8,9}; //declaration of array

min=minarray(numbers,6); //passing array with size

printf("minimum number is %d \n",min);

return 0;

}

```

Output

minimum number is 3

C function to sort the array

```

#include<stdio.h>

void Bubble_Sort(int[]);

void main ( )

{

```

```

    int arr[10] = { 10, 9, 7, 101, 23, 44, 12, 78, 34, 23};

    Bubble_Sort(arr);

}

void Bubble_Sort(int a[]) //array a[] points to arr.

{

    int i, j, temp;

    for(i = 0; i<10; i++)

    {

        for(j = i+1; j<10; j++)

        {

            if(a[j] < a[i])

            {

                temp = a[i];

                a[i] = a[j];

                a[j] = temp;

            }

        }

    }

    printf("Printing Sorted Element List ...\n");

    for(i = 0; i<10; i++)

    {

        printf("%d\n",a[i]);

    }

}

```

Output

Printing Sorted Element List ...

7

9

10

12

23

23

34

44

78

101

Returning array from the function

As we know that, a function can not return more than one value. However, if we try to write the return statement as `return a, b, c;` to return three values (a,b,c), the function will return the last mentioned value which is c in our case. In some problems, we may need to return multiple values from a function. In such cases, an array is returned from the function.

Returning an array is similar to passing the array into the function. The name of the array is returned from the function. To make a function returning an array, the following syntax is used.

```
int * Function_name() {  
  
    //some statements;  
  
    return array_type;  
  
}
```

To store the array returned from the function, we can define a pointer which points to that array. We can traverse the array by increasing that pointer since pointer initially

points to the base address of the array. Consider the following example that contains a function returning the sorted array.

```
#include<stdio.h>

int* Bubble_Sort(int[]);

void main ()
{
    int arr[10] = { 10, 9, 7, 101, 23, 44, 12, 78, 34, 23};

    int *p = Bubble_Sort(arr), i;

    printf("printing sorted elements ...\n");

    for(i=0;i<10;i++)
    {
        printf("%d\n", *(p+i));
    }
}

int* Bubble_Sort(int a[]) //array a[] points to arr.
{
    int i, j, temp;

    for(i = 0; i<10; i++)
    {
        for(j = i+1; j<10; j++)
        {
            if(a[j] < a[i])
            {
                temp = a[i];
                a[i] = a[j];
                a[j] = temp;
            }
        }
    }
    return a;
}
```

```
        a[j] = temp;
    }
}

return a;
}
```

Output

Printing Sorted Element List ...

7

9

10

12

23

23

34

44

78

101